

Управление данными

Файловые системы

Файловые системы – это набор программ, которые выполняют для пользователей некоторые операции (например, создание отчета). Каждая программа определяет свои собственные данные и управляет ими.

Ограничения, файловых систем:

- ☐ разделение и изоляция данных.
- ☐ дублирование данных.
- ☐ зависимость от данных.
- ☐ несовместимость файлов.
- ☐ фиксированные запросы и быстрое увеличение количества приложений.

Ограничения файловых систем являются следствием двух факторов:

1. *Определение данных содержится внутри приложений, а не хранится отдельно и независимо от них.*
2. *Помимо приложений не предусмотрено никаких других инструментов доступа к данным и их обработки.*

Системы баз данных (БД)

База данных – это совместно используемый набор логически связанных данных (и описание этих данных), предназначенный для удовлетворения информационных потребностей организации.

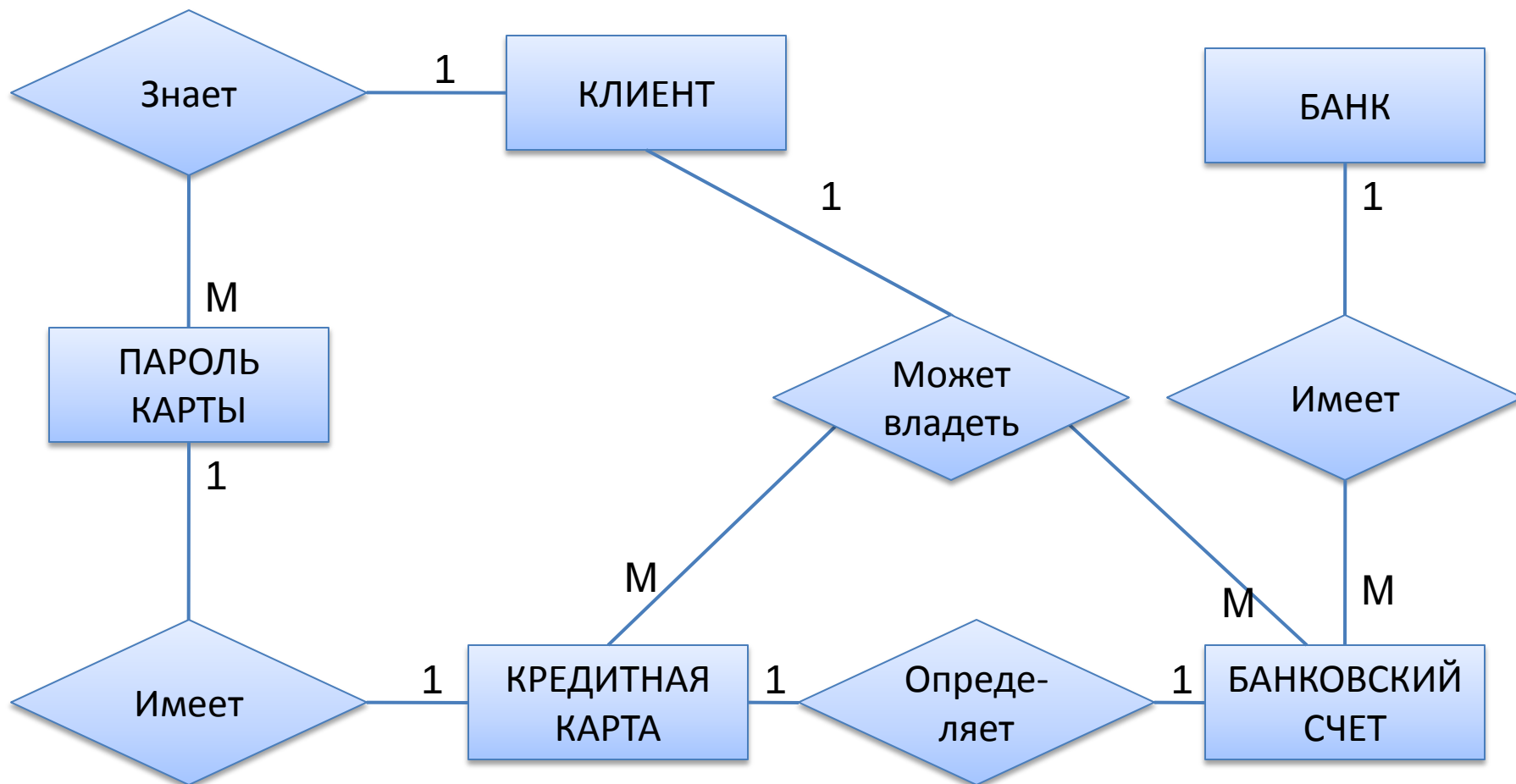
Основные понятия

Сущность (entity) – отдельный тип объекта организации (человек, место или вещь, понятие или событие), который необходимо представить в базе данных.

Атрибут (attribute) – свойство, которое описывает некоторую характеристику описываемого объекта.

Связь (relationship) – это объединение нескольких сущностей.

Диаграмма «сущность – связь»

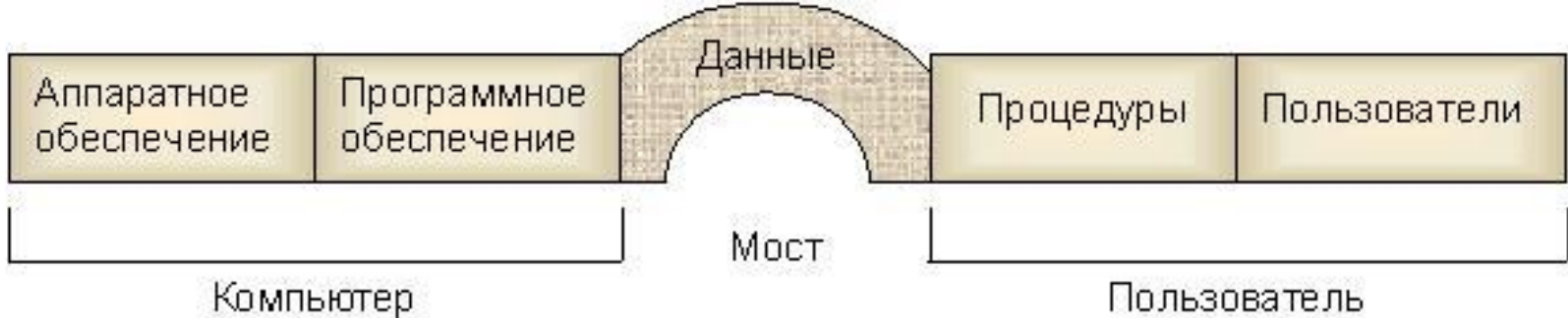


Системы баз данных (СУБД)

СУБД (система управления базами данных) – является программным обеспечением, которое взаимодействует с прикладными программами пользователей и базой данных и обладает следующими возможностями:

- позволяет определять базу данных с помощью **языка определения данных** (DDL – Data Definition Language).
- позволяет вставлять, обновлять, удалять и извлекать информацию из базы данных с помощью **языка управления данными** (DML – Data Manipulation Language).
- предоставляет контролируемый доступ к базе данных с помощью следующих средств:
 - ☐ системы обеспечения безопасности;
 - ☐ системы поддержки целостности данных;
 - ☐ системы управления параллельной работой приложений;
 - ☐ системы восстановления;
 - ☐ доступного пользователям каталога, содержащего описание хранимой в базе данных информации.

Компоненты среды СУБД



Процедуры

- ☐ регистрация в СУБД;
- ☐ использование отдельного инструмента СУБД или приложения;
- ☐ запуск и останов СУБД;
- ☐ создание резервных копий СУБД;
- ☐ обработка сбоев аппаратного и программного обеспечения;
- ☐ изменение структуры таблиц, реорганизация базы данных;
- ☐ способы улучшения производительности и методы архивирования данных на вторичных устройствах хранения.

Пользователи

- | | |
|---|--|
| ☐ Администратор данных | ☐ Прикладные программисты |
| ☐ Администраторы баз данных | ☐ Конечные пользователи |
| ☐ Разработчик логической базы данных | ➤ <i>наивные</i> пользователи |
| ☐ Разработчик физической базы данных | ➤ <i>опытные</i> пользователи |

Преимущества и недостатки СУБД

Преимущества СУБД:

- ✓ контроль за избыточностью данных.
- ✓ непротиворечивость данных.
- ✓ совместное использование данных.
- ✓ поддержка целостности данных.
- ✓ повышенная безопасность.
- ✓ применение стандартов.
- ✓ повышение доступности данных.
- ✓ улучшение показателей производительности.
- ✓ упрощение сопровождения системы за счет независимости от данных.
- ✓ улучшенное управление параллельностью.
- ✓ развитые службы резервного копирования и восстановления.

Недостатки СУБД :

- ❖ сложность.
- ❖ размер.
- ❖ стоимость СУБД.
- ❖ дополнительные затраты на аппаратное обеспечение.
- ❖ затраты на преобразование.
- ❖ производительность.
- ❖ более серьезные последствия при выходе системы из строя.

Объектная модель документа

Объектная модель документа (Document Object Model – DOM) обеспечивает программный интерфейс для HTML-документов. Она определяет логическую структуру документов и способы взаимодействия с ними.

Все встроенные объекты JavaScript берут свое начало от рабочей области документа.

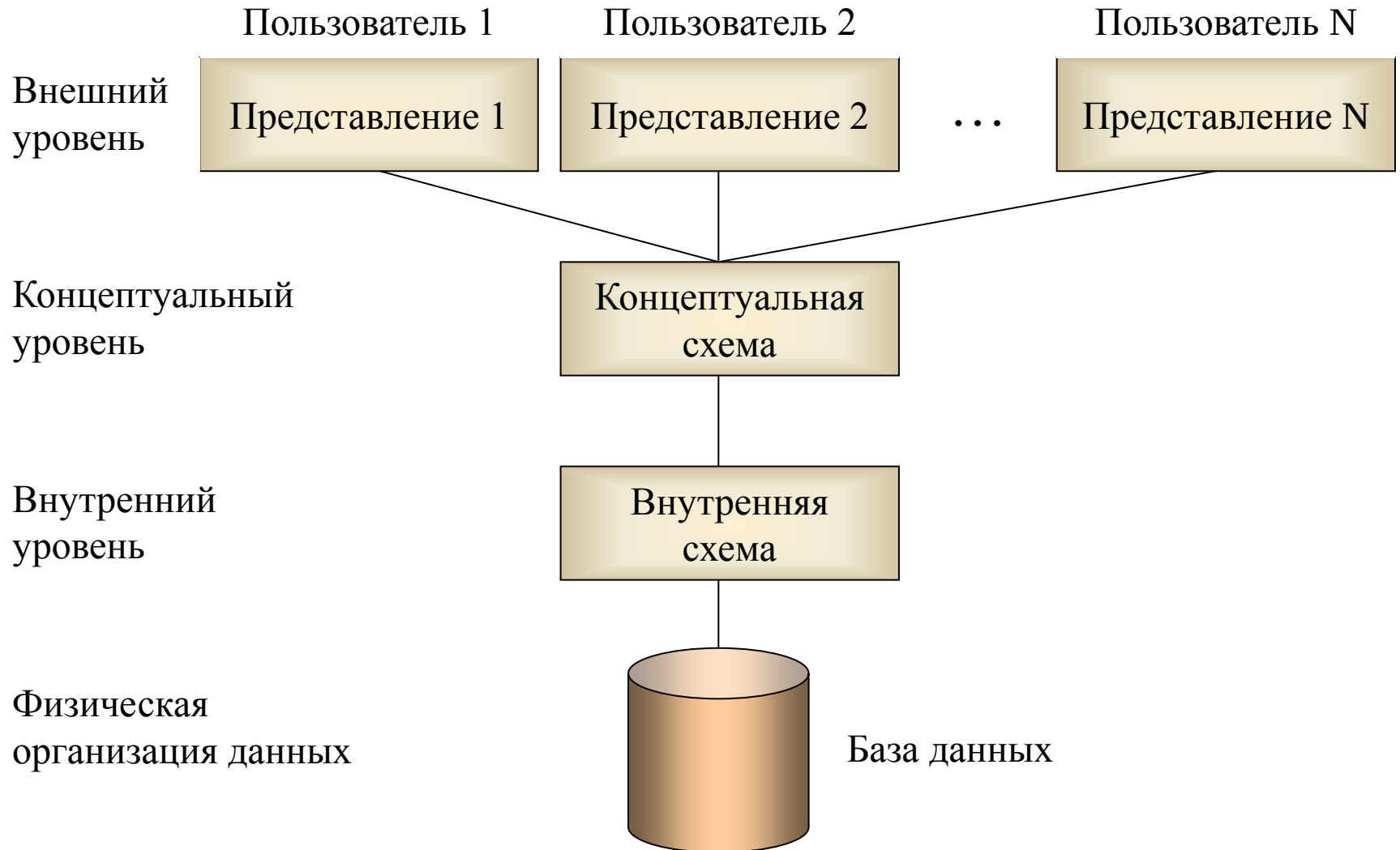
Кроме этих классов объектов пользователь может создавать и свои собственные. Но обычно большинство программ используют эту систему классов и не создают новых.

Все объекты в JavaScript берут свое начало от трех родительских объектов:

- Global — содержит глобальные переменные.
- Math — содержит большое количество разнообразных математических функций.
- Object — предок всех остальных встроенных классов.

Среда базы данных

Архитектура ANSI/SPARC



Среда базы данных

Архитектура ANSI/SPARC

Внешний уровень – это представление базы данных с точки зрения пользователей. Этот уровень описывает ту часть базы данных, которая относится к каждому пользователю.

Концептуальный уровень – это обобщающее представление базы данных. Этот уровень описывает то, какие данные хранятся в базе данных, а также связи, существующие между ними.

На концептуальном уровне представлены следующие компоненты:

- все сущности, атрибуты и связи;
- накладываемые на данные ограничения;
- семантическая информация о данных;
- информация о мерах обеспечения безопасности и поддержки целостности данных.

Внутренний уровень – это физическое представление базы данных в компьютере. Этот уровень описывает, как информация хранится в базе данных.

На внутреннем уровне хранится следующая информация:

- распределение дискового пространства для хранения данных и индексов;
- описание подробностей сохранения записей (с указанием реальных размеров сохраняемых элементов данных);
- сведения о размещении записей;
- сведения о сжатии данных и выбранных методах их шифрования.

Языки баз данных

Внутренний язык СУБД для работы с данными состоит из двух частей:

- ❑ Язык определения данных (Data Definition Language – DDL)
- ❑ Язык управления данными (Data Manipulation Language – DML)

Язык DDL – это описательный язык, который позволяет АБД или пользователю описать и поименовать сущности, необходимые для работы некоторого приложения, а также связи между различными сущностями.

Язык DML – это язык, содержащий набор операторов для поддержки основных операций манипулирования содержащимися в базе данными.

К операциям управления данными относятся следующие:

- вставка в базу данных новых сведений;
- модификация сведений, хранимых в базе данных;
- извлечение сведений, содержащихся в базе данных;
- удаление сведений из базы данных.

Процедурный язык DML – это язык, который позволяет сообщить системе о том, какие данные необходимы, и точно указать, как их можно извлечь.

Непроцедурный язык DML – это язык, который позволяет указать лишь то, какие данные требуются, но не то, как их следует извлекать

Языки баз данных

Языки 4GL (Fourth-Generation Language) выступают как непроцедурные, поскольку пользователь определяет, что должно быть сделано, а не как именно желаемый результат должен быть достигнут

Выделяют следующие типы языков четвертого поколения:

- ❑ языки представления информации, например, языки запросов или генераторы отчетов;
- ❑ специализированные языки, например, языки электронных таблиц;
- ❑ генераторы приложений, которые при создании приложений обеспечивают определение, вставку, обновление или извлечение сведений из базы данных;
- ❑ языки очень высокого уровня, предназначенные для генерации кода приложений.

Понятие информационного объекта

Информационный объект — это описание некоторой сущности (реального объекта, явления, процесса, события) в виде совокупности логически связанных реквизитов (информационных элементов). Такими сущностями для информационных объектов могут служить: цех, склад, материал, вуз, студент, сдача экзаменов и т.д.

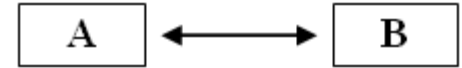
Информационный объект определенного реквизитного состава и структуры образует класс (тип), которому присваивается уникальное имя (символьное обозначение), например Студент, Сессия, Стипендия.

Типы связей

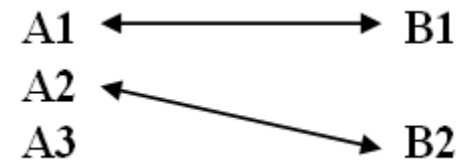
Все информационные объекты предметной области связаны между собой. Различаются связи нескольких типов, для которых введены обозначения:

- ☐ один к одному (1:1);
- ☐ один ко многим (1:M);
- ☐ многие ко многим (M:N).

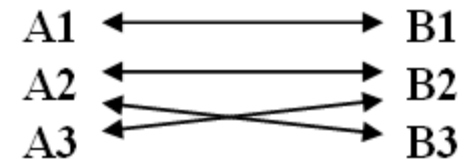
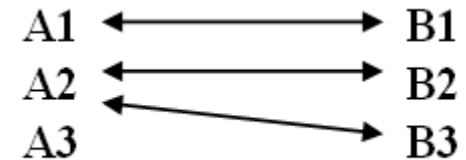
Связь **один к одному** (1:1) предполагает, что в каждый момент времени одному экземпляру информационного объекта **A** соответствует не более одного экземпляра информационного объекта **B** и наоборот.



При связи **один ко многим** (1:M) одному экземпляру информационного объекта **A** соответствует 0, 1 или более экземпляров объекта **B**, но каждый экземпляр объекта **B** связан не более чем с 1 экземпляром объекта **A**.



Связь **многие ко многим** (M:N) предполагает, что в каждый момент времени одному экземпляру информационного объекта **A** соответствует 0, 1 или более экземпляров объекта **B** и наоборот.



Модели данных

Модель данных – это интегрированный набор понятий для описания данных, связей между ними и ограничений, накладываемых на данные в некоторой организации. *Компоненты модели данных :*

- ❑ структурная часть, т. е. набор правил, по которым создается база данных;
- ❑ управляющая часть, которая определяет типы допустимых операций с данными;
- ❑ набор ограничений поддержки целостности данных (необязательно), гарантирующих корректность используемых данных.

В архитектуры ANSI/SPARC можно выделить следующие модели данных:

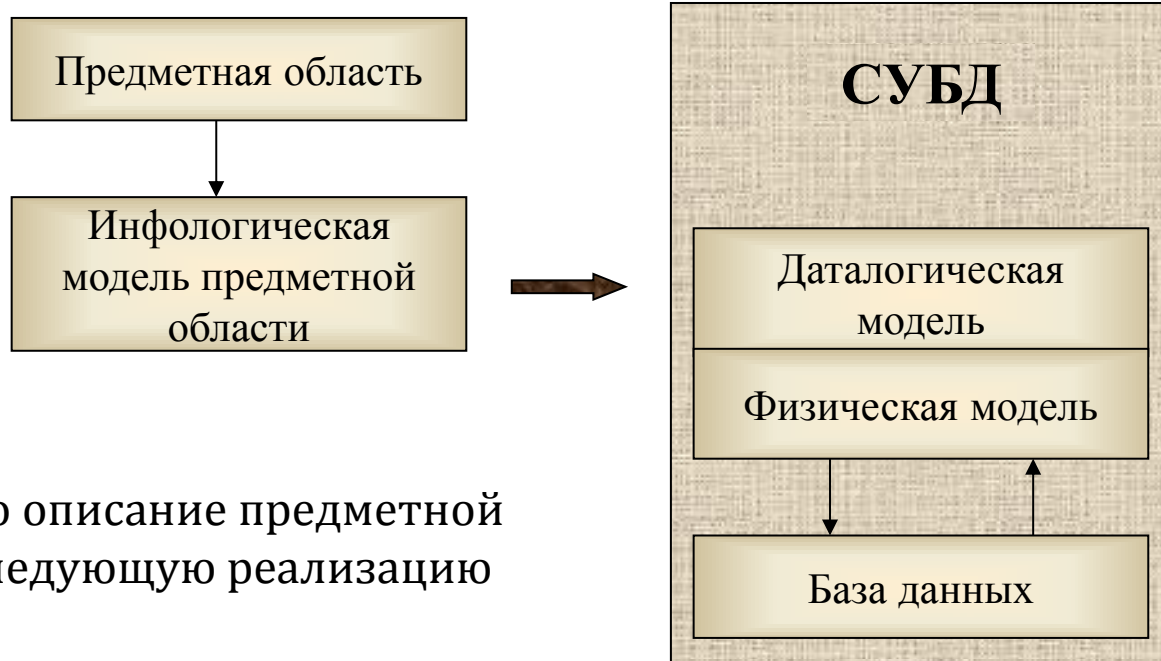
- внешнюю модель данных, которую иногда называют предметной областью;
- концептуальную модель данных, отображающую логическое (или обобщенное) представление о данных, независимое от типа выбранной СУБД;
- внутреннюю модель данных, отображающую концептуальную схему определенным образом, понятным выбранной целевой СУБД.

Категории моделей данных : **объектные** (object-based) модели данных; модели данных **на основе записей** (record-based) и **физические** модели данных.

Наиболее общие типы объектных моделей данных:

- **модель типа «сущность – связь»** или ER-модель (Entity-Relationship model);
- семантическая модель;
- функциональная модель;
- объектно-ориентированная модель.

Модели данных на основе записей



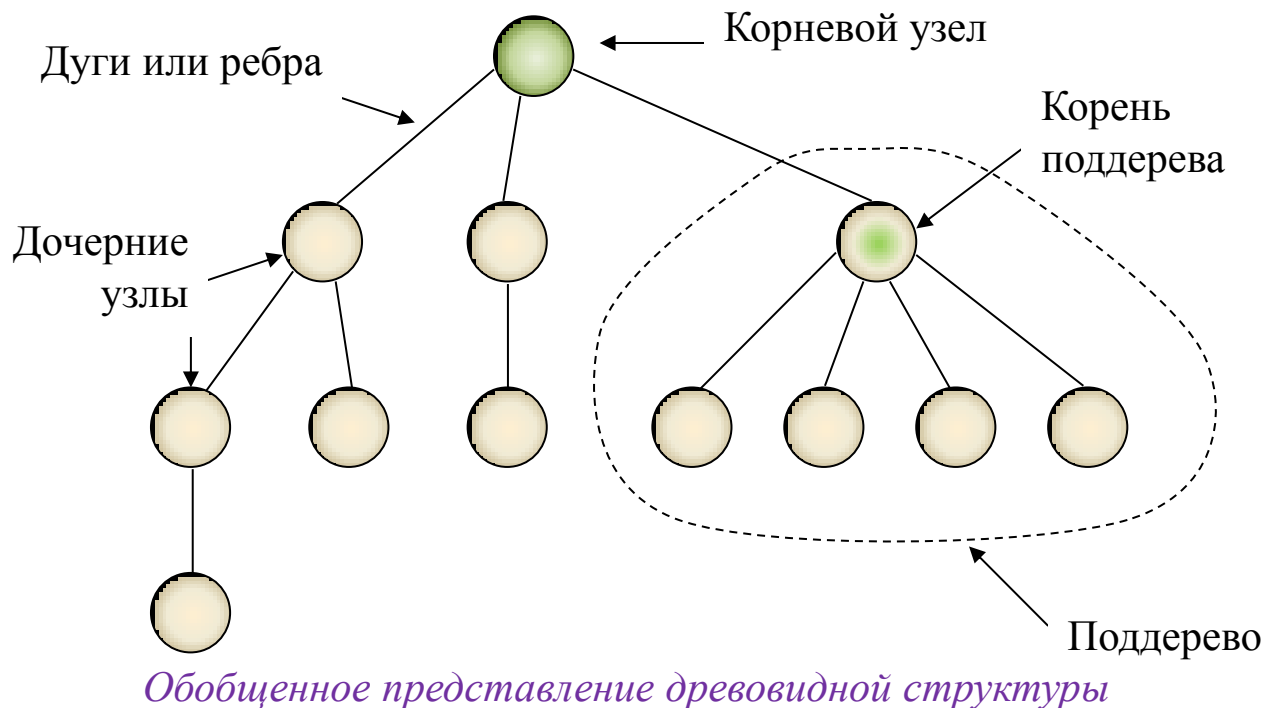
Инфологическая модель – это описание предметной области без акцента на ее последующую реализацию

Даталогическая модель – модель, отражающая Логические взаимосвязи между элементами данных безотносительно их содержания и физической организации.

Для конкретной реализации даталогической модели проектируется **физическая модель**, отображающая даталогическую модель на конкретные программные и аппаратные средства.

Иерархическая модель данных

Под **иерархической моделью данных** понимается модель, объединяющая **записи**, хранимые в общей древовидной структуре с одним корневым типом записи, который имеет несколько подчиненных типов записи или не имеет совсем. Каждый подчиненный тип записи также может иметь несколько подчиненных типов или не иметь их совсем.



Используются два метода доступа ко всем узлам (типам записей) внутри дерева:

- **прямой порядок обхода** дерева или нисходящий порядок обхода.
- **обратный порядок обхода** дерева или восходящий порядок обхода

Иерархическая модель данных



Особенностью иерархической структуры является то, что в ней дочерние экземпляры не могут существовать без наличия родительских экземпляров.

Преимущество

- ❑ целостность на уровне ссылок автоматически обеспечивается там, где экземпляры записей полностью зависят от других экземпляров записей

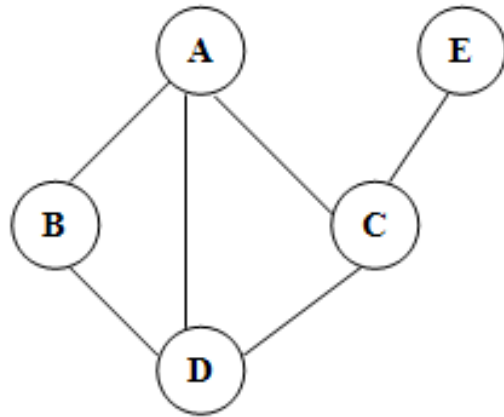
Недостатки

- ❑ Невозможность хранения экземпляров записей, которые не имеют никаких родительских записей
- ❑ Трудность моделирования связей типа «многие ко многим» (M:N) или других, более сложных неиерархических связей, в которых тип записи имеет несколько разных связей с несколькими другими типами записей

Сетевая модель данных

Под **сетевой моделью данных** понимается модель, состоящая из записей, элементов данных и связей типа “**один ко многим**” (1:M), установленных между записями. При этом связи типа “**многие ко многим**” (M:N) и рекурсивные связи поддерживаются с помощью декомпозиции.

Студент (№ зачетки, фамилия, группа)



недостатки сетевой модели данных:

- ❑ если концептуальная схема должна быть изменена в соответствии с изменениями структуры хранения записей, то это может вызвать необходимость изменения прикладных программ. Следовательно, реализация макета базы данных может существенно усложниться;

- ❑ новые или измененные приложения могут не обладать достаточной производительностью, т. к. база данных структурирована для существовавших ранее приложений;

- ❑ при проектировании базы данных потребуется квалифицированная помощь, особенно при определении необходимых структур данных и при разработке связанных с ними прикладных программ.

Базы данных (Модели данных)

Реляционная модель данных

В реляционных базах данных вся информация представляется в виде двумерных таблиц. Реляционная модель опирается на систему понятий реляционной алгебры, важнейшими из которых являются “таблица”, “отношение”, “строка”, “первичный ключ”.

Название таблицы — Сотрудники отдела

Атрибуты — Номер пропуска, ФИО, Должность, Телефон

Кортежи (строки) — {5234, Никитин Игорь Иванович, начальник сектора, 82-86}, {5245, Перов Сергей Алексеевич, ведущий инженер, 82-84}, {5243, Левин Андрей Сергеевич, инженер, 82-89}, {5290, Карпов Иван Степанович, инженер, 82-89}

Первичный ключ — Номер пропуска

Номер пропуска	ФИО	Должность	Телефон
5234	Никитин Игорь Иванович	начальник сектора	82-86
5245	Перов Сергей Алексеевич	ведущий инженер	82-84
5243	Левин Андрей Сергеевич	инженер	82-89
5290	Карпов Иван Степанович	инженер	82-89

Каждая реляционная таблица представляет собой двумерный массив и обладает следующими свойствами:

- ❑ каждый элемент таблицы — один элемент данных;
- ❑ все столбцы (атрибуты) в таблице однородные, т.е. все элементы в столбце имеют одинаковый тип (числовой, символьный и т.д.) и длину;
- ❑ каждый столбец (атрибут) имеет уникальное имя;
- ❑ одинаковые строки в таблице отсутствуют;
- ❑ порядок следования строк и столбцов может быть произвольным.

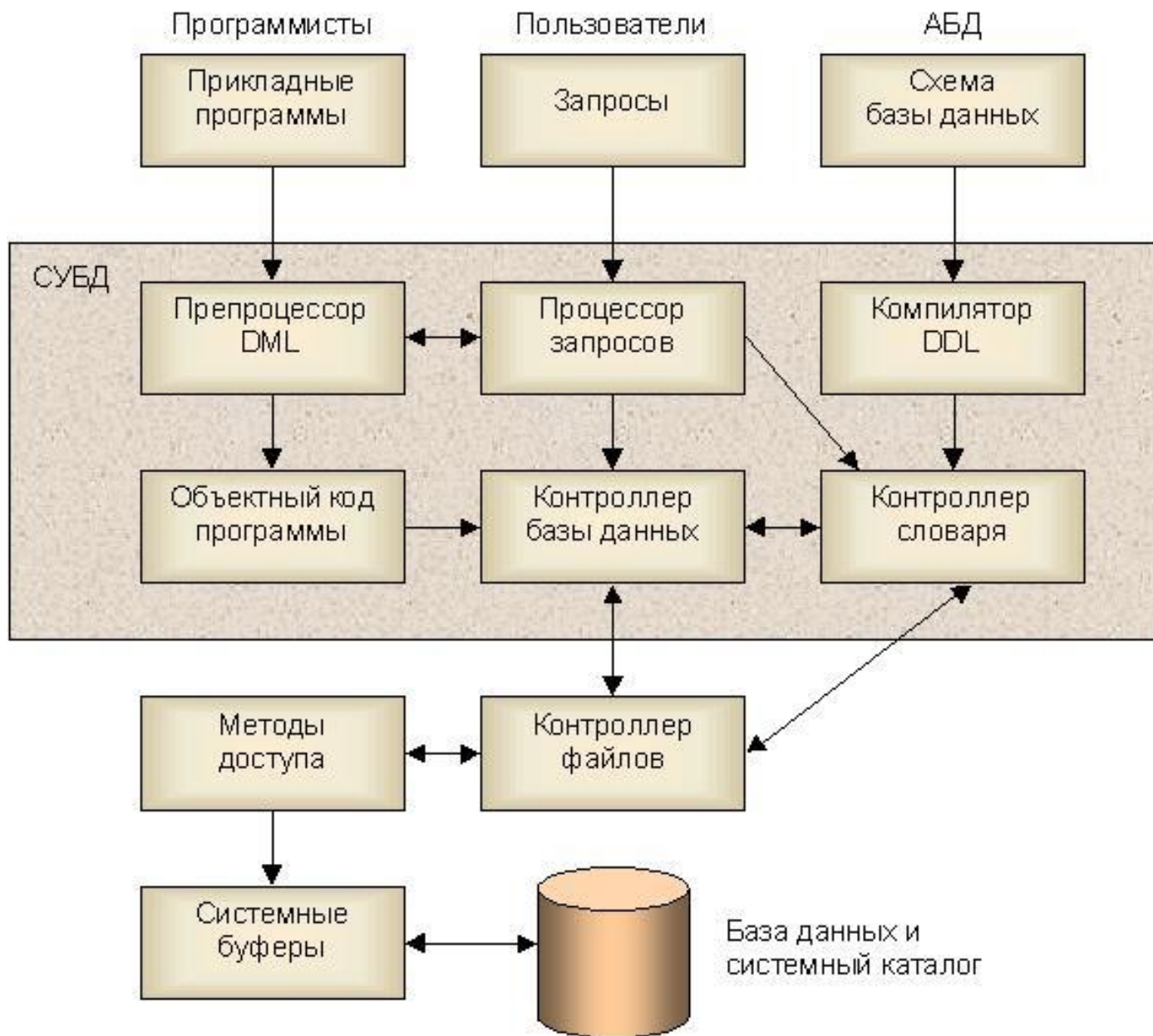
Функции СУБД

- Хранение, извлечение и обновление данных
- Каталог, доступный конечным пользователям.
- Поддержка транзакций
- Сервисы управления параллельностью.
- Сервисы восстановления.
- Сервисы контроля доступа к данным.
- Поддержка обмена данными.
- Службы поддержки целостности данных
- Службы поддержки независимости от данных
- Вспомогательные службы.

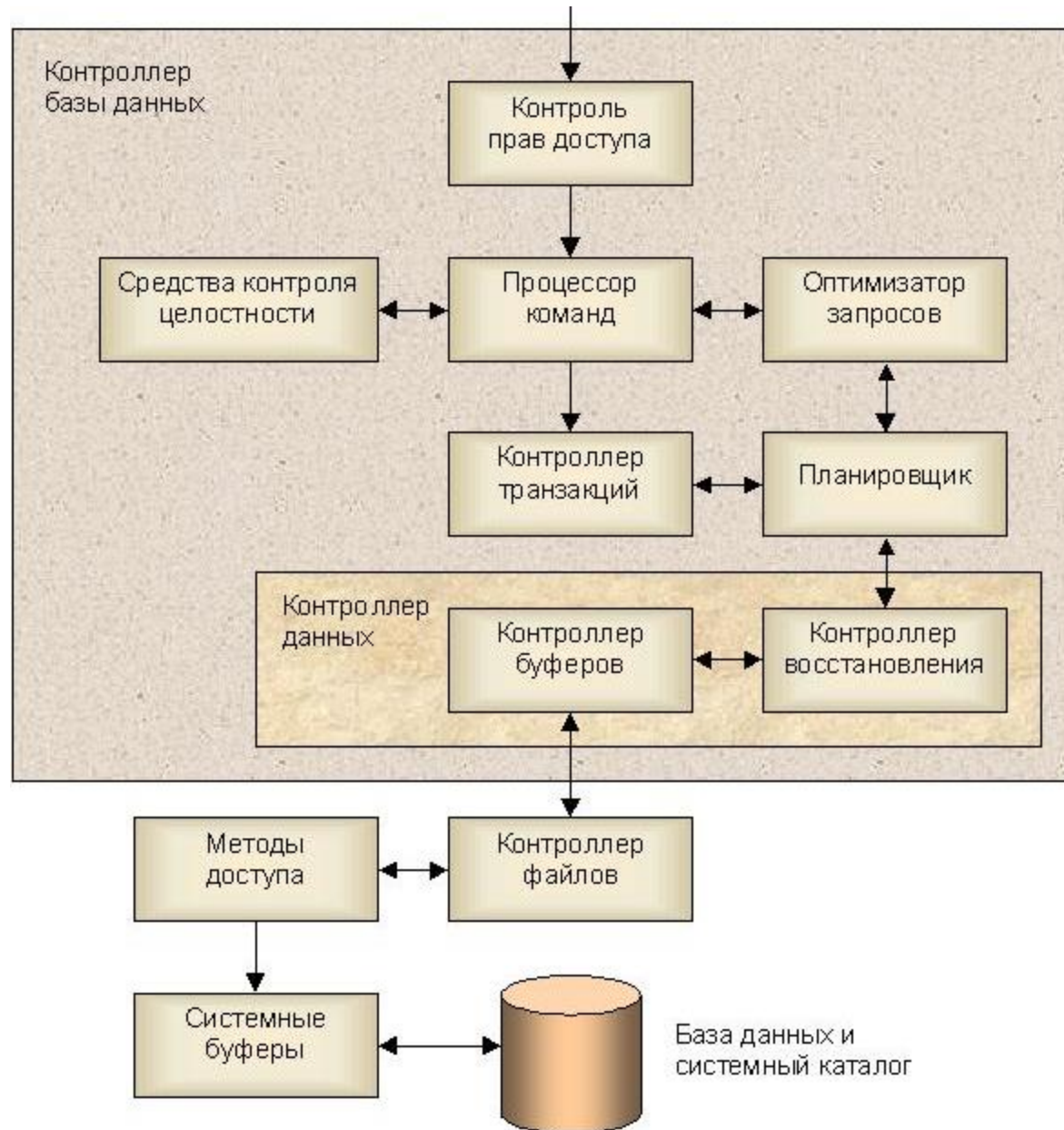
Вспомогательные службы

- **утилиты импортирования**, предназначенные для загрузки данных из файлов или других СУБД,
- **утилиты экспортирования**, которые служат для выгрузки базы данных в файлы или другие СУБД;
- **средства мониторинга**, предназначенные для отслеживания характеристик функционирования и использования базы данных;
- **программы статистического анализа**, позволяющие оценить производительность или степень использования базы данных;
- **инструменты реорганизации индексов**, предназначенные для перестройки индексов и обработки случаев их переполнения;
- **инструменты сборки мусора и перераспределения памяти** для физического устранения удаленных записей с запоминающих устройств, объединения освобожденного пространства и перераспределения памяти в случае необходимости

Компоненты типичной СУБД

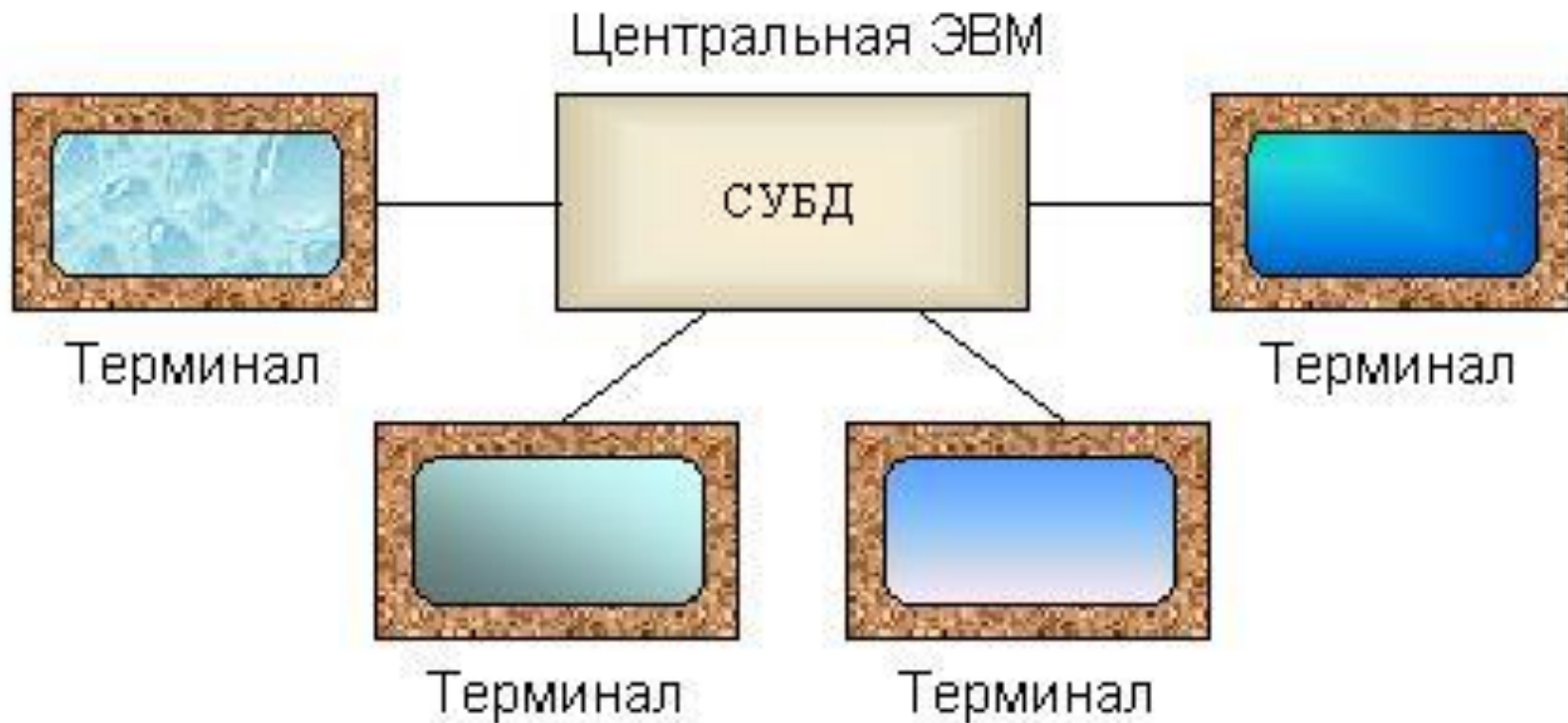


Компоненты контроллера базы данных



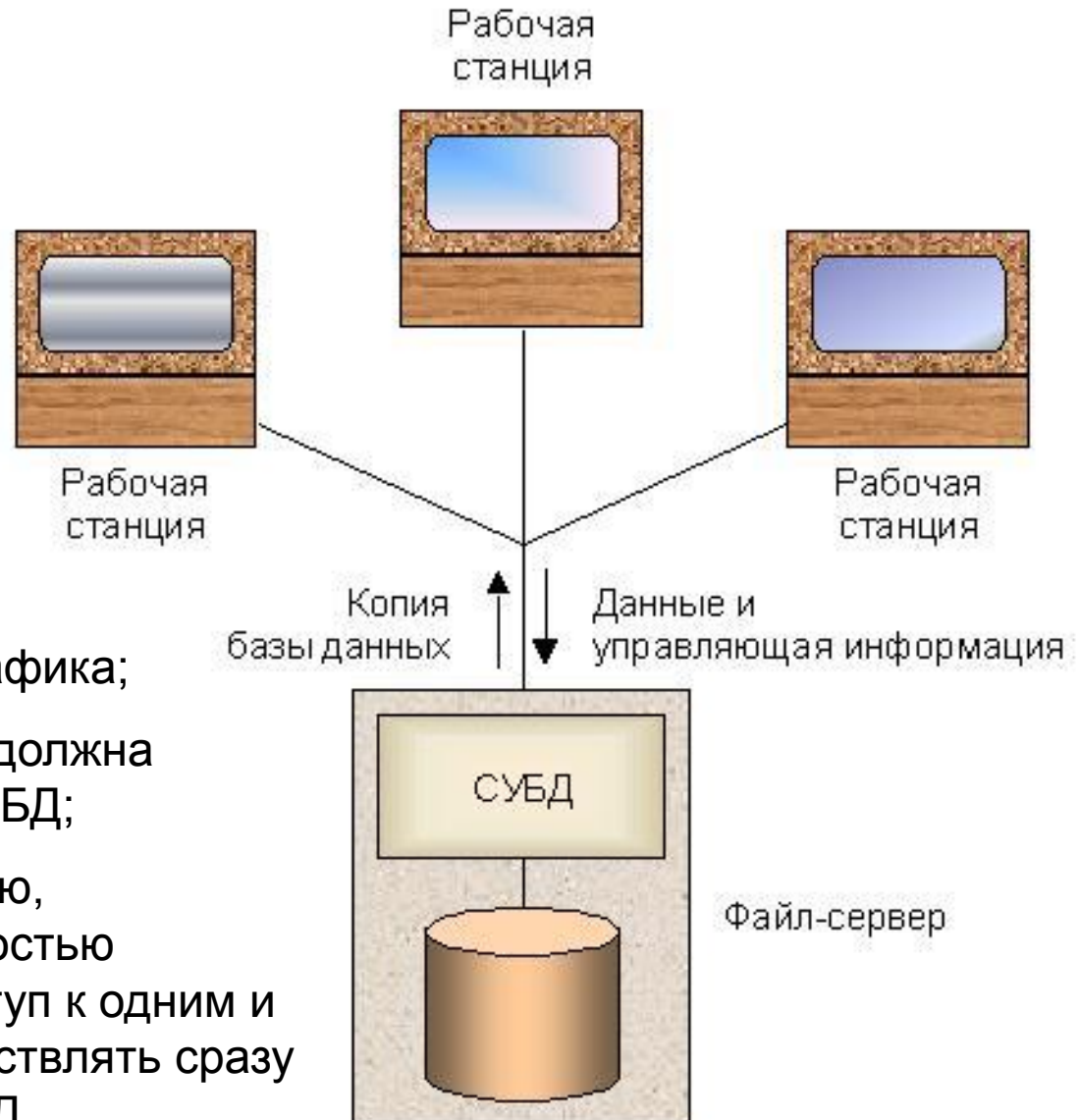
Архитектура многопользовательских СУБД

Телеобработка



Архитектура многопользовательских СУБД

Файловый сервер

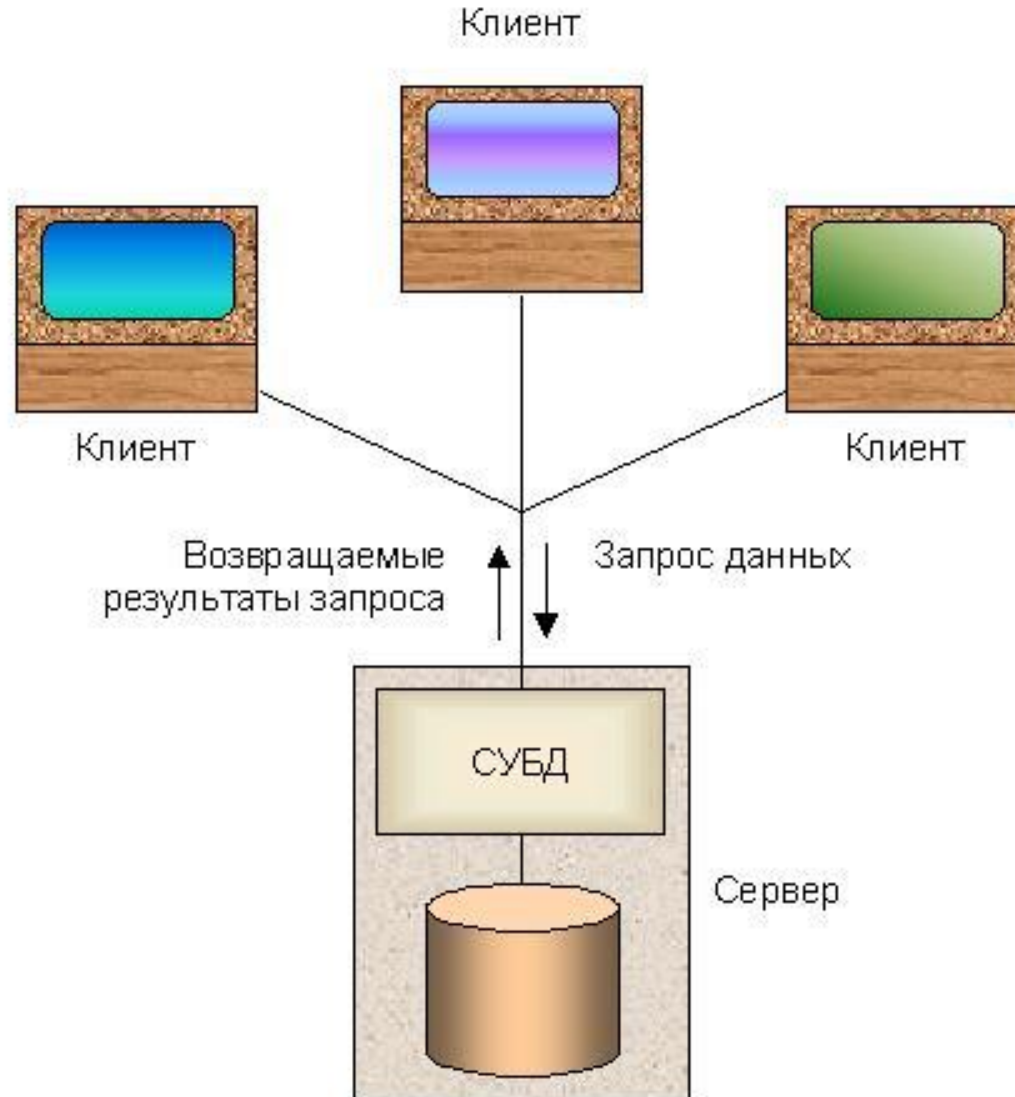


Основные недостатки:

- большой объем сетевого трафика;
- на каждой рабочей станции должна находиться полная копия СУБД;
- управление параллельностью, восстановлением и целостностью усложняется, поскольку доступ к одним и тем же файлам могут осуществлять сразу несколько экземпляров СУБД.

Архитектура многопользовательских СУБД

Технология клиент/сервер



Архитектура многопользовательских СУБД

Технология клиент/сервер

Функции, выполняемые участниками взаимодействия в среде клиент/сервер

Клиент	Сервер
Управляет пользовательским интерфейсом	Принимает и обрабатывает запросы к базе данных со стороны клиента
Принимает и проверяет синтаксис введенного пользователем запроса	Проверяет полномочия пользователей
Выполняет приложение	Гарантирует соблюдение ограничений целостности
Генерирует запрос к базе данных и передает его серверу	Выполняет запросы/обновления и возвращает результаты клиенту
Отображает полученные данные пользователю	Поддерживает системный каталог
	Обеспечивает параллельный доступ к базе данных
	Обеспечивает управление восстановлением

Архитектура многопользовательских СУБД

Технология клиент/сервер

Преимущества

- Обеспечивается более широкий доступ к существующим базам данных;
- Повышается общая производительность системы.
- Стоимость аппаратного обеспечения снижается.
- Сокращаются коммуникационные расходы.
- Повышается уровень непротиворечивости данных.

Сервер может самостоятельно управлять проверкой целостности данных, поскольку все ограничения определяются и проверяются только в одном месте. При этом каждому приложению не придется выполнять собственную проверку;

- Такая архитектура естественным образом отображается на архитектуру открытых систем.

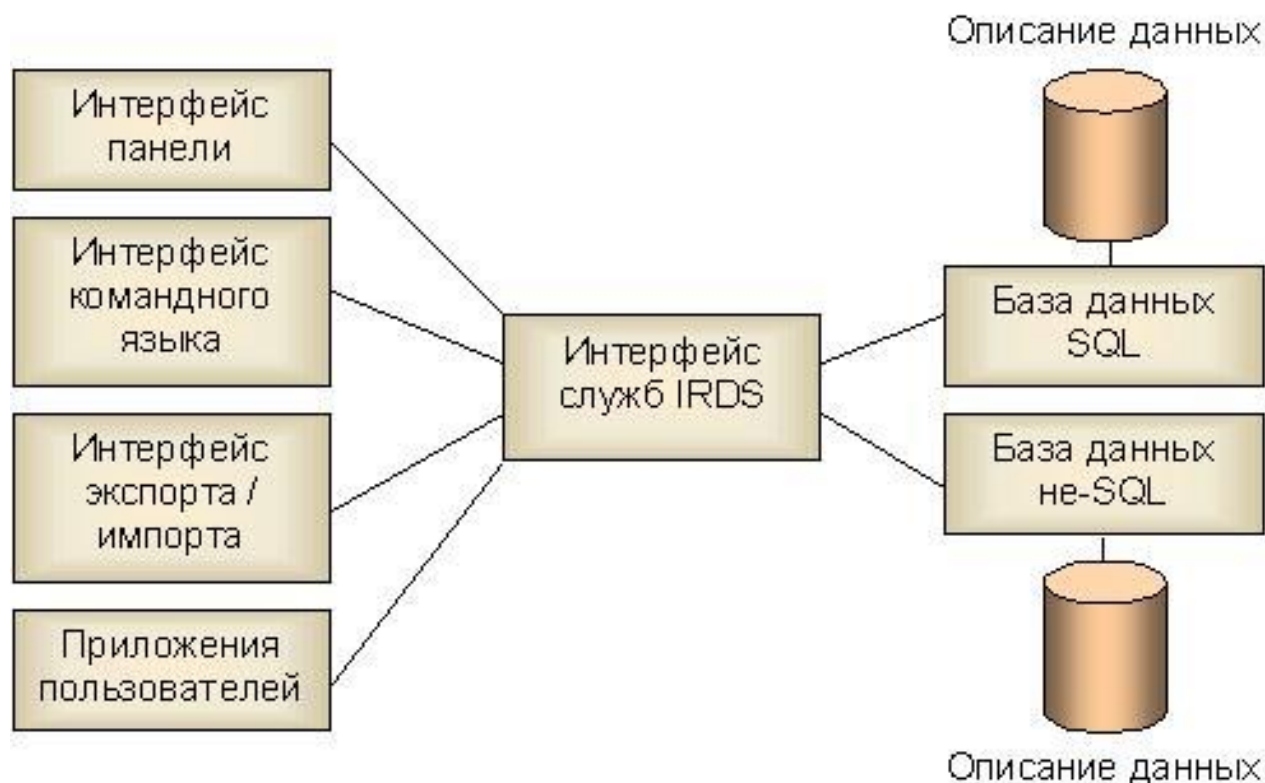
Системные каталоги

- **Системный каталог** – это хранилище данных, которые описывают сохраняемую в базе данных информацию, т. е. метаданные или “данные о данных”.
- Для проверки наличия у пользователя полномочий необходимо
 - имена пользователей, для которых разрешен доступ к базе данных;
 - имена элементов данных в базе данных;
 - элементы данных, к которым каждый пользователь имеет право доступа, и разрешенные типы доступа к ним – для вставки, обновления, удаления или чтения.
- Для проверки целостности данных, необходимо
 - имена элементов данных из базы данных;
 - типы и размеры элементов данных;
 - ограничения, установленные для каждого из элементов данных.

IRDS (Information Resource Dictionary System)

Стандарты IRDS определяют набор правил хранения информации в словаре данных и доступа к ней, преследуя при этом три следующих цели:

- расширяемость данных;
- целостность данных;
- контролируемый доступ к данным.



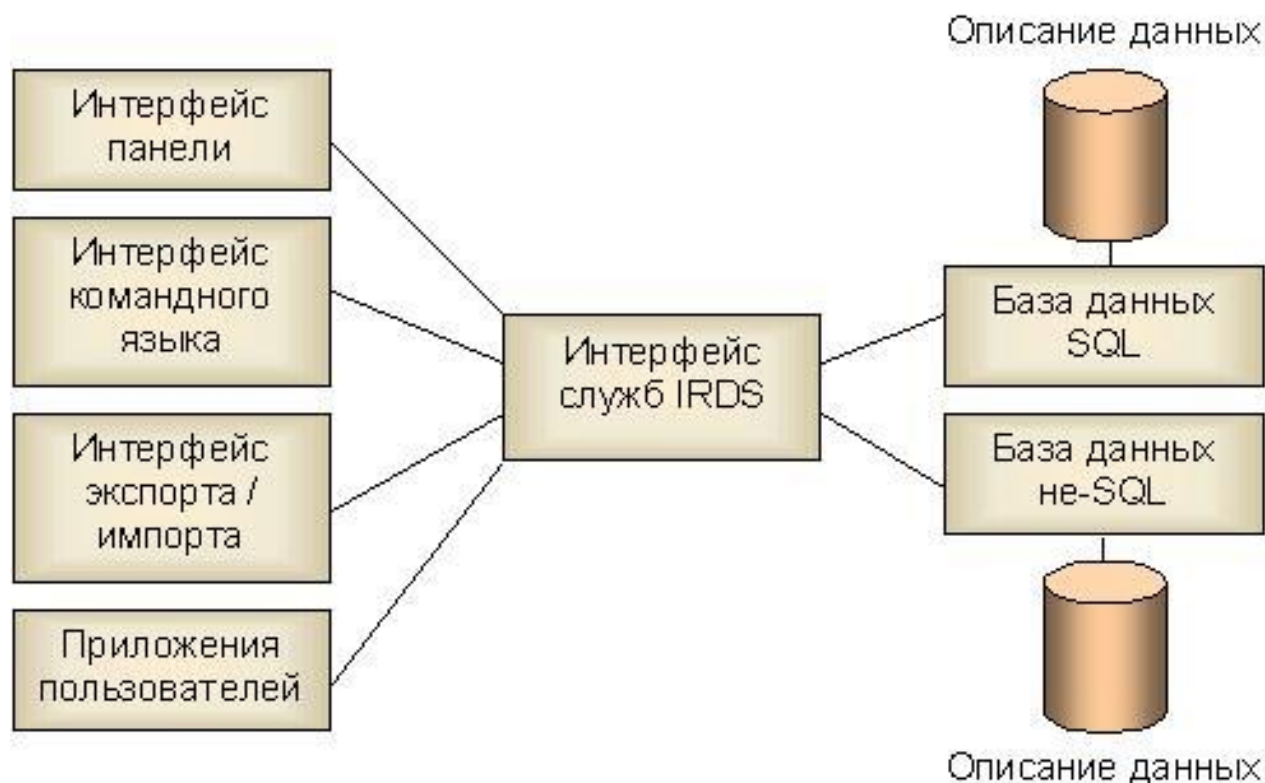
Системные каталоги

- ***Системный каталог*** – это хранилище данных, которые описывают сохраняемую в базе данных информацию, т. е. метаданные или “данные о данных”.
- Для проверки наличия у пользователя полномочий необходимо
 - имена пользователей, для которых разрешен доступ к базе данных;
 - имена элементов данных в базе данных;
 - элементы данных, к которым каждый пользователь имеет право доступа, и разрешенные типы доступа к ним – для вставки, обновления, удаления или чтения.
- Для проверки целостности данных, необходимо
 - имена элементов данных из базы данных;
 - типы и размеры элементов данных;
 - ограничения, установленные для каждого из элементов данных.

IRDS (Information Resource Dictionary System)

Стандарты IRDS определяют набор правил хранения информации в словаре данных и доступа к ней, преследуя при этом три следующих цели:

- расширяемость данных;
- целостность данных;
- контролируемый доступ к данным.



Краткий обзор истории реляционной модели

Реляционная модель впервые была предложена в 1970 году Э.Ф. Коддом

Цели создания реляционной модели формулировались следующим образом:

- обеспечение более высокой степени независимости от данных.
Прикладные программы не должны зависеть от изменений внутреннего представления данных, в частности от изменений организации файлов, переупорядочивания записей и путей доступа;
- создание прочного фундамента для решения семантических вопросов, а также проблем непротиворечивости и избыточности данных.
Использование понятия нормализованных отношений, т. е. отношений без повторяющихся групп;
- расширение языков управления данными за счет включения операций над множествами.

Краткий обзор истории реляционной модели

Наиболее значительные исследования по реляционным базам данных были проведены в рамках трех проектов.

- Система System R (конец 70-х годов, исследовательская лаборатория корпорации IBM, Калифорния)

Ответвления развития:

- создание языка структурированных запросов SQL
- создание различных коммерческих реляционных СУБД, которые впервые появились на рынке в начале 80-х годов, таких как DB2 и SQL/DS корпорации IBM, а также ORACLE корпорации ORACLE Corporation.
- Проект INGRES [INteractive GGraphics REtrieval System] (конец 70-х годов, Калифорнийский университет)
- Система Peterlee Relational Test Vehicle (1976 г, научный центр корпорации IBM, Великобритания)

Используемая терминология

Отношение – это плоская таблица, состоящая из столбцов и строк.

Атрибут – это поименованный столбец отношения.

Домен – это набор допустимых значений для одного или нескольких атрибутов.

Кортеж – это строка отношения.

Степень отношения – это количество атрибутов, которое оно содержит.

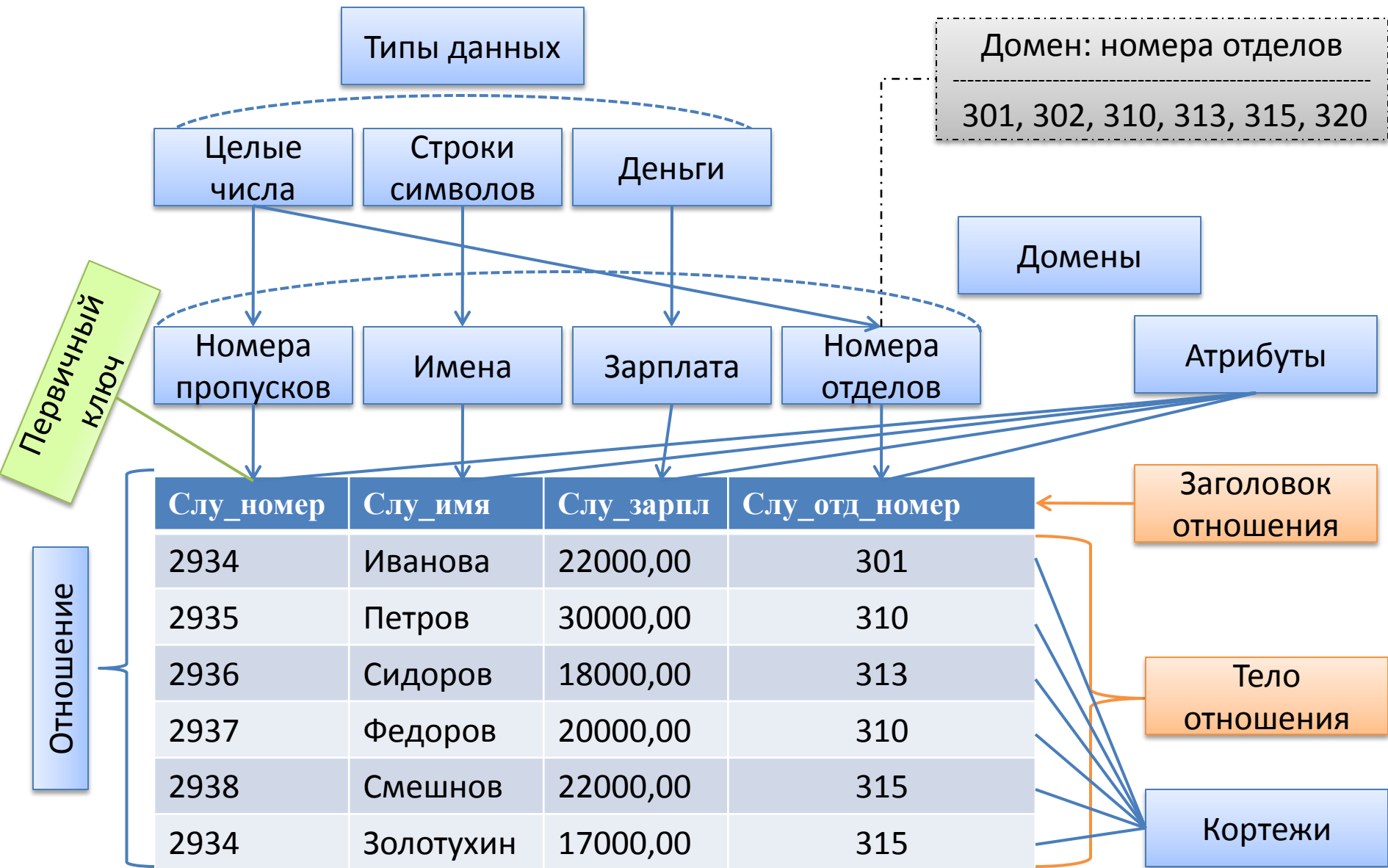
Кардинальность – это количество кортежей, которое содержит отношение.

Реляционная база данных – это набор нормализованных отношений.

Альтернативные варианты терминов в реляционной модели

Официальные термины	Альтернативный вариант 1	Альтернативный вариант 2
Отношение	Таблица	Файл
Кортеж	Строка	Запись
Атрибут	Столбец	Поле

Используемая терминология



Отношения в базе данных

Математические отношения

Для понимания термина **отношение** рассмотрим несколько математических понятий. Пусть имеется два множества D_1 и D_2 , где $D_1 = \{2, 4\}$ и $D_2 = \{1, 3, 5\}$.

Декартовым произведением этих двух множеств (обозначается как $D_1 \times D_2$) называется набор из всех возможных пар, в которых первым идет элемент множества D_1 , а вторым – элемент множества D_2 .

Альтернативный способ выражения этого произведения заключается в поиске всех комбинаций элементов, в которых первым идет элемент множества D_1 , а вторым – элемент множества D_2 .

В примере получим следующий результат

$$D_1 \times D_2 = \{(2, 1), (2, 3), (2, 5), (4, 1), (4, 3), (4, 5)\}$$

Любое подмножество этого декартова произведения является отношением. Например, в нем можно выделить отношение R

$$R = \{(2, 1), (4, 1)\}$$

Для определения всех возможных пар, которые будут входить в отношение, можно задать некоторые условия их выборки. Например, если обратить внимание на то, что отношение R содержит все возможные пары, в которых второй элемент равен 1, то определение отношения R можно сформулировать следующим образом:

$$R = \{ \{ (x, y) \mid x \in D_1, y \in D_2, \text{ и } y = 1 \} \}$$

Отношения в базе данных

Реляционная схема – это имя отношения, за которым следует множество пар имен атрибутов и доменов.

Для атрибутов $A_1, A_2, \dots, A_n$ с доменами $D_1, D_2, \dots, D_n$ реляционной схемой будет множество $\{A_1 : D_1, A_2 : D_2, \dots, A_n : D_n\}$. Отношение R , заданное реляционной схемой S , является множеством отображений имен атрибутов на соответствующие им домены. Таким образом, отношение R является множеством таких n -арных кортежей $\{A_1 : d_1, A_2 : d_2, \dots, A_n : d_n\}$, где $d_1 \in D_1, d_2 \in D_2, \dots, d_n \in D_n$.

Отношение обладает следующими характеристиками:

- отношение имеет имя, которое отличается от имен всех других отношений;
- каждая ячейка отношения содержит только атомарное (неделимое) значение;
- каждый атрибут имеет уникальное имя;
- значения атрибута берутся из одного и того же домена;
- порядок следования атрибутов не имеет никакого значения;
- каждый кортеж является уникальным, т. е. дубликатов кортежей быть не может;
- теоретически порядок следования кортежей в отношении не имеет никакого значения. Однако на практике этот порядок может существенно повлиять на эффективность доступа к ним.

Реляционные ключи

Суперключ (*superkey*) – атрибут или множество атрибутов, которое единственным образом идентифицирует кортеж данного отношения.

Потенциальный ключ – это суперключ, который не содержит подмножества, также являющегося суперключом данного отношения.

Потенциальный ключ K для данного отношения R обладает двумя свойствами:

- ❑ **уникальность.** В каждом кортеже отношения R значение ключа K единственным образом идентифицируют этот кортеж;
- ❑ **неприводимость.** Никакое допустимое подмножество ключа K не обладает свойством уникальности.

Составной ключ – это ключ состоящий из нескольких атрибутов

Первичный ключ – это потенциальный ключ, который выбран для уникальной идентификации кортежей внутри отношения.

Альтернативный ключ – это потенциальный ключ, который не выбран в качестве первичного ключа

Внешний ключ - это атрибут или множество атрибутов внутри отношения, которое соответствует потенциальному ключу некоторого (может быть, того же самого) отношения.

Базовые таблицы и представления

Базовые таблицы – исходные таблицы базы данных

Производные таблицы – таблицы, полученные **базовых** них путем выполнения реляционных выражений

Базовое отношение – это поименованное отношение, соответствующее сущности в концептуальной схеме, кортежи которого физически хранятся в базе данных.

Представление – это динамический результат одной или нескольких реляционных операций над базовыми отношениями с целью создания некоторого иного отношения.

*Представление является **виртуальным отношением**, которого реально в базе данных не существует, но которое создается по требованию отдельного пользователя в момент поступления этого требования.*

Причины использования механизма представлений:

- ☐ он предоставляет мощный и гибкий механизм защиты.
- ☐ он позволяет организовать доступ пользователей к данным наиболее удобным для них образом;
- ☐ он позволяет упрощать сложные операции с базовыми отношениями.

Признаки реляционной СУБД

Фундаментальные правила (правила 0 и 12)

❑ *Правило 0 – фундаментальное правило*

Любая система, которая рекламируется или представляется как реляционная СУБД, должна быть способна управлять базами данных исключительно с помощью ее реляционных функций.

❑ *Правило 12 – правило запрета обходных путей*

Если реляционная система имеет низкоуровневый язык (с последовательной построчной обработкой), он не может быть использован для отмены или обхода правил и ограничений целостности, составленных на реляционном языке более высокого уровня (с обработкой сразу нескольких строк).

Признаки реляционной СУБД

Структурные правила (правила 1 и 6)

❑ *Правило 1 – представление информации*

Вся информация в реляционной базе данных представляется в явном виде на логическом уровне и только одним способом – в виде значений в таблицах.

❑ *Правило 6 – обновление представления*

Все представления, которые являются теоретически обновляемыми, должны быть обновляемы и в данной системе.

Признаки реляционной СУБД

Правила целостности (правила 3 и 10)

❑ *Правило 3 – систематическая обработка неопределенных значений (NULL)*

Неопределенные значения (задаваемые с помощью определителя NULL), т. е. значения, отличные от пустой строки или строки пустых символов, а также от нуля или любого другого конкретного значения, поддерживаются для систематического представления отсутствующей или неприемлемой информации, причем независимо от типа данных.

❑ *Правило 10 – независимость ограничений целостности*

Специфические для данной РСУБД ограничения целостности должны определяться на языке реляционных данных и храниться в системном каталоге, а не в прикладных программах.

Признаки реляционной СУБД

Правила манипулирования данными (правила 2, 4, 5, 7)

❑ **Правило 2 – гарантированный доступ**

Для всех и каждого элемента данных (т. е. его атомарного значения) реляционной базы данных должен быть гарантирован логический доступ на основе комбинации имени таблицы, значения первичного ключа и значения имени столбца.

❑ **Правило 4 – динамический интерактивный каталог, построенный по правилам реляционной модели**

Описание базы данных должно представляться на логическом уровне таким же образом, как и обычные данные, что позволит санкционированным пользователям использовать для обращений к этому описанию тот же реляционный язык, что и при обращении к данным.

❑ **Правило 5 – исчерпывающий подязык данных**

Реляционная система может поддерживать несколько языков и различные режимы работы с терминалами. Однако должен существовать по крайней мере один язык, операторы которого позволяли бы выражать все следующие конструкции: 1) определение данных; 2) определение представлений; 3) команды манипулирования данными (доступные как интерактивно, так и из программ); 4) ограничения целостности; 5) авторизация пользователей; 6) организация транзакций (запуск, фиксация и откат).

❑ **Правило 7 – высокоуровневые операции вставки, обновления и удаления**

Способность обрабатывать базовые или производные отношения (т. е. представления) как единый операнд должна относиться не только к процедурам извлечения данных, но и к операциям вставки, обновления и удаления данных.

Признаки реляционной СУБД

Правила независимости от данных (правила 8, 9 и 11)

❑ **Правило 8 – физическая независимость от данных**

Прикладные программы и средства работы с терминалами должны оставаться логически незатронутыми при внесении любых изменений в способы хранения данных или методы доступа к ним.

❑ **Правило 9 – логическая независимость от данных**

Прикладные программы и средства работы с терминалами должны оставаться логически незатронутыми при внесении в базовые таблицы любых не меняющих информацию изменений, которые теоретически не должны затрагивать прикладное программное обеспечение.

❑ **Правило 11 – независимость от распределения данных**

Подъязык манипулирования данными в реляционной СУБД должен позволять прикладным программам и запросам оставаться логически неизменными независимо от того, как хранятся данные – физически централизовано или в распределенном виде.

Жизненный цикл информационных систем

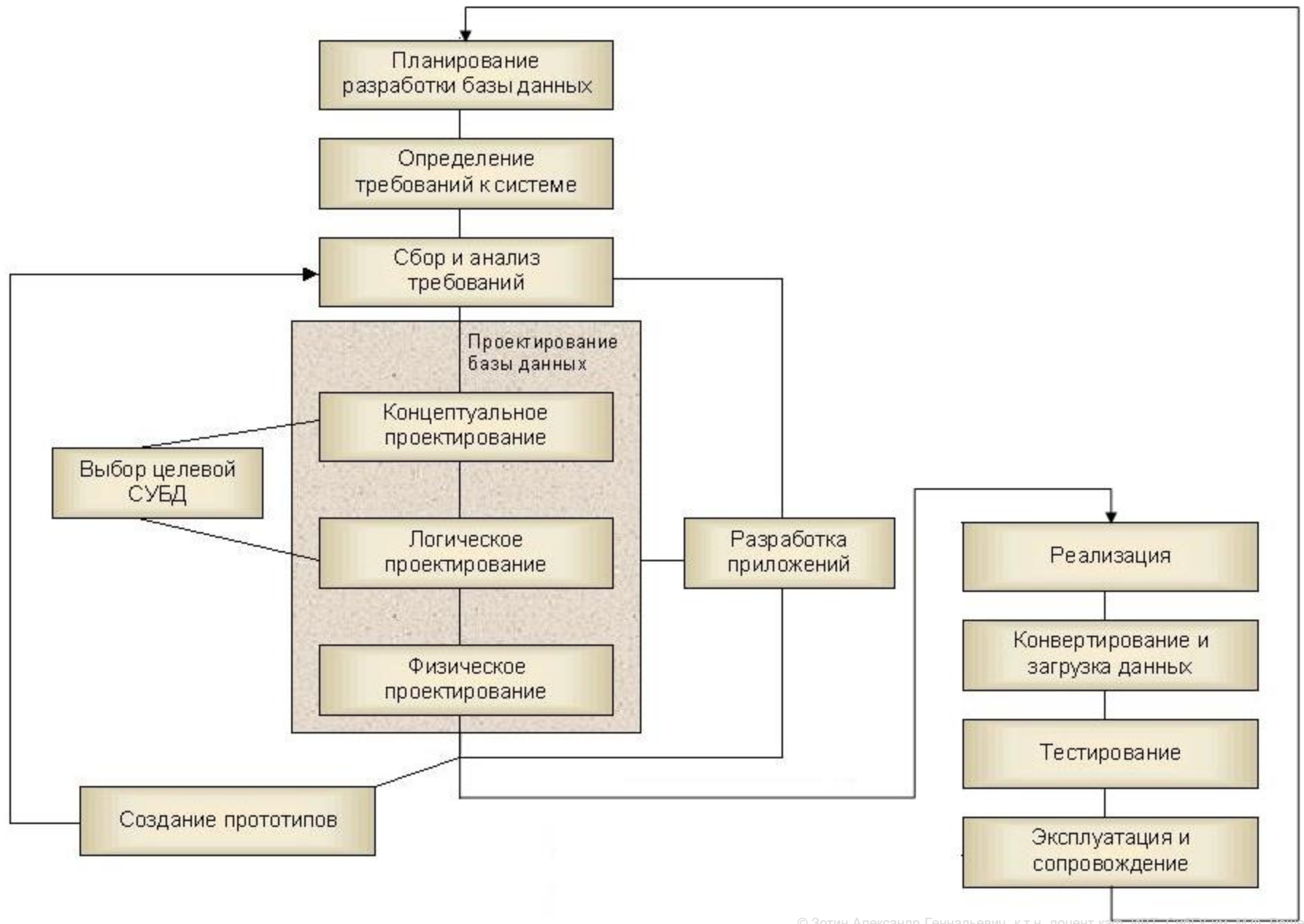
Информационная система (Information System – IS) – это ресурсы, которые позволяют выполнять сбор, корректировку и распространение информации внутри организации.

Компоненты типичной компьютеризированной информационной системы:

- База данных;
- Программное обеспечение базы данных;
- Прикладное программное обеспечение;
- Аппаратное обеспечение, в том числе устройства хранения;
- Персонал, использующий и разрабатывающий эту систему.

Жизненный цикл информационной системы обычно состоит из следующих этапов: планирование, сбор и анализ требований, проектирование (включая проектирование базы данных), создание прототипа, реализация, тестирование, преобразование данных и сопровождение.

Жизненный цикл приложения баз данных



Жизненный цикл приложения баз данных

Планирование разработки базы данных — это подготовительные действия, позволяющие с максимально возможной эффективностью реализовать этапы жизненного цикла приложения баз данных.

Решаемые задачи:

- ✓ определение бизнес-правил и целей организации с последующим выделением ее потребностей в информационных технологиях;
- ✓ оценка показателей уже существующих информационных систем с целью выявления их сильных и слабых сторон;
- ✓ оценка возможностей использования информационных технологий для достижения конкурентоспособного преимущества.

Определение требований к системе — это определение диапазона действия и границ приложения базы данных, состава его пользователей и областей применения.

Жизненный цикл приложения баз данных

Под **сбором и анализом требований пользователей** понимается сбор и анализ информации той части организации, работа которой будет поддерживаться с помощью создаваемого приложения базы данных, а также использование этой информации для определения требований пользователей к создаваемой системе

Требование – это некоторая функция, которая должна быть включена в создаваемую систему.

Способы сбора информации :

- посредством опроса отдельных сотрудников предприятия, особенно специалистов в наиболее важных областях ее деятельности;
- с помощью наблюдений за деятельностью предприятия;
- посредством изучения документов, особенно тех, которые используются для сбора или представления информации;
- с помощью анкет, предназначенных для сбора информации у широкого круга пользователей;
- за счет использования опыта проектирования других подобных систем.

Жизненный цикл приложения баз данных

Проектирование базы данных – это процесс создания проекта базы данных, предназначенной для поддержки функционирования предприятия и способствующей достижению его целей.

Основными целями проектирования базы данных являются:

- ❑ Представление данных и связей между ними, необходимых для всех основных областей применения данного приложения и любых существующих групп его пользователей;
- ❑ Создание модели данных, способной поддерживать выполнение любых требуемых транзакций обработки данных;
- ❑ Разработка предварительного варианта проекта, структура которого позволяет удовлетворить все основные требования, предъявляемые к производительности системы (например, ко времени реакции системы).

Основные подходы к проектированию систем баз данных:

- Восходящий
- Нисходящий
- Изнутри наружу
- Смешанная стратегии

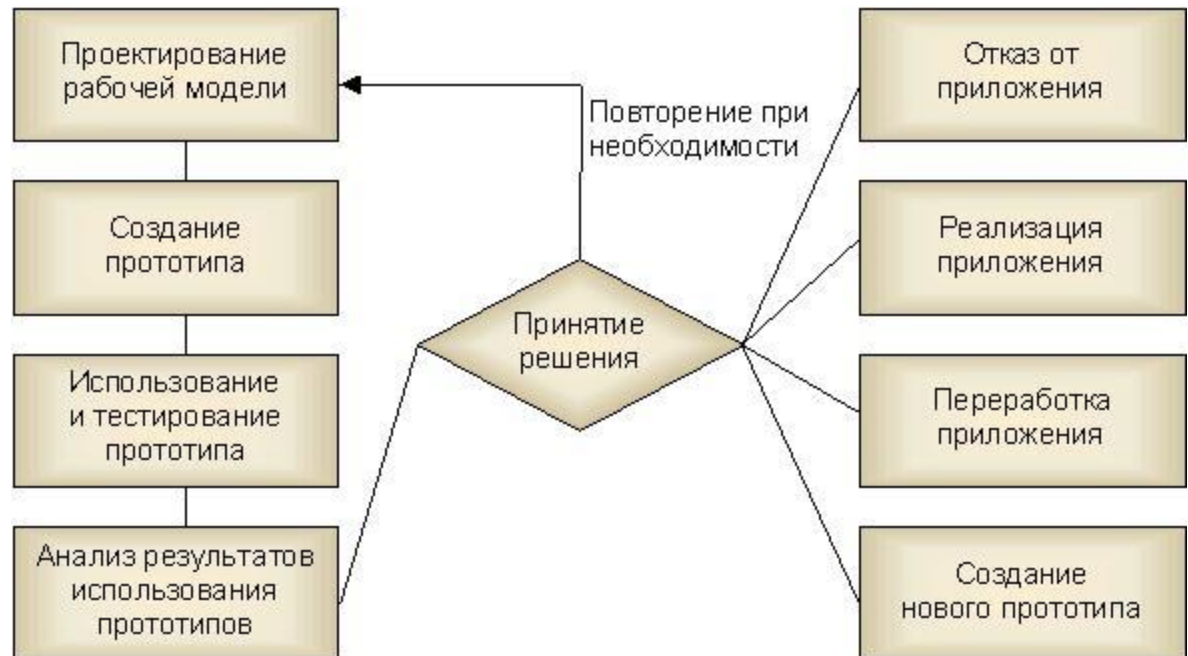
Жизненный цикл приложения баз данных

Выбор целевой СУБД – это выбор СУБД подходящего типа, предназначенной для поддержки создаваемого приложения базы данных.

Разработка приложений – это проектирование интерфейса пользователя и прикладных программ, предназначенных для работы с базой данных.

Создание прототипа – это создание рабочей модели приложения баз данных.

Прототип - это рабочая модель, которая обычно обладает лишь частью требуемых возможностей и не обеспечивает всей функциональности готовой системы.



Жизненный цикл приложения баз данных

Под **реализацией** понимается физическая реализация базы данных и разработанных приложений.

Под **конвертированием и загрузкой данных** понимается перенос любых существующих данных в новую базу данных и модификация любых существующих приложений с целью организации совместной работы с новой базой данных.

Тестирование – это процесс выполнения прикладных программ с целью поиска ошибок. ***Стратегии тестирования:***

- ☐ Нисходящее тестирование;
- ☐ Восходящее тестирование;
- ☐ Тестирование потоков;
- ☐ Интенсивное тестирование.

Под **эксплуатацией и сопровождением** понимают наблюдение за системой и поддержку ее нормального функционирования по окончании развертывания. ***Включает выполнение действий:***

- ❖ Контроль производительности системы.
- ❖ Сопровождение и модернизация (при необходимости) приложений баз данных.

Процедуры проектирования базы данных

Моделирование данных

(Критерии оценки модели данных)

Критерий	Пояснение
Структурная достоверность	Соответствие способу определения и организации информации на данном предприятии
Простота	Легкость понимания модели как профессионалами в области разработки информационных систем, так и обычными пользователями
Выразительность	Способность представлять отличия между разными типами данных, связей между данными и ограничений
Отсутствие избыточности	Исключение излишней информации, т. е. любая часть данных должна быть представлена только один раз
Способность к совместному использованию	Отсутствие принадлежности к какому-то особому приложению или технологии - следовательно, возможность использования модели во многих приложениях и технологиях
Расширяемость	Способность эволюционировать с целью включения новых требований с минимальным влиянием на уже существующих пользователей
Целостность	Согласованность со способом использования и управления информацией внутри предприятия
Представление в виде диаграмм	Способность представления модели с помощью понятных диаграммных обозначений

Процедуры проектирования базы данных

Концептуальное и Логическое проектирование

Концептуальное проектирование базы данных – это процесс создания модели используемой на предприятии информации, не зависящей от любых физических аспектов ее представления.

Логическое проектирование базы данных – это процесс создания модели используемой на предприятии информации с учетом выбранной модели организации данных, но независимо от типа целевой СУБД и других физических аспектов реализации.

Подходы создания глобальной логической модели:

- ❑ **Централизованный подход** – это слияние требований отдельных пользователей, выраженных в виде различных пользовательских представлений, в единый набор требований всех пользователей, который используется для создания глобальной логической модели данных.
- ❑ **Метод интеграции представления** – это слияние отдельных локальных логических моделей данных, отражающих представления различных групп пользователей, в единую глобальную логическую модель данных.

Процедуры проектирования базы данных

Физическое проектирование

Физическое проектирование данных – это процесс создания описания реализации базы данных на вторичных запоминающих устройствах с указанием структур хранения и методов доступа, используемых для организации эффективной обработки данных.

Основная цель физического проектирования базы данных состоит в описании способа физической реализации логического проекта базы данных.

Для реляционной модели это:

- ☐ создание набора реляционных таблиц и ограничений для них на основе информации, представленной в глобальной логической модели данных;
- ☐ определение конкретных структур хранения данных и методов доступа к ним, обеспечивающих оптимальную производительность системы с базой данных;
- ☐ разработка средств защиты создаваемой системы.

Проектирование приложения

Проектирование транзакций

Транзакция – это действие или последовательность действий, выполняемых одним и тем же пользователем (или прикладной программой), осуществляющим доступ к базе данных или изменение ее содержимого.

Цель проектирования транзакций заключается в определении и документировании высокоуровневых характеристик всех транзакций, которые должны будут выполняться в разрабатываемой базе данных.

Основные типы транзакций:

- **транзакции извлечения** используются для выборки некоторых данных с целью отображения их на экране или помещения в отчет.
- **транзакции обновления** используются для вставки новых, удаления старых или же изменения уже существующих записей базы данных.
- **смешанные транзакции** включают как операции извлечения, так и операции обновления данных

Проектирование приложения

Проектирование пользовательского интерфейса

рекомендации по созданию макетов любых форм и отчетов:

- содержательное название;
- ясные и понятные инструкции;
- логически обоснованные группировки и последовательности полей;
- визуально привлекательный вид окна формы или поля отчета;
- легко узнаваемые названия полей;
- согласованную терминологию и сокращения;
- согласованное использование цветов;
- визуальное выделение пространства и границ полей ввода данных;
- удобные средства перемещения курсора;
- средства исправления отдельных ошибочных символов и целых полей;
- средства вывода сообщений об ошибках при вводе недопустимых значений;
- особое выделение необязательных для ввода полей;
- средства вывода пояснительных сообщений с описанием полей;
- средства вывода сообщения об окончании заполнения формы.

Использование CASE-инструментов

Компоненты CASE-инструментов :

- ❑ словарь данных, предназначенный для хранения информации о данных, используемых в создаваемом приложении;
- ❑ инструменты проектирования, обеспечивающие проведение анализа данных;
- ❑ инструменты разработки корпоративной модели данных, а также концептуальных и логических моделей данных;
- ❑ инструменты, позволяющие создавать прототипы приложений.

Категории CASE-инструментов:

- CASE-инструменты высокого уровня применяются на начальных стадиях жизненного цикла разработки баз данных,
- CASE-инструменты низкого уровня – на более поздних, начиная со стадии реализации, в ходе тестирования и на протяжении всего процесса сопровождения функционирующей системы.
- Интегрированные CASE-инструменты применяются на всех стадиях жизненного цикла системы, а потому они должны поддерживать все функции CASE-инструментов как высокого, так и низкого уровней.

Выбор СУБД

Основные этапы процедуры выбора СУБД

➤ **Определение области компетенции проводимого изучения.**

На этом этапе устанавливаются границы компетенции проводимого исследования, его цели и степень охвата доступного материала, а также формулируются задачи, которые необходимо решать. Подготовленный по итогам анализа документ должен включать описание критериев оценки возможностей СУБД, предварительный список анализируемых продуктов, сведения о временных рамках исследования и т. п.

➤ **Сокращение списка претендентов до двух-трех продуктов.**

➤ **Оценка продуктов.**

Для оценки возможностей СУБД могут использоваться самые разнообразные параметры как в виде групп (определение данных, физические параметры, доступность, обработка транзакций, утилиты, средства разработки, другие параметры), так и по отдельности.

➤ **Проведение обоснованного выбора и подготовка отчета.**

Заключительным этапом выбора СУБД является документирование всего проведенного анализа, подготовка заключений с оценкой отобранных продуктов и выдача рекомендаций по выбору целевой СУБД проекта.

Рекомендуемые параметры оценки СУБД

Группа	Параметры в группе
Определение данных	Расширенная поддержка первичных ключей
	Определение внешних ключей
	Предусмотренные типы данных
	Расширяемость типов данных
	Определение доменов
	Простота реструктуризации
	Средства поддержки целостности данных
	Реализация механизма представлений
	Поддержка словаря данных
	Независимость от данных
	Тип базовой модели организации данных
	Поддержка эволюции схемы
Физические параметры	Предусмотренные файловые структуры
	Поддержка определения файловых структур
	Простота реорганизации
	Средства индексирования
	Поля/записи с переменной длиной
	Сжатие данных
	Возможности шифрования
	Требования к памяти
	Требования к устройствам хранения данных

Рекомендуемые параметры оценки СУБД

Группа	Параметры в группе
Определение данных	Расширенная поддержка первичных ключей
	Определение внешних ключей
	Предусмотренные типы данных
	Расширяемость типов данных
	Определение доменов
	Простота реструктуризации
	Средства поддержки целостности данных
	Реализация механизма представлений
	Поддержка словаря данных
	Независимость от данных
	Тип базовой модели организации данных
	Поддержка эволюции схемы
Физические параметры	Предусмотренные файловые структуры
	Поддержка определения файловых структур
	Простота реорганизации
	Средства индексирования
	Поля/записи с переменной длиной
	Сжатие данных
	Возможности шифрования
	Требования к памяти
	Требования к устройствам хранения данных

Рекомендуемые параметры оценки СУБД

Группа	Параметры в группе
Доступность	Язык запросов: совместимость со стандартами
	Интерфейс для других систем
	Интерфейс для языков третьего поколения
	Многопользовательский доступ
	Защиты базы данных: управление доступом к данным и поддержка механизма авторизации
Обработка транзакций	Процедуры резервного копирования и восстановления
	Поддержка контрольных точек
	Средства ведения системного журнала
	Поддерживаемый уровень детализации параллельности
	Возможные стратегии для решения тупиковых ситуаций
	Поддержка усовершенствованных моделей управления транзакциями
	Параллельная обработка запросов
Утилиты	Измерение производительности
	Настройка производительности базы данных
	Инструменты загрузки/выгрузки данных
	Контроль активности пользователей
	Поддержка процедур администрирования базы данных

Рекомендуемые параметры оценки СУБД

Группа	Параметры в группе	
Определение данных	Инструменты, использующие языки четвертого и пятого поколений	
	CASE-инструменты	
	Инструменты для работы с оконным интерфейсом	
	Поддержка хранимых процедур, триггеров и правил	
Физические параметры	Способность к модернизации	База пользователей
	Поддержка работы в сети	Производительность
	Поддержка работы в Internet	Утилиты репликации
	Используемые стандарты	Масштабируемость
	Переносимость	Стоимость
	Требуемая операционная система	
	Расширенная оптимизация запросов	
	Требуемое аппаратное обеспечение	
	Возможности распределенной работы	
	Поддержка двух- или трехуровневой архитектуры клиент/сервер	
	Взаимодействие с другими СУБД и прочими системами	
	Пропускная способность при обработке транзакций	
	Максимальное количество одновременно работающих пользователей	

Роли АД и АБД на этапах жизненного цикла

Этап	Основная роль	Вспомогательная роль
Планирование разработки базы данных	АД	АБД
Определение требований к системе	АД	АБД
Сбор и анализ требований пользователей	АД	АБД
Концептуальное проектирование базы данных	АД	АБД
Выбор целевой СУБД	АБД	АД
Логическое проектирование базы данных	АД	АБД
Разработка приложений	АБД	АД
Физическое проектирование базы данных	АБД	АД
Создание прототипов	АБД	АД
Реализация	АБД	АД
Конвертирование и загрузка данных	АБД	АД
Тестирование	АБД	АД
Эксплуатация и сопровождение	АБД	АД

Администрирование данных

Администрирование данных – это управление информационными ресурсами, включая планирование базы данных, разработку и внедрение стандартов, определение ограничений и процедур, а также концептуальное и логическое проектирование баз данных.

Основные задачи администрирования данных:

- выбор подходящих инструментов разработки;
- помощь в разработке корпоративных стратегий построения информационной системы, развития информационных технологий и бизнес-стратегий;
- предварительная оценка осуществимости и планирование процесса создания базы данных;
- разработка корпоративной модели данных;
- определение требований организации к используемым данным;
- определение стандартов сбора данных и выбор формата их представления;
- оценка объемов данных и вероятности их роста;
- определение способов и интенсивности использования данных;

Администрирование данных

Основные задачи администрирования данных (продолжение):

- определение правил доступа к данным и мер безопасности, соответствующих правовым нормам и внутренним требованиям организации;
- концептуальное и логическое проектирование базы данных;
- взаимодействие с АБД и разработчиками приложений с целью обеспечения соответствия создаваемых приложений всем существующим требованиям;
- обучение пользователей – изучение существующих стандартов обработки данных и юридической ответственности за их некорректное применение;
- постоянная модернизация используемых информационных систем и технологий по мере развития бизнес-процессов;
- обеспечение полноты всей требуемой документации, включая корпоративную модель, стандарты, ограничения, процедуры, использование словаря данных, а также элементы управления работой конечных пользователей;
- поддержка словаря данных организации;
- взаимодействие с конечными пользователями для определения новых требований и разрешения проблем, связанных с доступом к данным и недостаточной производительностью их обработки.

Администрирование базы данных

Администрирование базы данных – это управление физической реализацией приложений баз данных: физическое проектирование базы данных и ее реализация, организация поддержки целостности и защиты данных, наблюдение за текущим уровнем производительности системы, а также реорганизация базы данных по мере необходимости.

Основные задачи администрирования базы данных:

- оценка и выбор целевой СУБД;
- физическое проектирование базы данных;
- реализация физического проекта базы данных в среде целевой СУБД;
- определение требований защиты и поддержки целостности данных;
- взаимодействие с разработчиками приложений баз данных;
- разработка стратегии тестирования;
- обучение пользователей;
- ответственность за сдачу в эксплуатацию готового приложения базы данных;

Администрирование базы данных

Основные задачи администрирования базы данных (продолжение):

- контроль текущей производительности системы и соответствующая настройка базы данных;
- регулярное резервное копирование;
- разработка требуемых механизмов и процедур восстановления;
- обеспечение полноты используемой документации, включая материалы, разработанные внутри организации;
- поддержка актуальности используемого программного и аппаратного обеспечения, включая заказ и установку пакетов обновлений в случае необходимости.

Основные отличия в задачах, выполняемых АД и АБД

Администрирование данных	Администрирование базы данных
Участствует в стратегическом планировании информационной системы организации	Оценивает новые СУБД
Определяет долгосрочные цели	Выполняет планы достижения целей
Применяет стандарты, стратегии и процедуры	Применяет стандарты, стратегии и процедуры
Определяет требования к данным	Реализует требования к данным
Выполняет концептуальное и логическое проектирование базы данных	Выполняет логическое и физическое проектирование базы данных
Разрабатывает и сопровождает корпоративную модель данных	Реализует физической проект базы данных
Координирует разработку системы	Выполняет текущий контроль и управление базой данных
Управленческая направленность	Техническая направленность
Работа АД не зависит от типа целевой СУБД	Работа АД зависит от типа целевой СУБД

Функциональные зависимости

Пусть **R** – это отношение, а **X** и **Y** – произвольные подмножества атрибутов отношения **R**.

Y функционально зависит от **X**, тогда и только тогда, когда каждое значение множества **X** отношения **R** связано в точности с одним значением множества **Y** отношения **R**.

Обозначается $X \rightarrow Y$ (читается как «**X функционально определяет Y**»),

S#	CITY	P#	QTY
S1	London	P1	100
S1	London	P2	100
S2	Paris	P1	200
S2	Paris	P2	200
S3	Paris	P2	300
S4	London	P2	400
S4	London	P4	400
S4	London	P5	400

Пример:

$(S\#) \rightarrow (CITY)$

Также

$(S\#, P\#) \rightarrow (QTY)$

$(S\#, P\#) \rightarrow (CITY)$

и д.р.

Функциональная зависимость **тривиальна** тогда и только тогда, когда правая часть символической записи данной функциональной зависимости является подмножеством (не обязательно собственным) левой части.

Замыкание множества зависимостей

Некоторые ФЗ могут означать другие ФЗ. Например, приведенная функциональная зависимость: $(S\#, P\#) \rightarrow (CITY, QTY)$

подразумевает следующие ФЗ: $(S\#, P\#) \rightarrow CITY$ $(S\#, P\#) \rightarrow QTY$

Множество всех ФЗ, которые задаются данным множеством функциональных зависимостей S , называется замыканием S и обозначается символом S^+

Правила вывода Армстронга

1. **Рефлексивность:** если B является подмножеством A , то $A \rightarrow B$.
2. **Дополнение:** если $A \rightarrow B$, то $AC \rightarrow BC$.
3. **Транзитивность:** если $A \rightarrow B$ и $B \rightarrow C$, то $A \rightarrow C$.
4. **Самоопределение:** $A \rightarrow A$.
5. **Декомпозиция:** если $A \rightarrow BC$, то $A \rightarrow B$ и $A \rightarrow C$.
6. **Объединение:** если $A \rightarrow B$ и $A \rightarrow C$, то $A \rightarrow BC$.
7. **Композиция:** если $A \rightarrow B$ и $C \rightarrow D$, то $AC \rightarrow BD$.
8. **Теорема всеобщего объединения.**

Если $A \rightarrow B$ и $C \rightarrow D$, то $A \cup (C - B) \rightarrow BD$ (символ « $\cup$ » обозначает объединение множеств, а символ « $-$ » — их разность).

Неприводимое множество зависимостей

Пусть $S1$ и $S2$ являются двумя множествами ФЗ. Если любая ФЗ, которая является зависимостью множества $S1$, также является зависимостью множества $S2$, т. е. $S1^+$ является подмножеством $S2^+$, то $S2$ называется **покрытием** для $S1$.

Если $S2$ является покрытием для $S1$, а $S1$ – покрытием для $S2$, т. е. если $S1^+ = S2^+$, то $S1$ и $S2$ **эквивалентны**..

Множество ФЗ называется **неприводимым** тогда и только тогда, когда выполняются свойства:

1. Правая часть (зависимая часть) каждой ФЗ множества S содержит только один атрибут (т. е. является одноэлементным множеством);
2. Левая часть (детерминант) каждой ФЗ множества S является неприводимой, т. е. ни один атрибут не может быть опущен из детерминанта без изменения замыкания S^+ . В таком случае ФЗ является **неприводимой слева**;
3. Ни одна функциональная зависимость в S не может быть опущена из S без изменения замыкания S^+

Избыточность данных и аномалии обновления

Таблица 1

<u>staffNo</u>	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Таблица 2

<u>branchNo</u>	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

Таблица 3

<u>staffNo</u>	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Нормализация

Нормализация – это метод создания набора отношений с заданными свойствами на основе требований к данным, установленным в некоторой организации.

Ненормализованная форма – это таблица, содержащая одну или несколько повторяющихся групп данных.



Нормализация

Первая нормальная форма

Первая нормальная форма – это отношение, в котором все используемые домены содержат только скалярные значения.

- запрещает повторяющиеся столбцы (содержащие одинаковую по смыслу информацию)
- запрещает множественные столбцы (содержащие значения типа списка и т.п.)
- требует определить первичный ключ для таблицы, то есть тот столбец или комбинацию столбцов, которые однозначно определяют каждую строку

Наим.	Город	Адрес	Эл. почта	WWW	Вид	Конт. лица
Поршневой з-д	Владимир	Ул. 2-я Кольцевая, 17	info@plunger.ru	www.plunger.ru	Поставщик	Иванов И.И., зам. дир., тел (3254) 76- 15-95 Петров П.П., нач. отд. сбыта, тел (3254) 76-15-35
ООО Вымпел	Курск	Ул. Гоголя, 25	pennon@mail.ru		Клиент	Сидоров С.С., директор, тел. (7634) 66-65-38
ИЧП Альфа	Владимир	Ул. Пушкинская, 37, оф. 565	alpha323@list.ru		Клиент	Васильев В.В., директор, тел (3254) 74-57-45

Нормализация

Пример 1НФ

Наим.	Город	Адрес	Эл. почта	WWW	Вид	Конт. лица
Поршневой з-д	Владимир	Ул. 2-я Кольцевая, 17	info@plunger.ru	www.plunger.ru	Поставщик	Иванов И.И., зам. дир., тел (3254) 76-15-95
Поршневой з-д	Владимир	Ул. 2-я Кольцевая, 17	info@plunger.ru	www.plunger.ru	Поставщик	Петров П.П., нач. отд. сбыта, тел (3254) 76-15-35
ООО Вымпел	Курск	Ул. Гоголя, 25	pennon@mail.ru		Клиент	Сидоров С.С., директор, тел. (7634) 66-65-38
ИЧП Альфа	Владимир	Ул. Пушкинская, 37, оф. 565	alpha323@list.ru		Клиент	Васильев В.В., директор, тел (3254) 74-57-45

Нормализация

Пример 1НФ

Наим.	Город	Адрес	Эл. почта	WWW	Вид	Должнос ть	Ф.И.О.	Код города	Тел.
Поршне вой з-д	Владими р	Ул. 2-я Кольцевая , 17	info@plunger.ru	www.plunger.ru	Постав щик	зам. дир.	Иванов И.И.	3254	76-15-95
Поршне вой з-д	Владими р	Ул. 2-я Кольцевая , 17	info@plunger.ru	www.plunger.ru	Постав щик	нач. отд. сбыта	Петров П.П.,	3254	76-15-35
ООО Вымпел	Курск	Ул. Гоголя, 25	pennon@mail.ru		Клиент	директор	Сидоров С.С.	7634	66-65-38
ИЧП Альфа	Владими р	Ул. Пушкинск ая, 37, оф. 565	alpha323@list.ru		Клиент	директор	Васильев В.В.	3254	74-57-45

Нормализация

2НФ

Вторая нормальная форма (2НФ) основана на понятии полной функциональной зависимости:

В некотором отношении атрибут В называется **полностью функционально зависимым** от атрибута А, если атрибут В функционально зависит от полного значения атрибута А и не зависит ни от какого подмножества полного значения атрибута А.

Функциональная зависимость $A \rightarrow B$ является **полной** функциональной зависимостью, если удаление какого-либо атрибута из А приводит к утрате этой зависимости.

Частичной функциональной зависимостью называется такая зависимость $A \rightarrow B$, если в А есть некий атрибут, при удалении которого эта зависимость сохраняется.

Вторая нормальная форма – это отношение, которое находится в первой нормальной форме и каждый атрибут которого, не входящий в состав первичного ключа, характеризуется полной функциональной зависимостью от этого первичного ключа

Нормализация

Пример 2НФ

Наим.	Город	Адрес	Эл. почта	WWW	Вид	Должност	Ф.И.О.	Тел.
Поршнево й 3-д	Владимир	Ул. 2-я Кольцевая , 17	info@plunger.ru	www.plunger.ru	Поставщик	зам. дир.	Иванов И.И.	76-15-95
Поршнево й 3-д	Владимир	Ул. 2-я Кольцевая , 17	info@plunger.ru	www.plunger.ru	Поставщик	нач. отд. сбыта	Петров П.П.,	76-15-35
ООО Вымпел	Курск	Ул. Гоголя, 25	pennon@mail.ru		Клиент	директор	Сидоров С.С.	66-65-38
ИЧП Альфа	Владимир	Ул. Пушкинс кая, 37, оф. 565	alpha323@list.ru		Клиент	директор	Васильев В.В.	74-57-45

Город	Код города
Владимир	3254
Курск	7634

Нормализация

ЗНФ

Транзитивная зависимость является описанием такого типа функциональной зависимости, которая возникает при наличии следующих функциональных зависимостей между атрибутами А, В и С:

$$A \rightarrow B \text{ и } B \rightarrow C$$

В данном случае транзитивная зависимость $A \rightarrow C$ осуществляется через атрибут В. Это утверждение справедливо только в том случае, если атрибут А функционально не зависит от атрибутов В и С.

Третья нормальная форма – это отношение, которое находится в первой и второй нормальных формах и не имеет не входящих в первичный ключ атрибутов, которые находились бы в транзитивной функциональной зависимости от этого первичного ключа.

Нормализация

Пример ЗНФ

Наим.	Город	Адрес	Эл. почта	WWW	Вид
Поршневой з-д	Владимир	Ул. 2-я Кольцевая, 17	info@plunger.ru	www.plunger.ru	Поставщик
ООО Вымпел	Курск	Ул. Гоголя, 25	pennon@mail.ru		Клиент
ИЧП Альфа	Владимир	Ул. Пушкинская, 37, оф. 565	alpha323@list.ru		Клиент

Наим.	Город	Ф.И.О.	Должность	Тел.
Поршневой з-д	Владимир	Иванов И.И.	зам. дир.	76-15-95
Поршневой з-д	Владимир	Петров П.П.,	нач. отд. сбыта	76-15-35
ООО Вымпел	Курск	Сидоров С.С.	директор	66-65-38
ИЧП Альфа	Владимир	Васильев В.В.	директор	74-57-45

Город	Код города
Владимир	3254
Курск	7634

Нормализация

Нормальная форма Бойса – Кодда

Начальное определение Э. Ф. Кодда для 3НФ не совсем подходит для отношений с перечисленными ниже условиями:

- а) отношение имеет два (или более) потенциальных ключа;
- б) два потенциальных ключа являются сложными;
- с) потенциальные ключи перекрываются (т. е. имеют, по крайней мере, один общий атрибут)

Отношение находится в **нормальной форме Бойса – Кодда** тогда и только тогда, когда каждая нетривиальная и неприводимая слева ФЗ обладает потенциальным ключом в качестве детерминанта.

Нормализация

Четвертая нормальная форма

Отношение R находится в **четвертой нормальной форме** тогда и только тогда, когда существуют такие подмножества A и B атрибутов отношения R , что не выполняется (нетривиальная) многозначная зависимость $A \twoheadrightarrow B$. Тогда все атрибуты отношения R также функционально зависят от атрибута A .

Пусть A , B и C являются произвольными подмножествами множества атрибутов отношения R . Тогда B **многозначно зависит** от A , тогда и только тогда, когда множество значений B , соответствующее заданной паре (значение A , значение C) отношения R , зависит только от A , но не зависит от C .

Символически выражается записью:

$A \twoheadrightarrow B$

читается как « A многозначно определяет B » или « A двойная стрелка B »

Нормализация

Пример 4НФ

Исходная таблица		
<u>Ресторан</u>	<u>Вид пиццы</u>	<u>Район доставки</u>
A1 Pizza	Thick Crust	Springfield
A1 Pizza	Thick Crust	Shelbyville
A1 Pizza	Thick Crust	Capital City
A1 Pizza	Stuffed Crust	Springfield
A1 Pizza	Stuffed Crust	Shelbyville
A1 Pizza	Stuffed Crust	Capital City
Elite Pizza	Thin Crust	Capital City
Elite Pizza	Stuffed Crust	Capital City
Vincenzo's Pizza	Thick Crust	Springfield
Vincenzo's Pizza	Thick Crust	Shelbyville
Vincenzo's Pizza	Thin Crust	Springfield
Vincenzo's Pizza	Thin Crust	Shelbyville

<u>Ресторан</u>	<u>Вид пиццы</u>
A1 Pizza	Thick Crust
A1 Pizza	Stuffed Crust
Elite Pizza	Thin Crust
Elite Pizza	Stuffed Crust
Vincenzo's Pizza	Thick Crust
Vincenzo's Pizza	Thin Crust

<u>Ресторан</u>	<u>Район доставки</u>
A1 Pizza	Springfield
A1 Pizza	Shelbyville
A1 Pizza	Capital City
Elite Pizza	Capital City
Vincenzo's Pizza	Springfield
Vincenzo's Pizza	Shelbyville

Такая таблица не соответствует 4NF, так как существует многозначная зависимость:

- {Ресторан} – >> {Вид пиццы}
- {Ресторан} – >> {Район доставки}

Нормализация

Декомпозиций

Декомпозицией отношения R называется замена R на совокупность отношений $\{R_1, R_2, \dots, R_n\}$ такую, что каждое из них есть проекция R , и каждый атрибут R входит хотя бы в одну из проекций декомпозиции.

R		
a	b	c
Москва	Россия	столица
Томск	Россия	не столица
Берлин	Германия	столица

варианты декомпозиции:

$\{a\}, \{b\}, \{c\}$

$\{a\}, \{b, c\}$

$\{a, b\}, \{c\}$

$\{b\}, \{a, c\}$

$\{a, b\}, \{b, c\}$

$\{a, b\}, \{a, c\}$

$\{b, c\}, \{a, c\}$

$\{a, b\}, \{b, c\}, \{a, c\}$

Нормализация

Пример декомпозиций

R		
a	b	c
Москва	Россия	столица
Томск	Россия	не столица
Берлин	Германия	столица

R1	R2	
a	b	c
Москва	Россия	столица
Томск	Россия	не столица
Берлин	Германия	столица

R' = R1 NATURAL JOIN R2		
a	b	c
Москва	Россия	столица
Москва	Россия	не столица
Москва	Германия	столица
Томск	Россия	столица
Томск	Россия	не столица
Томск	Германия	столица
Берлин	Россия	столица
Берлин	Россия	не столица
Берлин	Германия	столица

Нормализация

Пример декомпозиций

R		
a	b	c
Москва	Россия	столица
Томск	Россия	не столица
Берлин	Германия	столица

R1	
a	b
Москва	Россия
Томск	Россия
Берлин	Германия

R2	
a	c
Москва	столица
Томск	не столица
Берлин	столица

R' = R1 NATURAL JOIN R2		
a	b	c
Москва	Россия	столица
Томск	Россия	не столица
Берлин	Германия	столица

Нормализация

Пятая нормальная форма

Пусть R является отношением, а $A, B, \dots, Z$ – произвольные подмножества множества атрибутов отношения R . Отношение R удовлетворяет **зависимости соединения** $*$ ($A, B, \dots, Z$) тогда и только тогда, когда оно равносильно соединению своих проекций с подмножествами атрибутов $A, B, \dots, Z$.

Декомпозиция называется декомпозицией с **соединением без потерь** (или беспроигрышным, или неаддитивным соединением), когда при ее выполнении сохраняются все данные исходного отношения, а также исключается создание дополнительных ложных строк.

Отношение R находится в **пятой нормальной форме** (5НФ) тогда и только тогда, когда каждая зависимость соединения в отношении R подразумевается потенциальными ключами отношения R .

Нормализация

Пример 5НФ исходные данные

Ассортимент (продавцы, фирмы, товары)		
<u>Продавец</u>	<u>Фирма</u>	<u>Товар</u>
Иванов	Рога и Копыта	Пылесос
Иванов	Рога и Копыта	Хлебница
Петров	Безенчук&Ко	Сучкорез
Петров	Безенчук&Ко	Пылесос
Петров	Безенчук&Ко	Хлебница
Петров	Безенчук&Ко	Зонт
Сидоров	Безенчук&Ко	Пылесос
Сидоров	Безенчук&Ко	Телескоп
Сидоров	Рога и Копыта	Пылесос
Сидоров	Рога и Копыта	Лампа
Сидоров	Геркулес	Вешалка

есть нетривиальная зависимость соединения

*{{Продавец, Фирма}, {Фирма, Товар}, {Продавец, Товар}},

Нормализация

Пример 5НФ Разбиение

Товары продавцов	
<u>Продавец</u>	<u>Товар</u>
Иванов	Пылесос
Иванов	Хлебница
Петров	Сучкорез
Петров	Пылесос
Петров	Хлебница
Петров	Зонт
Сидоров	Телескоп
Сидоров	Пылесос
Сидоров	Лампа
Сидоров	Вешалка

Фирмы продавцов	
<u>Продавец</u>	<u>Фирма</u>
Иванов	Рога и Копыта
Петров	Безенчук&Ко
Сидоров	Безенчук&Ко
Сидоров	Рога и Копыта
Сидоров	Геркулес

Товары фирм	
<u>Фирма</u>	<u>Товар</u>
Рога и Копыта	Пылесос
Рога и Копыта	Хлебница
Рога и Копыта	Лампа
Безенчук&Ко	Сучкорез
Безенчук&Ко	Пылесос
Безенчук&Ко	Хлебница
Безенчук&Ко	Зонт
Безенчук&Ко	Телескоп
Геркулес	Вешалка

Нормализация

Другие нормальные формы

Доменно-ключевая нормальная форма (ДКНФ). Отношение R находится в ДКНФ тогда и только тогда, когда каждое ограничение, наложенное на отношение R, является логическим следствием *ограничений доменов* и *ограничений ключей*, наложенных на отношение R.

Ограничение домена – это ограничение на использование значений для данного атрибута только из некоторого предписанного домена.

Ограничение ключа – это ограничение на то, что некоторый атрибут или некоторая комбинация атрибутов представляют собой потенциальный ключ.

Нормальная форма типа “выборка – объединение” предполагает возможность декомпозиции на основе **выборки** по строкам. При этом оператором декомпозиции является *непересекающаяся выборка*, а соответствующим оператором композиции – *непересекающееся объединение*.

Нормализация

Пример 5НФ Разбиение

Товары продавцов	
<u>Продавец</u>	<u>Товар</u>
Иванов	Пылесос
Иванов	Хлебница
Петров	Сучкорез
Петров	Пылесос
Петров	Хлебница
Петров	Зонт
Сидоров	Телескоп
Сидоров	Пылесос
Сидоров	Лампа
Сидоров	Вешалка

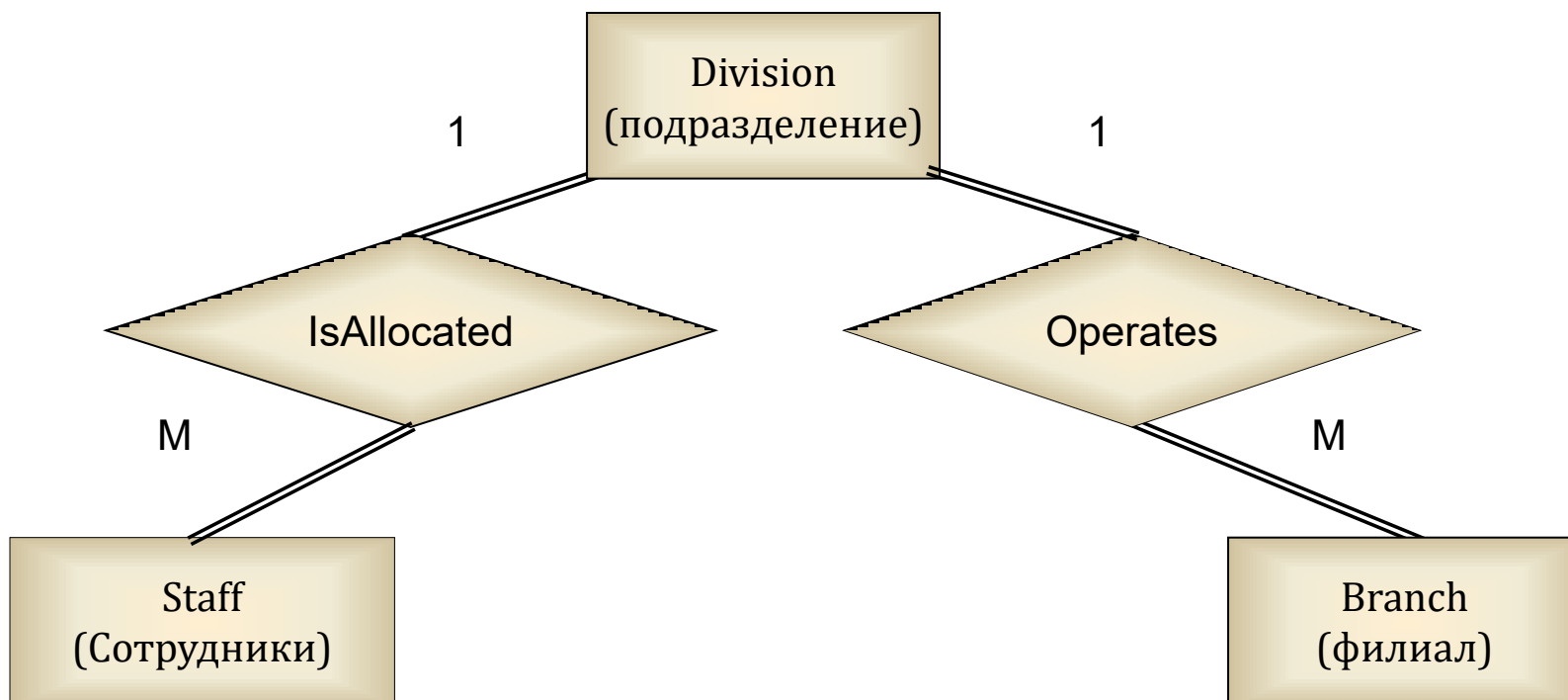
Фирмы продавцов	
<u>Продавец</u>	<u>Фирма</u>
Иванов	Рога и Копыта
Петров	Безенчук&Ко
Сидоров	Безенчук&Ко
Сидоров	Рога и Копыта
Сидоров	Геркулес

Товары фирм	
<u>Фирма</u>	<u>Товар</u>
Рога и Копыта	Пылесос
Рога и Копыта	Хлебница
Рога и Копыта	Лампа
Безенчук&Ко	Сучкорез
Безенчук&Ко	Пылесос
Безенчук&Ко	Хлебница
Безенчук&Ко	Зонт
Безенчук&Ко	Телескоп
Геркулес	Вешалка

Модель «Сущность – Связь»

Проблемы ER-моделирования

Ловушка разветвления имеет место в том случае, когда модель отображает связь между типами сущностей, но путь между отдельными сущностями этого типа определен неоднозначно.

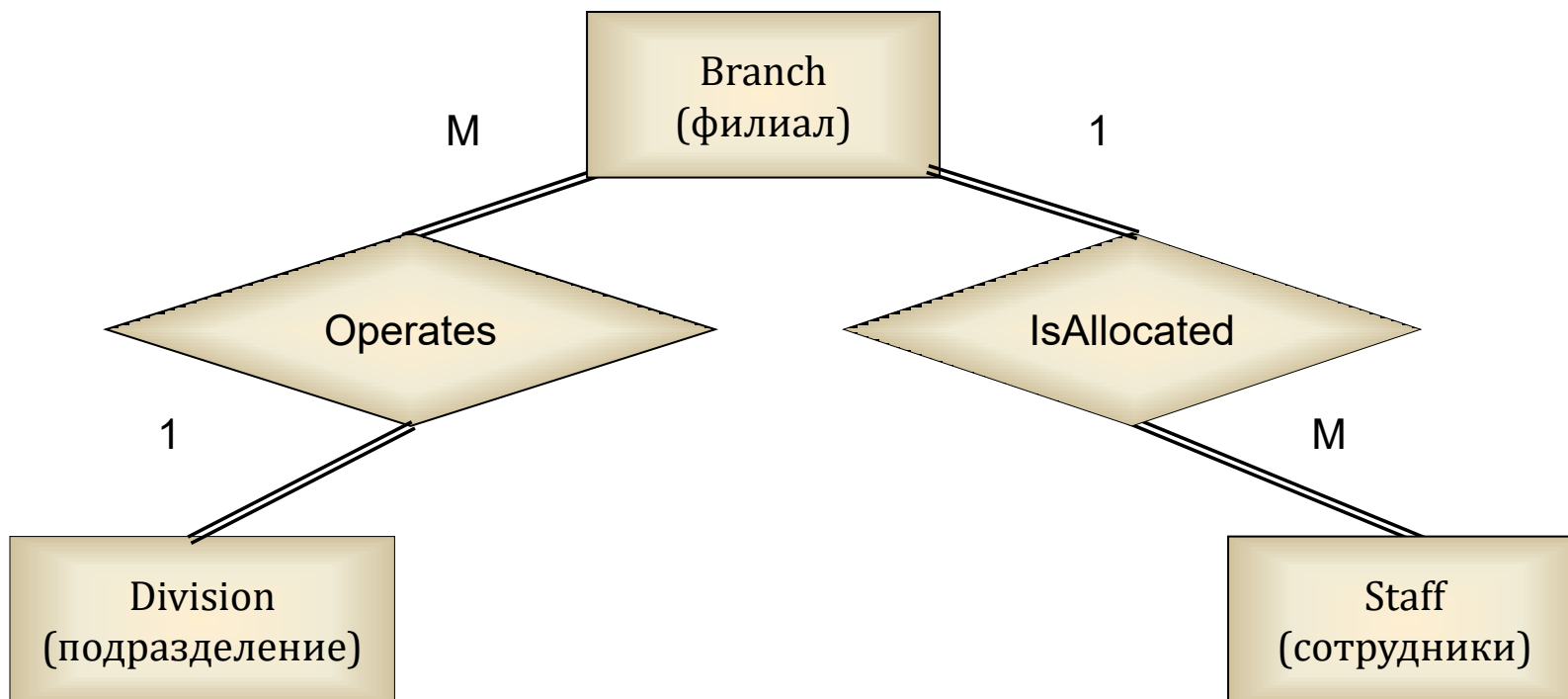


Пример ловушки разветвления

Модель «Сущность – Связь»

Проблемы ER-моделирования

Ловушка разветвления имеет место в том случае, когда модель отображает связь между типами сущностей, но путь между отдельными сущностями этого типа определен неоднозначно.

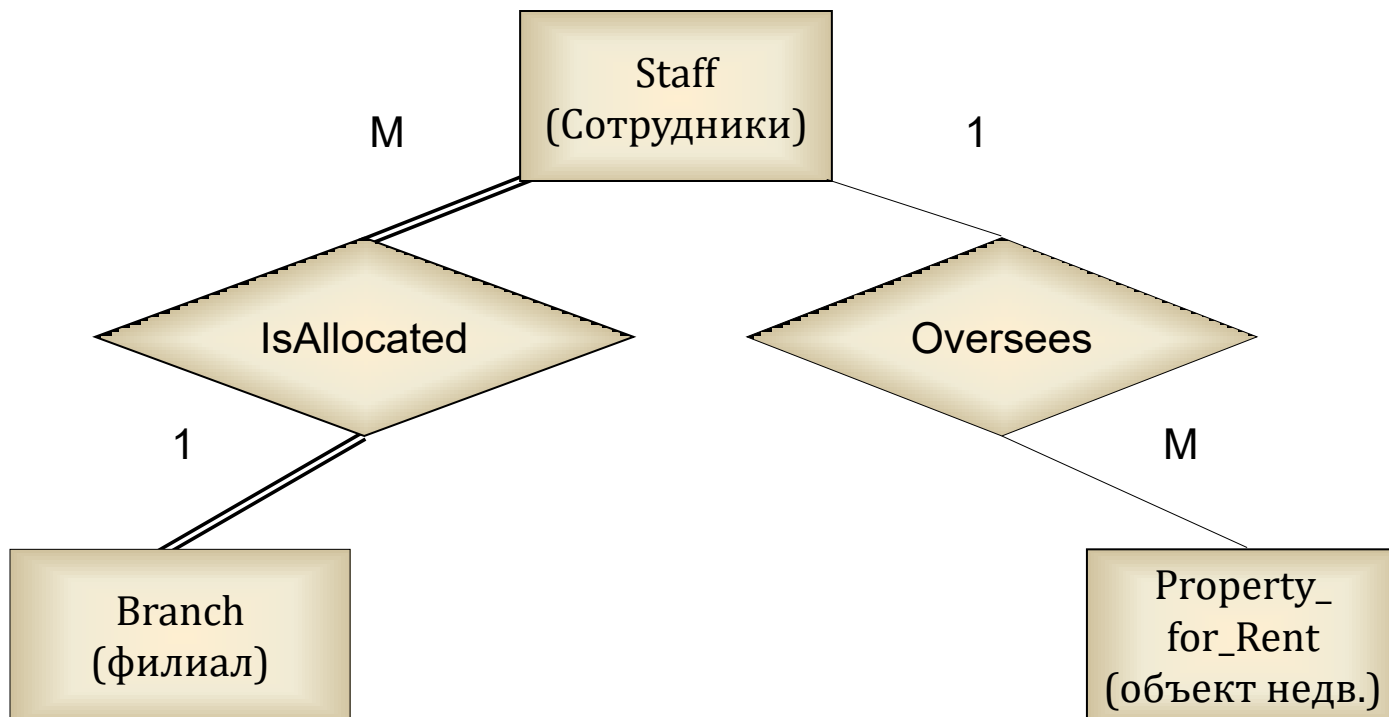


Пример преобразования ER-модели с целью устранения ловушки разветвления

Модель «Сущность – Связь»

Проблемы ER-моделирования

Ловушка разрыва появляется в том случае, когда в модели предполагается наличие связи между типами сущностей, но не существует пути между отдельными сущностями этих типов.

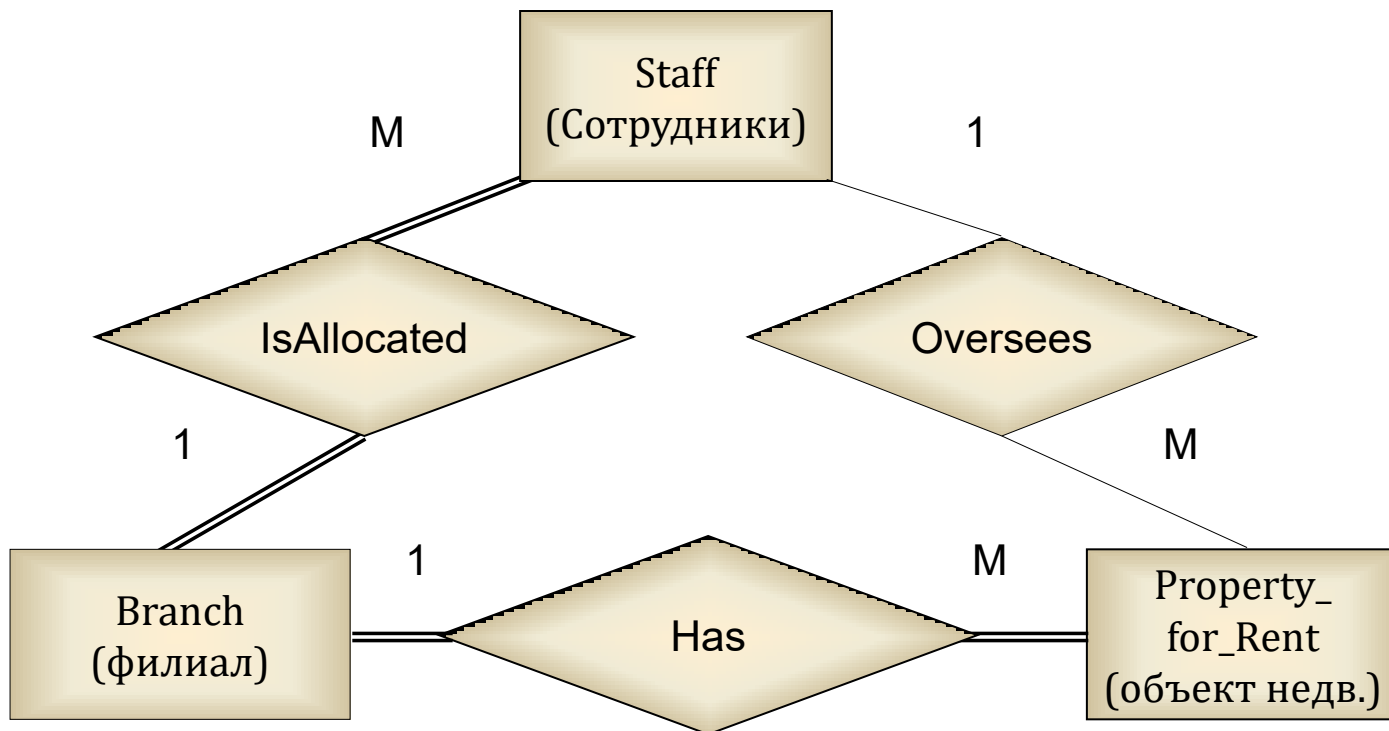


Пример ловушки разрыва

Модель «Сущность – Связь»

Проблемы ER-моделирования

Ловушка разрыва появляется в том случае, когда в модели предполагается наличие связи между типами сущностей, но не существует пути между отдельными сущностями этих типов.



Пример преобразования ER-модели с целью устранения ловушки разрыва

Введение в методологию проектирования БД

Методология проектирования – это структурированный подход, предусматривающий использование специализированных процедур, технических приемов, инструментов, документации и направленный на поддержку и упрощение процесса проектирования.

Концептуальное проектирование базы данных – это процедура конструирования информационной модели предприятия, не зависящей от каких-либо физических условий реализации.

Логическое проектирование базы данных – это процедура конструирования информационной модели предприятия на основе существующих конкретных моделей данных, не зависящей от используемой СУБД и других физических условий реализации.

Физическое проектирование базы данных – это процесс создания описания конкретной реализации базы данных, размещаемой во вторичной памяти. Предусматривает описание структуры хранения данных и методов доступа, предназначенных для осуществления наиболее эффективного доступа к информации.

Процедуры проектирования базы данных

Концептуальное проектирование

Этап 1. Создание локальной концептуальной модели данных исходя из представлений о предметной области каждого из типов пользователей.

Этап 1.1. Определение типов сущностей.

Этап 1.2. Определение типов связей.

Этап 1.3. Определение атрибутов и связывание их с типами сущностей и связей.

Этап 1.4. Определение доменов атрибутов.

Этап 1.5. Определение атрибутов, являющихся потенциальными и первичными ключами.

Этап 1.6. Специализация или генерализация типов сущностей (необязательный этап).

Этап 1.7. Создание диаграммы «сущность – связь».

Этап 1.8. Обсуждение локальных концептуальных моделей данных с конечными пользователями.

Процедуры проектирования базы данных

Логическое проектирование

Этап 2. Построение и проверка локальной логической модели данных на основе представления о предметной области каждого из типов пользователей.

Этап 2.1. Преобразование локальной концептуальной модели данных в локальную логическую модель.

Этап 2.2. Определение набора отношений исходя из структуры локальной логической модели данных.

Этап 2.3. Проверка модели с помощью правил нормализации.

Этап 2.4. Проверка модели с помощью транзакций пользователей.

Этап 2.5. Создание диаграмм «сущность – связь».

Этап 2.6. Определение требований поддержки целостности данных.

Этап 2.7. Обсуждение разработанных локальных логических моделей данных с конечными пользователями.

Процедуры проектирования базы данных

Логическое проектирование

Этап 3. Создание и проверка глобальной логической модели данных.

Этап 3.1. Слияние локальных логических моделей данных в единую глобальную модель данных.

Этап 3.2. Проверка глобальной логической модели данных.

Этап 3.3. Проверка возможностей расширения модели в будущем.

Этап 3.4. Создание окончательного варианта диаграммы «сущность – связь».

Этап 3.5. Обсуждение глобальной логической модели данных с пользователями.

Процедуры проектирования базы данных

Физическое проектирование

Этап 4. Перенос глобальной логической модели данных в среду целевой СУБД.

Этап 4.1. Проектирование основных таблиц в среде целевой СУБД.

Этап 4.2. Реализация бизнес-правил предприятия в среде целевой СУБД.

Этап 5. Проектирование физического представления базы данных.

Этап 5.1. Анализ транзакций.

Этап 5.2. Выбор файловой структуры.

Этап 5.3. Определение вторичных индексов.

Этап 5.4. Анализ необходимости введения контролируемой избыточности данных.

Этап 5.5. Определение требований к дисковой памяти.

Этап 6. Разработка механизмов защиты.

Этап 6.1. Разработка пользовательских представлений (видов).

Этап 6.2. Определение прав доступа.

Этап 7. Организация мониторинга и настройка функционирования системы.

ОСНОВЫ SQL

Основные операторы SQL

Инструкция	Описание
Обработка данных	
SELECT	Извлекает данные из таблицы (таблиц)
INSERT	Добавляет новые строки а таблицу
DELETE	Удаляет строки из таблицы
UPDATE	Обновляет данные, существующие в таблице
Определение данных	
CREATE TABLE	Добавляет новую таблицу в базу данных
DROP TABLE	Удаляет таблицу
ALTER TABLE	Изменяет структуру существующей таблицы
CREATE VIEW	Добавляет новое представление в базу данных
DROP VIEW	Удаляет представление
CREATE INDEX	Создает индекс для столбца
DROP INDEX	Удаляет индекс столбца
CREATE SCHEMA	Добавляет новую схему в базу данных
DROP SCHEMA	Удаляет схему
CREATE DOMAIN	Добавляет новый домен в базу данных
ALTER DOMAIN	Изменяет определение домена
DROP DOMAIN	Удаляет домен

Основы SQL

Основные операторы SQL

Инструкция	Описание
Управление доступом	
GRANT	Предоставляет пользователю определенную привилегию доступа
REVOKE	Отменяет указанную привилегию доступа
Управление транзакциями	
COMMIT	Завершает текущую транзакцию
ROLLBACK	Отменяет текущую транзакцию
SET TRANSACTION	Определяет режим доступа к данным для текущей транзакции
Программный SQL	
DECLARE	Определяет набор записей, в который будут возвращены результаты запроса
OPEN	Открывает набор записей
FETCH	Извлекает строку из таблицы результатов запроса
CLOSE	Закрывает набор записей
PREPARE	Подготавливает оператор SQL к динамическому выполнению
EXECUTE	Динамически выполняет оператор SQL
DESCRIBE	Возвращает описание подготовленного запроса

Основы SQL

Структура оператора SQL



Рекомендуемые правила при написании SQL запросов:

- каждая фраза в операторе должна начинаться с новой строки;
- начало каждой фразы должно быть выровнено с началом остальных фраз оператора;
- если фраза имеет несколько частей, каждая из них должна начинаться с новой строки с некоторым отступом относительно начала фразы, что будет указывать на их подчиненность.

Основы SQL

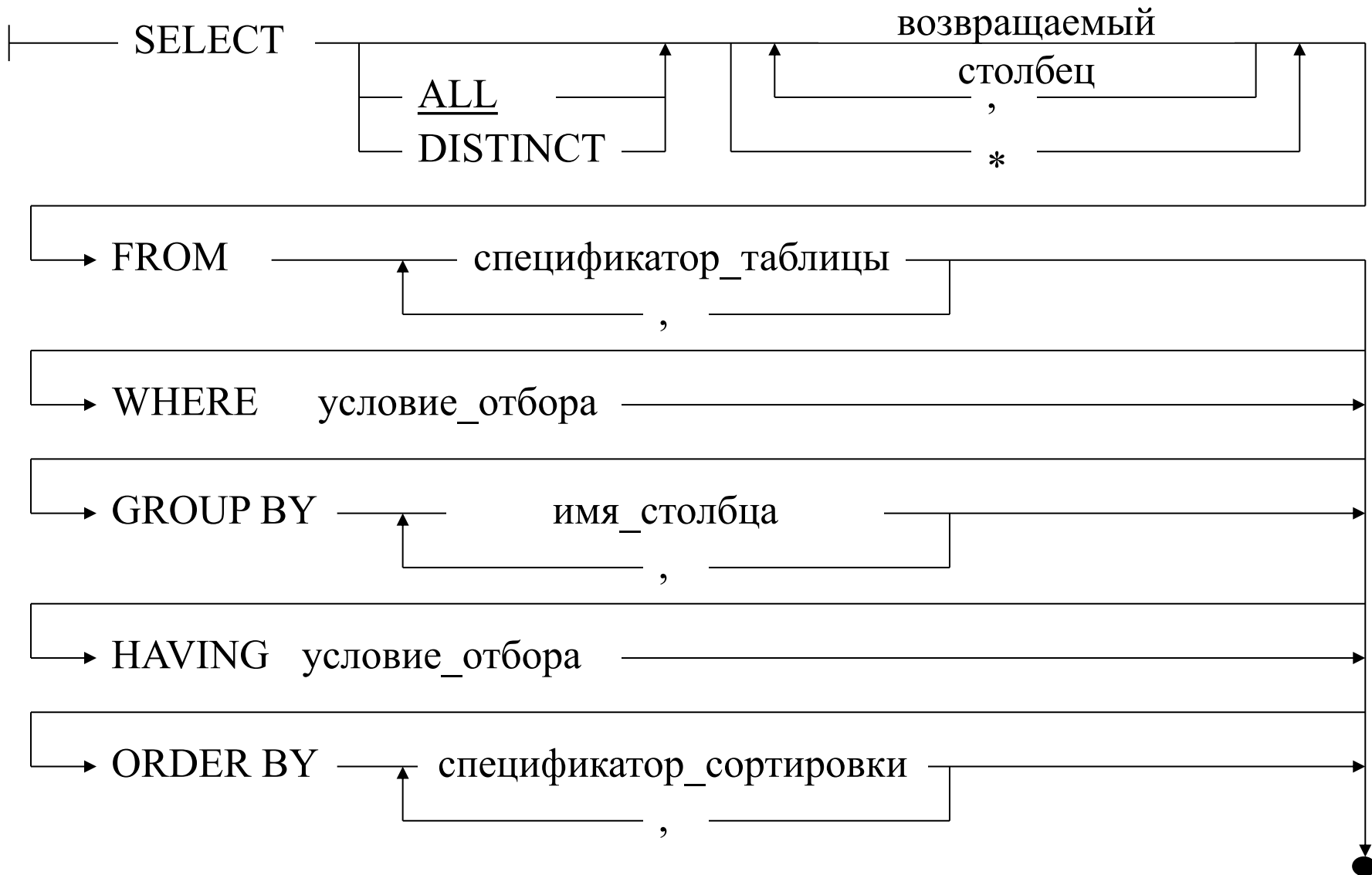
Тип данных	Описание
CHAR CHARACTER	Строки символов постоянной длины
VARCHAR CHAR VARYING CHARACTER VARYING	Строки символов переменной длины
NCHAR NATIONAL CHAR NATIONAL CHARACTER	Строки локализованных символов постоянной длины
NCHAR VARYING NATIONAL CHAR VARYING NATIONAL CHARACTER VARYING	Строки локализованных символов переменной длины
INTEGER INT	Целые числа
SMALLINT	Малые целые числа
BIT	Цепочки битов постоянной длины
BIT VARYING	Цепочки битов переменной длины
NUMERIC	DECIMAL DEC
FLOAT	Числа с плавающей запятой
REAL	Числа с плавающей запятой низкой точности
DOUBLE PRECISION	Числа с плавающей запятой высокой точности
DATE	Дата
TIME	Время
TIMESTAMP	Дата и время
INTERVAL	Временной интервал

Основы SQL

Функция	Возвращаемые данные
BIT_LENGTH (строка)	Количество битов в цепочке
CAST (значение AS тип данных)	Значение, преобразованное в указанный тип данных (например, дата, преобразованная в строку)
CHAR_LENGTH (строка)	Длина строки символов
CONVERT (строка USING функция)	Строка, преобразованная в соответствии с указанной функцией
CURRENT_DATE	Текущая дата
CURRENT_TIME (точность)	Текущее время с указанной точностью
CURRENT_TIMESTAMP (точность)	Текущие дата и время с указанной точностью
EXTRACT (часть FROM значение)	Указанная часть (DAY, HOUR и т. д.) из значения типа DATETIME
LOWER (строка)	Строка, переведенная в нижний регистр
OCTET_LENGTH (строка)	Число байтов в строке символов
POSITION (подстрока IN строка)	Позиция, с которой начинается вхождение подстроки в строку
SUBSTRING (строка FROM n FOR длина)	Часть строки, начинающаяся с n-го символа и имеющая указанную длину
TRANSLATE (строка USING функция)	Строка, транслированная с помощью указанной функции
TRIM (BOTH символ FROM строка)	Строка, из которой удалены ведущие и хвостовые указанные символы
TRIM (LEADING символ FROM строка)	Строка, из которой удалены ведущие указанные символы
TRIM (TRAILING символ FROM строка)	Строка, из которой удалены хвостовые указанные символы
UPPER (строка)	Строка, переведенная в верхний регистр

Простые запросы на выборку

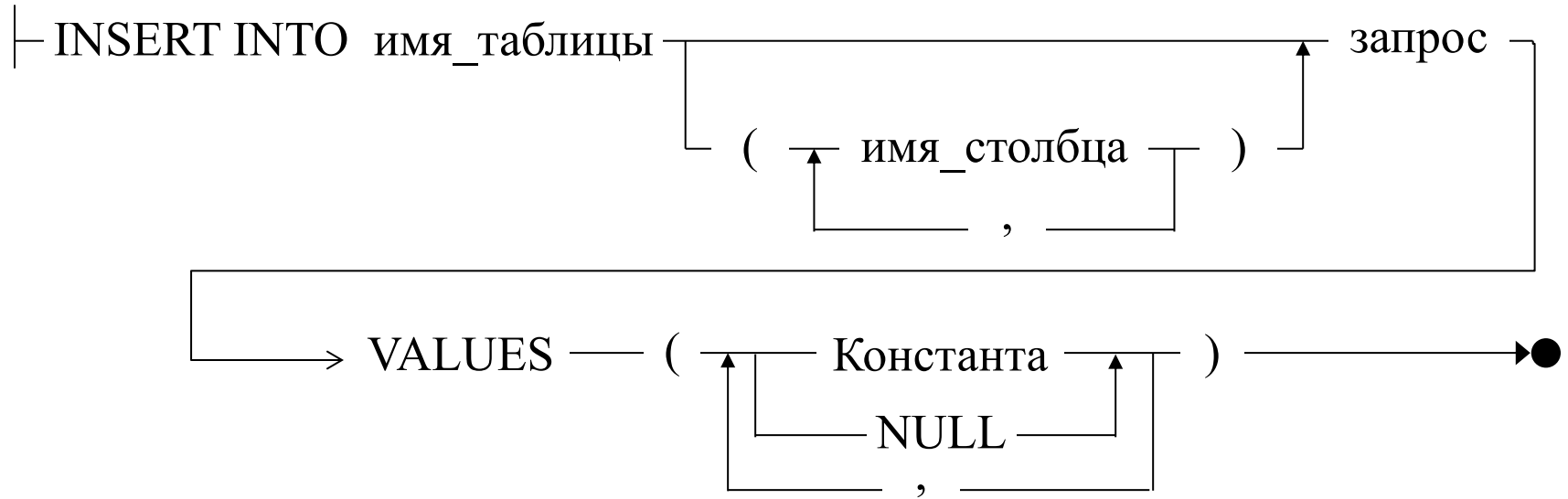
Оператор SELECT



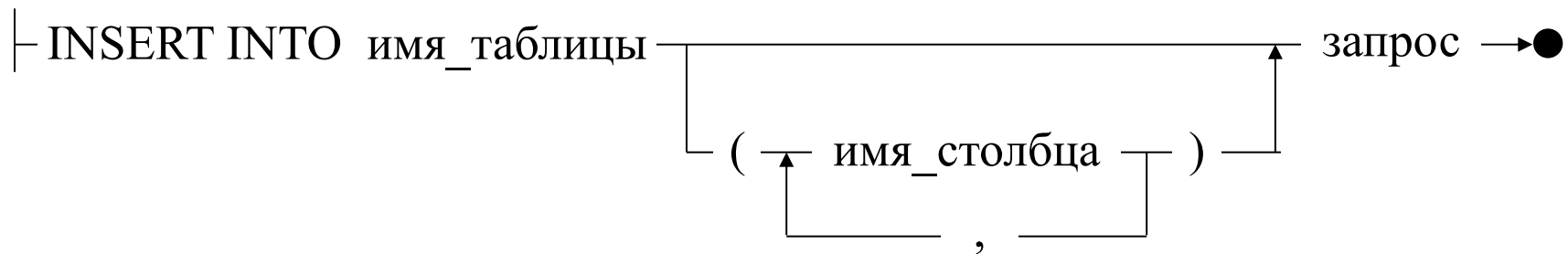
Изменения данных

Оператор INSERT

Однострочный оператор INSERT



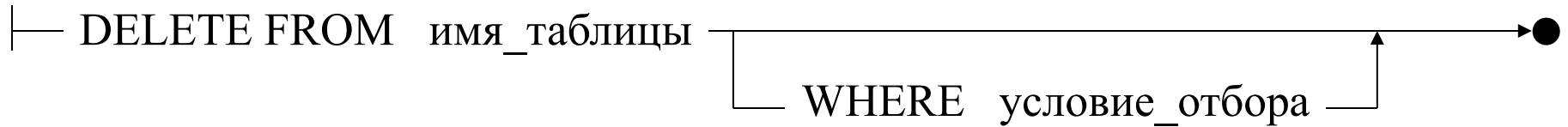
Многострочный оператор INSERT



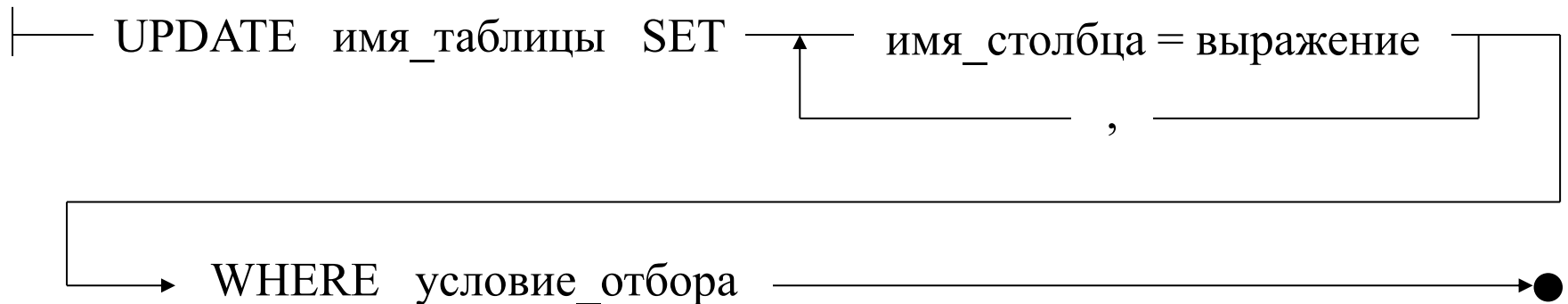
Изменения данных

Операторы DELETE UPDATE

Синтаксическая диаграмма оператора DELETE



Синтаксическая диаграмма оператора UPDATE



Управление транзакциями

Поддержка транзакций

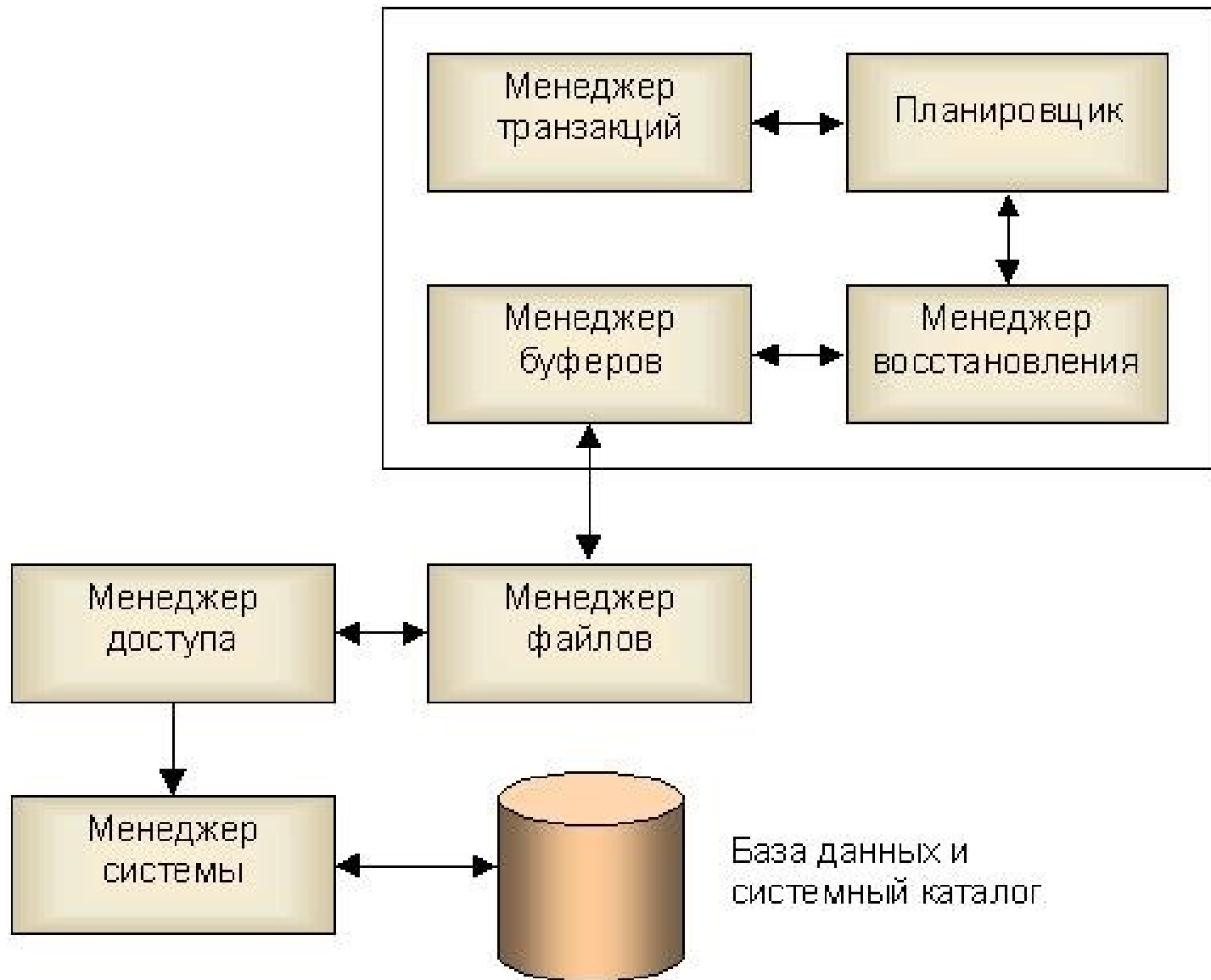
Транзакция – это действие или серия действий, выполняемых одним пользователем или прикладной программой, которые осуществляют доступ или изменение содержимого базы данных.

Свойства, которыми должна обладать любая из транзакций, так называемые ACID – аббревиатура:

- **атомарность** (*atomicity*). Это свойство типа “все или ничего”. Любая транзакция представляет собой неделимую единицу работы, которая может быть либо выполнена вся целиком, либо не выполнена вовсе;
- **согласованность** (*consistency*). Каждая транзакция должна переводить базу данных из одного согласованного состояния в другое согласованное состояние;
- **изолированность** (*isolation*). Все транзакции выполняются независимо одна от другой. Другими словами, промежуточные результаты незавершенной транзакции не должны быть доступны другим транзакциям;
- **продолжительность** (*durability*). Результаты успешно завершенной (зафиксированной) транзакции должны сохраняться в базе данных постоянно и не должны быть утеряны в результате последующих сбоев.

Управление транзакциями

Поддержка транзакций



Управление транзакциями

Управление параллельностью

Управление параллельностью – это процесс организации одновременного выполнения в базе данных различных операций, гарантирующий исключение их взаимного влияния друг на друга.

Пример проблемы потерянного обновления

Время	Транзакция T_1	Транзакция T_2	Поле bal_x
t_1		begin_transaction	100
t_2	begin_transaction	read(bal_x)	100
t_3	read(bal_x)	$bal_x = bal_x + 100$	100
t_4	$bal_x = bal_x - 10$	write(bal_x)	200
t_5	write(bal_x)	commit	90
t_6	commit		90

Управление транзакциями

Пример проблемы зависимости от нефиксированных результатов

Время	Транзакция T_3	Транзакция T_4	Поле bal_x
t_1		begin_transaction	100
t_2		read(bal_x)	100
t_3		$bal_x = bal_x + 100$	100
t_4	begin_transaction	write(bal_x)	200
t_5	read(bal_x)	...	200
t_6	$bal_x = bal_x - 10$	rollback	100
t_7	write(bal_x)		190
t_8	commit		190

Пример проблемы несогласованности обработки

Время	Транзакция T_5	Транзакция T_6	Поле bal_x	Поле bal_y	Поле bal_z	Поле sum
t_1		begin_transaction	100	50	25	
t_2	begin_transaction	sum = 0	100	50	25	0
t_3	read(bal_x)	read(bal_x)	100	50	25	0
t_4	$bal_x = bal_x - 10$	sum = sum + bal_x	100	50	25	100
t_5	write(bal_x)	read(bal_y)	90	50	25	100
t_6	read(bal_z)	sum = sum + bal_y	90	50	25	150
t_7	$bal_z = bal_z + 10$	...	90	50	25	150
t_8	write(bal_z)	...	90	50	35	150
t_9	commit	read(bal_z)	90	50	35	150
t_{10}		sum = sum + bal_z	90	50	35	185
t_{11}		commit	90	50	35	185

Управление транзакциями

Упорядочиваемость и восстанавливаемость

График – это последовательность запуска операций множества параллельно выполняемых транзакций, сохраняющая очередность выполнения операций в каждой отдельной транзакции.

Последовательный график – это график, в котором операции каждой из транзакций выполняются строго последовательно и не могут чередоваться с операциями, выполняемыми в других транзакциях.

Непоследовательный график – это график, в котором чередуются операции из некоторого набора одновременно выполняемых транзакций.

Упорядоченный график – это график, который приводит к получению тех же результатов, которые получаются при использовании некоторого последовательного графика в случае параллельного выполнения заданного множества транзакций

Управление транзакциями

Методы управления параллельностью

Блокировка – это процедура для управления параллельным доступом к данным. Когда некоторая транзакция получает доступ к базе данных, механизм блокировки позволяет (с целью исключения получения некорректных результатов) отклонить попытки получения доступа к этим же данным со стороны других транзакций.

Блокировки обычно используют следующие правила:

- любая транзакция, которой необходимо получить доступ к элементу данных, должна вначале выполнить блокировку этого объекта.
- если элемент еще не заблокирован какой-либо иной транзакцией, блокировка элемента будет выполнена успешно;
- если элемент данных в настоящий момент заблокирован, СУБД проанализирует, является ли тип полученного запроса совместимым с типом уже существующего блока. Если запрашивается доступ для чтения к элементу, который заблокирован для чтения, доступ к элементу данных будет разрешен. В противном случае транзакция будет переведена в состояние **ожидания**, которое будет продолжаться до тех пор, пока существующий блок не будет снят;
- транзакция продолжает удерживать блокировку элемента данных до тех пор, пока она явным образом не освободит его либо в ходе выполнения транзакции, либо по ее окончании (успешном или неуспешном).

Управление транзакциями

Методы управления параллельностью

Транзакция выполняется по протоколу **двухфазной блокировки**, если в ней все операции блокирования предшествуют первой операции разблокирования. *Используемые правила в протоколе:*

- прежде чем начать работу с элементом данных, транзакция должна выполнить его блокировку. Блокировка может устанавливаться для чтения или для записи в зависимости от типа требуемого доступа;
- как только транзакция освободит некоторый установленный ею блок, она уже не имеет право затребовать блокировки новых элементов.

Временная отметка – это уникальный идентификатор, создаваемый СУБД с целью обозначения относительного момента времени запуска транзакции.

Под **методом использования временных отметок** понимают протокол управления параллельностью, основная цель которого состоит в установлении глобальной очередности выполнения транзакций, при которой более старые транзакции (транзакции с меньшим значением временной отметки) имеют более высокий приоритет при разрешении возникающих конфликтов.

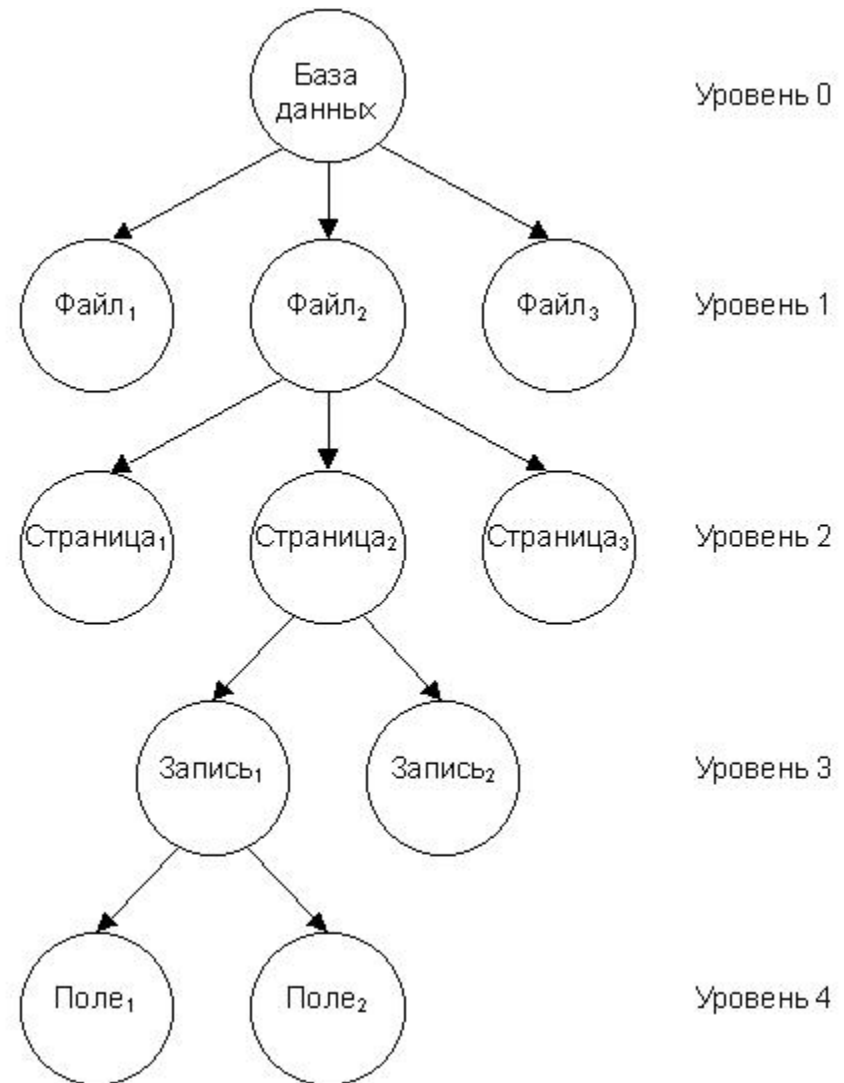
Управление транзакциями

Уровень детализации блокируемых элементов данных

Уровень детализации – это размер элементов данных, выбранных в качестве защищаемой единицы для протокола управления параллельностью.

Блокируемые элементы данных:

- вся база данных;
- отдельный файл;
- отдельная страница данных
- отдельная запись;
- отдельное поле в записи.



Восстановление базы данных

Восстановление базы данных – это процесс возвращения базы данных в корректное состояние, утраченное в результате сбоя или отказа.

Причины, способные вызвать отказы:

- ❖ ***аварийное прекращение работы системы***, вызванное ошибкой оборудования или программного обеспечения, приведшей к разрушению содержимого оперативной памяти;
- ❖ ***отказ носителей информации***, например, разрушение магнитной головки или появление неустранимого сбоя чтения, имеющее следствием потерю части вторичной памяти системы;
- ❖ ***ошибка прикладных программ***, например, логические ошибки в программах, получающих доступ к базе данных, послужившие причиной сбоев при выполнении одной или нескольких транзакций;
- ❖ ***стихийные бедствия***, например, пожары, наводнения, землетрясения или отказы в сети электропитания;
- ❖ ***небрежное или легкомысленное отношение***, послужившее причиной непреднамеренного разрушения данных или программ со стороны оператора или пользователей системы;
- ❖ ***диверсии*** и преднамеренное разрушение или уничтожение данных, оборудования или программного обеспечения.

Восстановление базы данных

Транзакции и восстановление

Транзакции представляют собой основную *единицу восстановления* в системах с базами данных.

При выполнении *операций чтения* СУБД осуществляет следующие действия:

- определяет дисковый адрес блока данных, содержащего запись с заданным значением первичного ключа;
- считывает блок данных с диска и помещает его в буфер СУБД в оперативной памяти;
- копирует сведения заданного поля из буфера СУБД в элемент данных.

При выполнении *операции записи* СУБД осуществляет следующие действия:

- определяет дисковый адрес блока данных, содержащего запись с заданным значением первичного ключа;
- считывает блок данных с диска и помещает его в буфер СУБД в оперативной памяти;
- копирует сведения из элемента данных в буфер СУБД;
- выводит блок данных из буфера СУБД в оперативной памяти на диск.

Восстановление базы данных

Функции восстановления

СУБД должна предоставлять такие функции восстановления, как:

- механизм резервного копирования, предназначенный для периодического создания копий базы данных;
- средства ведения журнала, в котором фиксируются текущее состояние транзакций и вносимые в базу данных изменения;
- функция создания контрольных точек, обеспечивающая перенос выполняемых в базе данных изменений во вторичную память с целью сделать их постоянными;
- менеджер восстановления, обеспечивающий восстановление согласованного состояния базы данных, нарушенного в результате отказа.

В файл журнала может помещаться следующая информация:

- **Записи о транзакциях** (идентификатор транзакции, тип записи журнала, идентификатор элемента данных, **копию** элемента данных **до** операции, **копию** элемента данных **после** операции, Служебную информацию файла журнала)
- **Записи контрольных точек.**

Восстановление базы данных

Методы восстановления

- Базе данных нанесены обширные повреждения, то потребуется восстановить ее последнюю резервную копию, после чего повторить в ней все выполненные транзакции, сведения о которых присутствуют в журнале регистрации. Безусловно предполагается, что файл журнала не был поврежден;
- База данных не получила физических повреждений, но лишь утратила согласованность размещенных в ней данных (например, из-за аварийного останова системы в процессе обработки транзакций), то достаточно будет отменить те изменения, которые вызвали переход базы данных в несогласованное состояние. Возможно потребуется повторно прогнать некоторые транзакции, чтобы иметь уверенность в том, что внесенные в них изменения действительно зафиксированы во вторичной памяти. В данном случае нет необходимости обращаться к резервной копии базы данных.

Метод восстановления с использованием отложенного обновления

Метод восстановления с использованием немедленного обновления

Метод теневых страниц

Безопасность баз данных

Аспекты безопасности баз данных

- правовые, общественные и этические аспекты (имеет ли право некоторое лицо получить запрашиваемую информацию, например, о кредите клиента);
- физические условия (например, закрыт ли данный компьютер или терминальная комната);
- организационные вопросы (например, как в рамках предприятия организован доступ к данным);
- вопросы реализации управления (например, если используется метод доступа по паролю, то как организована реализация управления и как часто меняются пароли);
- аппаратное обеспечение (обеспечиваются ли меры безопасности на аппаратном уровне, например, с помощью защитных ключей или привилегированного режима управления);
- безопасность операционной системы (например, затирает ли базовая операционная система содержание структуры хранения и файлов с данными при прекращении работы с ними);
- безопасность системы управления базами данных (например, существует ли для базы данных некоторая концепция предоставления прав владения данными).

Безопасность баз данных

Отличие подходов к обеспечению безопасности

- ❑ В случае **избирательного управления** пользователь обладает различными правами (**привилегиями** или **полномочиями**) при работе с разными объектами. Более того, различные пользователи обычно обладают и разными правами доступа к одному и тому же объекту. Таким образом, избирательные схемы характеризуются значительной гибкостью;
- ❑ В случае **обязательного управления**, наоборот, каждому объекту данных присваивается некоторый **классификационный уровень**, а каждый пользователь обладает некоторым **уровнем допуска**. Следовательно, при таком подходе доступом к определенному объекту данных обладают только пользователи с соответствующим уровнем допуска. Таким образом, обязательные схемы являются достаточно жесткими и статичными.

Безопасность баз данных

Избирательное управление доступом

При использовании простой модели безопасности баз данных необходимо поддерживать два функциональных принципа: **проверку полномочий** (**санкционирование**) и **проверку подлинности** (**аутентификация**).

❑ **Проверка полномочий** основана на том, что для каждого пользователя или процесса информационной системы устанавливается набор санкционированных действий, которые он может выполнять по отношению к определенным объектам.

❑ **Проверка подлинности** означает достоверное подтверждение того, что пользователь или процесс, выполняющий санкционированные действия, действительно является тем, за кого он себя выдает.

Информация хранящаяся в записи файла контрольного следа:

- – запрос (исходный текст запроса);
- – терминал, с которого была вызвана операция;
- – пользователь, задавший операцию;
- – дата и время запуска операции;
- – вовлеченные в процесс исполнения операции базовые отношения, кортежи и атрибуты;

Безопасность баз данных

Обязательное управление доступом

избирательная защита. Класс С делится на два подкласса – С1 и С2 , которые поддерживают избирательное управление доступом. Согласно требованиям для:

- **класса С1** необходимо разделение данных и пользователя, т. е. наряду с поддержкой взаимного доступа к данным здесь необходимо организовать раздельное использование данных пользователями; согласно требованиям класса
- **класса С2** необходимо дополнительно организовать учет на основе процедур входа в систему, аудита и изоляции ресурсов;

обязательная защита. Класс В содержит требования к методам обязательного управления доступом и делится на подклассы В1, В2 и В3. Согласно требованиям для:

- **класса В1** необходимо обеспечить “отмеченную защиту” (это означает, что каждый объект данных должен содержать отметку о его уровне классификации, например, секретно, для служебного пользования и т. д.), а также неформальное сообщение о действующей стратегии безопасности;
- **класса В2** необходимо дополнительно обеспечить формальное утверждение о действующей стратегии безопасности, а также обнаружить и исключить плохо защищенные каналы передачи данных;
- **класса В3** необходимо дополнительно обеспечить поддержку аудита и восстановления данных, а также назначение администратора режима безопасности;

• **проверенная защита.** Класс А является наиболее безопасным и согласно

- необходимо математическое доказательство того, что данный метод обеспечения безопасности совместим и адекватен заданной стратегии безопасности.

Безопасность баз данных

Шифрование данных

Шифрование – это кодирование данных с использованием специального алгоритма, в результате чего данные становятся недоступными для чтения любой программой, не имеющей ключа дешифрования.

Компоненты системы шифрования

- ☐ ключ шифрования, предназначенный для шифрования исходных данных (обычного текста);
- ☐ алгоритм шифрования, который описывает, как с помощью ключа шифрования преобразовать обычный текст в шифротекст;
- ☐ ключ дешифрования, предназначенный для дешифрования шифротекста;
- ☐ алгоритм дешифрования, который описывает, как с помощью ключа дешифрования преобразовать шифротекст в исходный обычный текст.

Управление распределенной информацией

Основные концепции

Распределенная база данных – это набор логически связанных между собой разделяемых данных (и их описаний), которые физически распределены в некоторой компьютерной сети.

Распределенная СУБД – это программный комплекс, предназначенный для управления распределенными базами данных и позволяющий сделать распределенность информации прозрачной для конечного пользователя.

СУРБД должна иметь следующие особенности:

- ☐ Набор логически связанных разделяемых данных;
- ☐ Сохраняемые данные должны быть разбиты на некоторое количество фрагментов;
- ☐ Между фрагментами может быть организована репликация данных;
- ☐ Фрагменты и их реплики распределены по различным сайтам;
- ☐ Сайты связаны между собой сетевыми соединениями;
- ☐ Работа с данными на каждом сайте управляется СУБД;
- ☐ СУБД на каждом сайте способна поддерживать автономную работу локальных приложений;
- ☐ СУБД каждого сайта поддерживает хотя бы одно глобальное приложение.

Управление распределенной информацией

Правила для типичной СУРБД

Правило 0. Основной принцип

С точки зрения конечного пользователя распределенная система должна выглядеть в точности так, как и обычная, нераспределенная система.

Правило 1. Локальная автономность

Сайты в распределенной системе должны быть автономными. В данном контексте автономность означает следующее:

- локальные данные принадлежат локальным владельцам и сопровождаются локально;
- все локальные процессы остаются чисто локальными;
- все процессы на заданном сайте контролируются только этим сайтом.

Правило 2. Отсутствие опоры на центральный сайт

В системе не должно быть ни одного сайта, без которого система не сможет функционировать. Это означает, что в системе не должно существовать центральных серверов таких служб, как управление транзакциями, выявление взаимных блокировок, оптимизация запросов и управление глобальным системным каталогом.

Управление распределенной информацией

Правила для типичной СУРБД

Правило 3. Локальная автономность

В идеале в системе никогда не должна возникать потребность в плановом останове ее функционирования для выполнения таких операций, как:

- добавление или удаление сайта из системы;
- динамическое создание или удаление фрагментов из одного или нескольких сайтов.

Правило 4. Независимость от расположения

Независимость от расположения эквивалентна прозрачности расположения. Пользователь должен получать доступ к базе данных с любого из сайтов. Более того, пользователь должен получать доступ к любым данным так, как если бы они хранились на его сайте независимо от того, где они физически сохраняются.

Правило 5. Независимость от фрагментации

Пользователь должен получать доступ к данным независимо от способа их фрагментации.

Управление распределенной информацией

Правила для типичной СУРБД

Правило 6. Независимость от репликации

Пользователь не должен нуждаться в сведениях о наличии репликации данных. Это значит, пользователь не будет иметь средств для получения прямого доступа к конкретной копии элемента данных, а также не должен заботиться об обновлении всех имеющихся копий элемента данных.

Правило 7. Обработка распределенных запросов

Система должна поддерживать обработку запросов, ссылающихся на данные, расположенные на более чем одном сайте.

Правило 8. Обработка распределенных транзакций

Система должна поддерживать выполнение транзакций как единицы восстановления. Система должны гарантировать, что выполнение как глобальных, так и локальных транзакций будет происходить с сохранением четырех основных свойств транзакций, а именно: атомарности, согласованности, изолированности и продолжительности.

Управление распределенной информацией

Правила для типичной СУРБД

Правило 9. Независимость от типа оборудования

СУРБД должна быть способна функционировать на оборудовании с различными вычислительными платформами.

Правило 10. Независимость от операционной системы

Прямым следствием предыдущего правила является требование, согласно которому СУРБД должна быть способна функционировать под управлением различных операционных систем.

Правило 11. Независимость от сетевой архитектуры

СУРБД должна быть способна функционировать в сетях с различной архитектурой и типами носителей.

Правило 12. Независимость от типа СУБД

СУРБД должна быть способна функционировать поверх различных локальных СУБД, возможно, с разным типом используемой модели данных (иными словами, СУРБД должна поддерживать гетерогенность).

Управление распределенной информацией

Виды систем

Распределенная обработка – это обработка с использованием централизованной базы данных, доступ к которой может осуществляться с различных компьютеров сети.

Параллельная СУБД – это система управления базой данных, функционирующая с использованием нескольких процессоров и устройств жестких дисков, что позволяет ей (если это возможно) распараллеливать выполнение некоторых операций с целью повышения общей производительности обработки.

Архитектуры параллельных СУБД:

- Системы с разделением памяти;
- Системы с разделением дисков;
- Системы без разделения.

Управление распределенной информацией

Виды систем

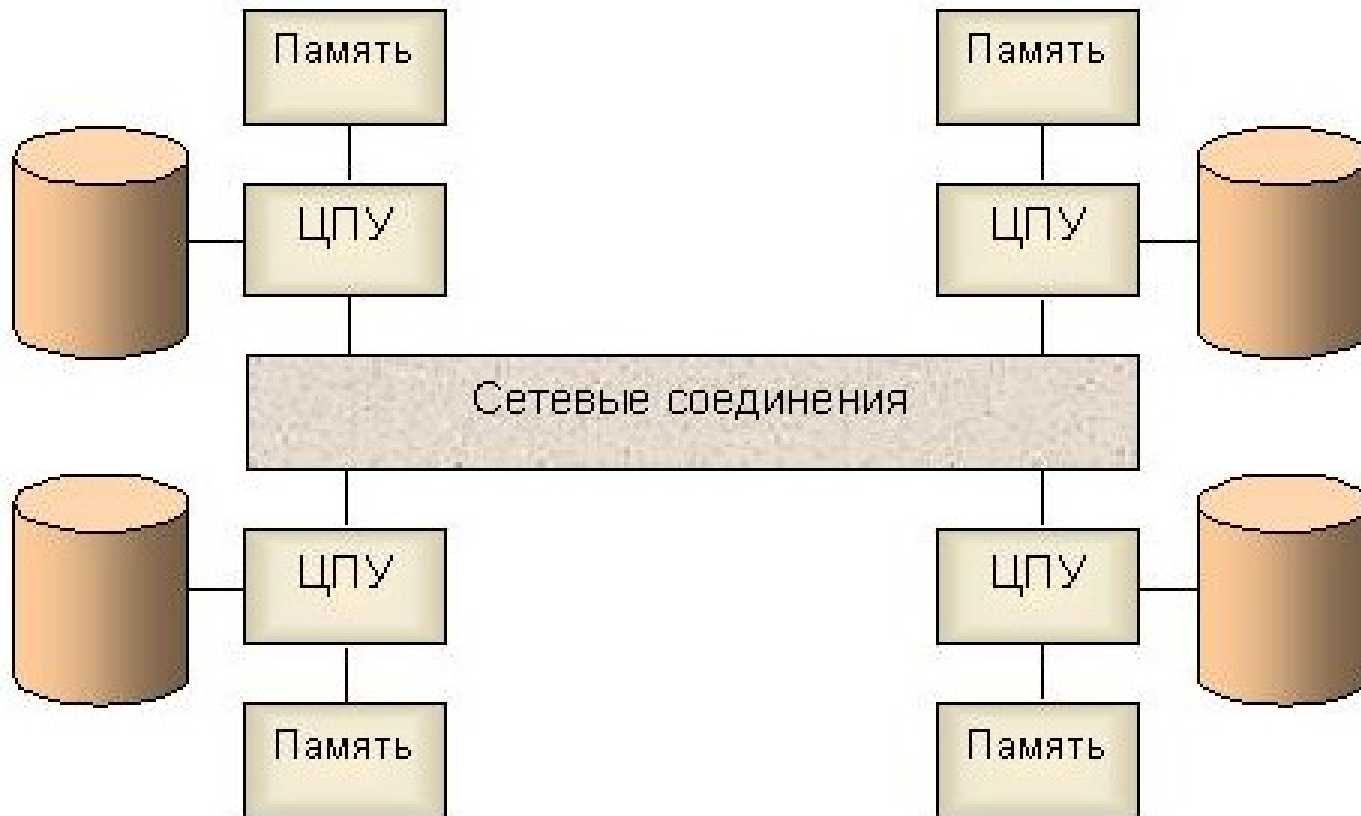
Системы с разделением памяти состоят из тесно связанных между собой компонентов, в число которых входит несколько процессоров, разделяющих общую системную память. Такая архитектура, называемая симметричной многопроцессорной обработкой (Symmetrical Multiple Processing – SMP)



Управление распределенной информацией

Виды систем

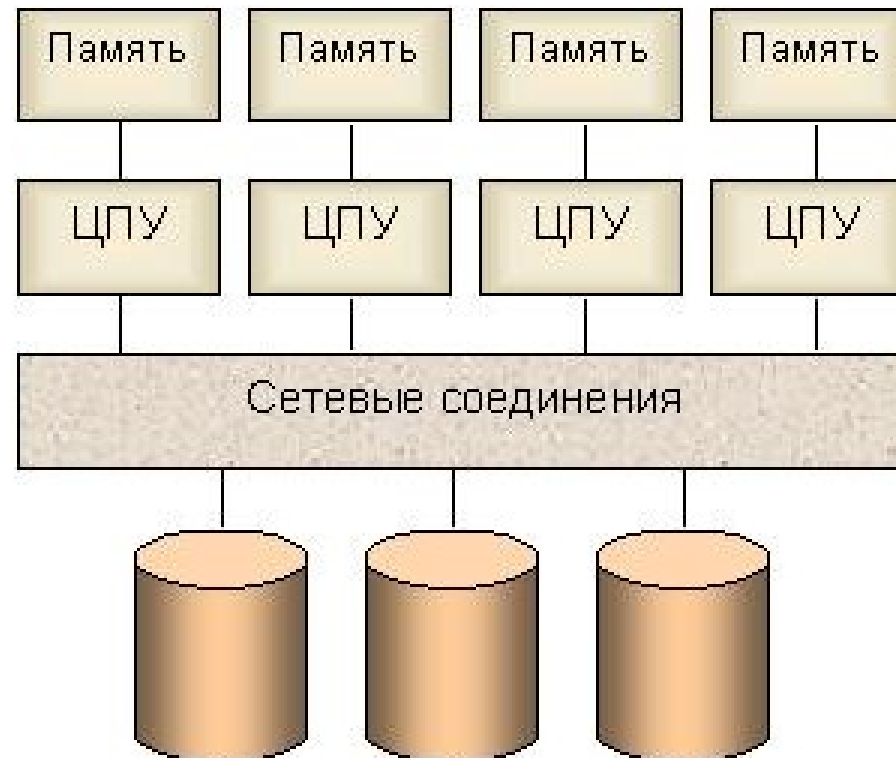
Системы без разделения, иначе называемые массовой параллельной обработкой (Massively Parallel Processing – MPP), используют схему, в которой каждый процессор, являющийся частью системы, имеет свою собственную оперативную и дисковую память.



Управление распределенной информацией

Виды систем

Системы с разделением дисков строятся из менее тесно связанных между собой компонентов. Они являются оптимальным вариантом для приложений, которые унаследовали высокую централизацию обработки и должны обеспечивать самые высокие показатели доступности и производительности.



Управление распределенной информацией

Преимущества и недостатки распределенных СУБД

Преимущества

- ☐ *Отражение структуры организации*
- ☐ *Разделяемость и локальная автономность*
- ☐ *Повышение доступности данных*
- ☐ *Повышение надежности*
- ☐ *Повышение производительности*
- ☐ *Экономические выгоды*
- ☐ *Модульность системы*

Недостатки

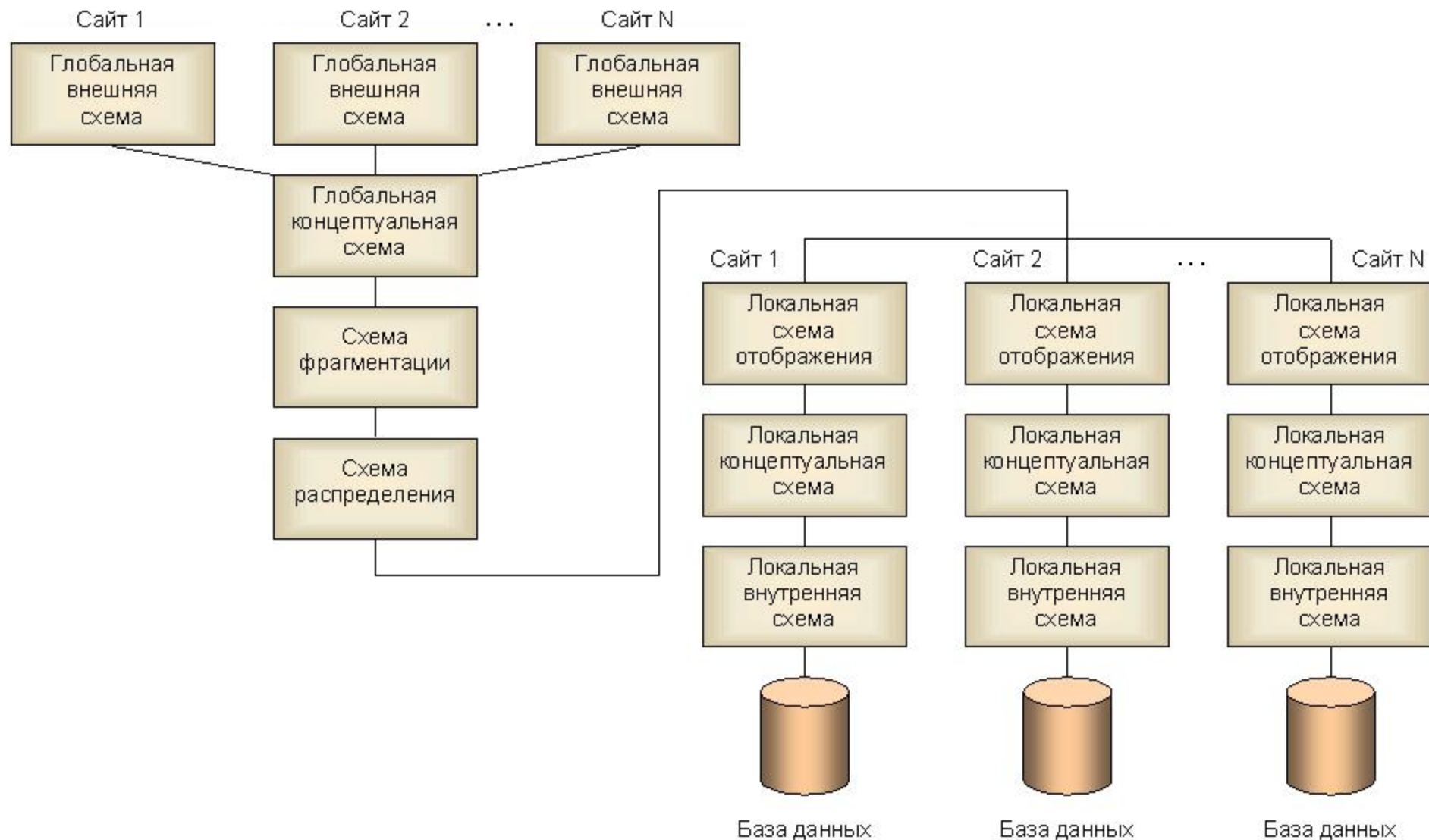
- ☐ *повышение сложности*
- ☐ *увеличение стоимости*
- ☐ *проблемы защиты*
- ☐ *усложнение контроля за целостностью данных*
- ☐ *отсутствие стандартов*
- ☐ *недостаток опыта*
- ☐ *усложнение процедуры разработки базы данных*

Функции распределенных СУБД

Типичная СУРБД должна обеспечивать, по крайней мере, тот же набор функциональных возможностей, который определен для централизованных СУБД. Кроме того, СУРБД должна предоставлять следующий набор функциональных возможностей:

- ☐ Расширенные службы установки соединений должны обеспечивать доступ к удаленным сайтам и позволять передавать запросы и данные между сайтами, входящими в сеть;
- ☐ Расширенные средства ведения каталога, позволяющие сохранять сведения о распределении данных в сети;
- ☐ Средства обработки распределенных запросов, включая механизмы оптимизации запросов и организации удаленного доступа;
- ☐ Расширенные функции управления параллельностью, позволяющие поддерживать целостность реплицируемых данных;
- ☐ Расширенные функции восстановления, учитывающие возможность отказов в работе отдельных сайтов и отказов линий связи.

Рекомендуемая архитектура распределенных СУБД

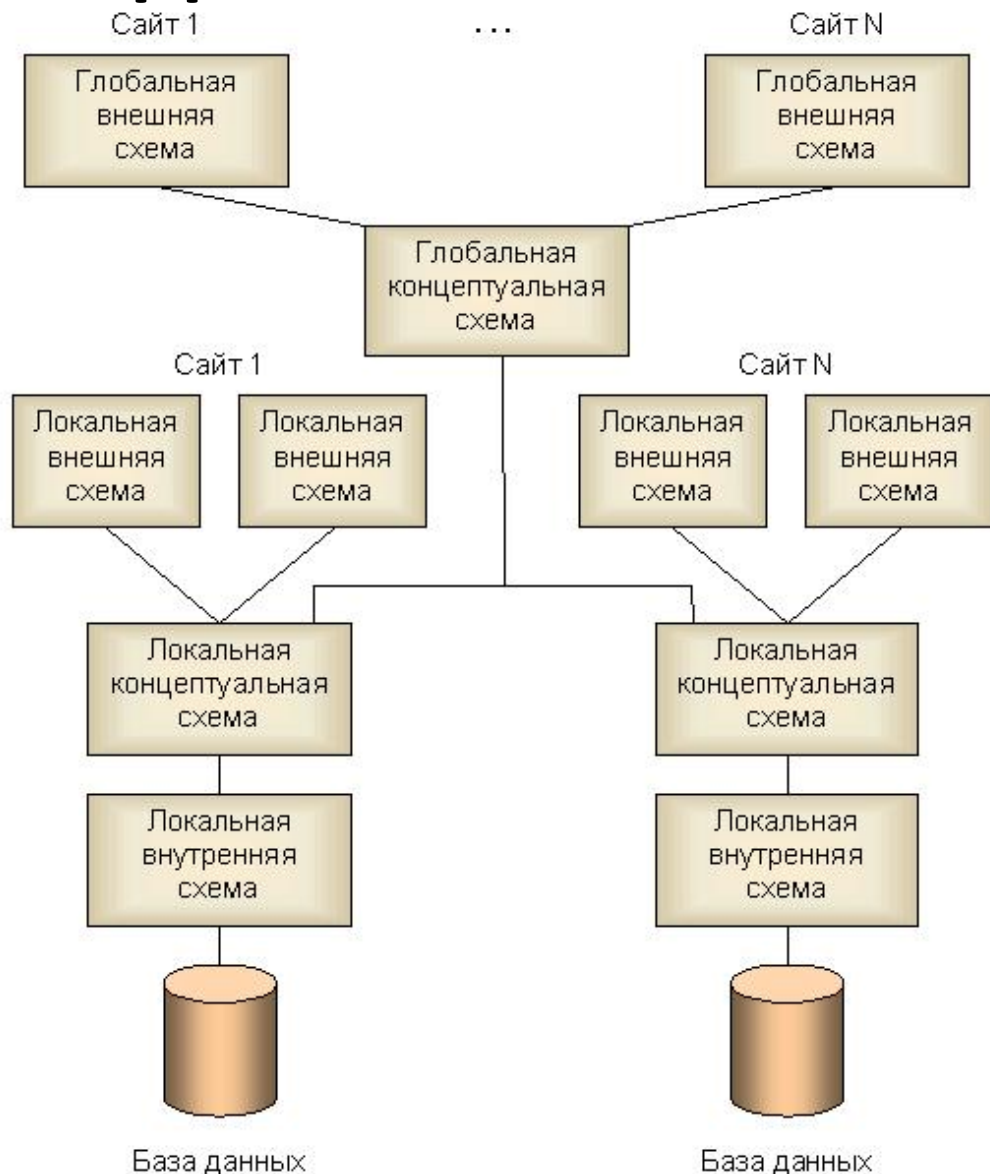


Рекомендуемая архитектура мультибазовых СУБД

Мультибазовая система – это распределенная система управления базами данных, в которой управление каждым из сайтов осуществляется совершенно автономно.

Виды мультибазовых систем:

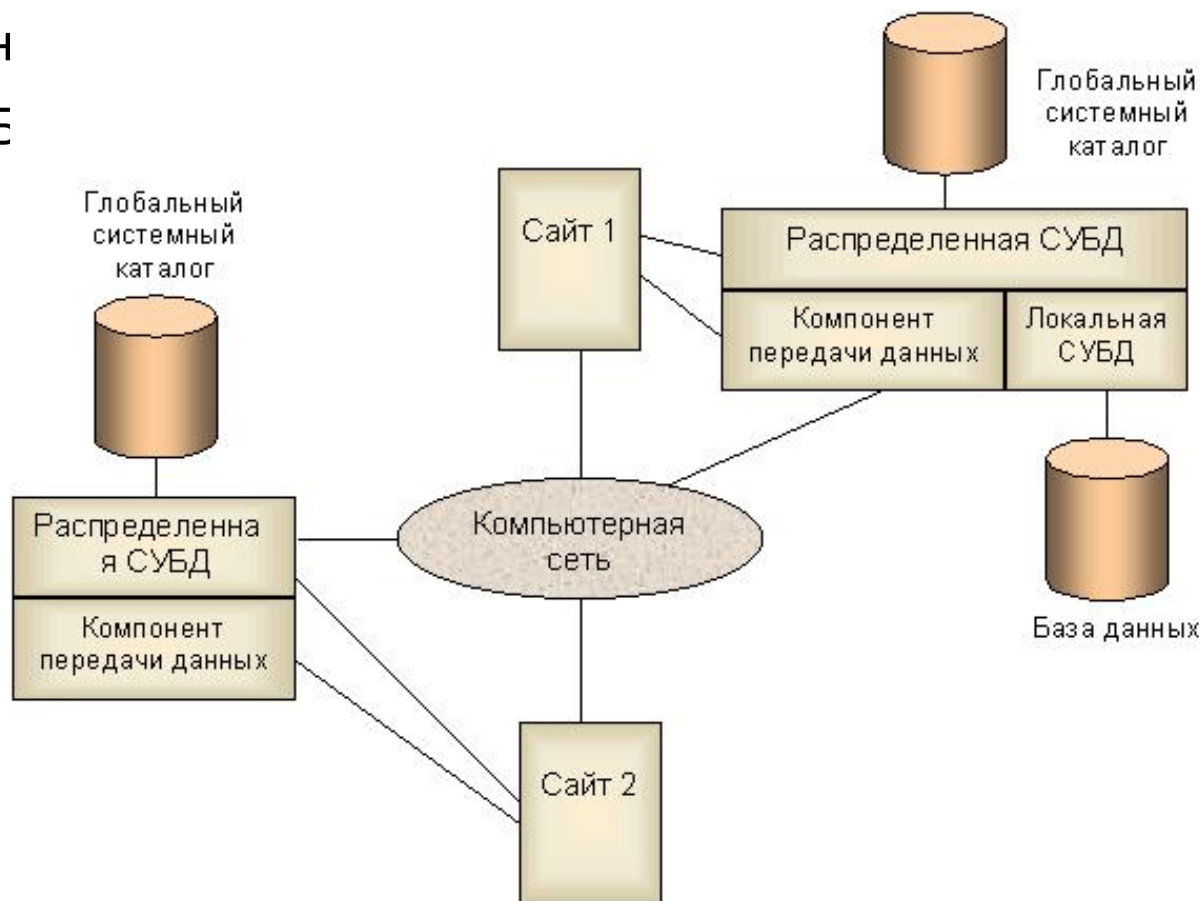
- ☐ **нефедеральные** – не имеющие локальных пользователей
- ☐ **федеральные** мультибазовые системы.



Компонентная архитектура распределенных СУБД

Основные компоненты:

- ☐ локальная СУБД;
- ☐ компонент передачи данных;
- ☐ глобальный системный каталог
- ☐ распределенная СУБД



Разработка распределенных реляционных баз данных

Аспекты проектирования распределенных систем, дополняющих методологию концептуального и логического проектирования централизованных баз данных:

Фрагментация. Любое отношение может быть разделено на некоторое количество частей, называемых фрагментами, которые затем распределяются по различным сайтам.

Основные типы фрагментов:

- ☐ **Горизонтальные фрагменты** представляют собой подмножества кортежей.
- ☐ **Вертикальные фрагменты** представляют собой подмножества атрибутов.

Распределение. Каждый фрагмент сохраняется на сайте, выбранном с учетом “оптимальной” схемы их размещения;

Репликация. СУРБД может поддерживать актуальную копию некоторого фрагмента на нескольких различных сайтах.

Обеспечение прозрачности в СУРБД

Под **прозрачностью** понимается сокрытие от пользователей деталей реализации системы.

основных типа прозрачности, которые могут иметь место в системе с распределенной базой данных:

- ☐ Прозрачность распределенности;
 - ☐ прозрачность фрагментации
 - ☐ прозрачность расположения
 - ☐ прозрачность репликации
 - ☐ прозрачность локального отображения
 - ☐ прозрачность именования
- ☐ Прозрачность транзакций;
 - ☐ прозрачность параллельности
 - ☐ прозрачностью отказов
- ☐ Прозрачность выполнения;
- ☐ Прозрачность использования СУБД

Специализированные приложения баз данных

Специализированные приложения, чьи требования сильно отличаются от требований традиционных бизнес-приложений и для которых использование реляционных СУБД становится неадекватным:

- автоматизированное проектирование;
- автоматизированное производство;
- автоматизированная разработка программного обеспечения;
- офисные информационные системы и системы мультимедиа;
- цифровое издательское дело;
- геоинформационные системы;
- научные и медицинские приложения;
- экспертные системы;
- приложения, использующих сложные и взаимосвязанные объекты и процедурные данные.

Недостатки реляционных СУБД

Наиболее значительные недостатки :

- слабое представление сущностей реального мира;
- семантическая перегрузка;
- слабая поддержка ограничений целостности и корпоративных ограничений;
- однородная структура данных;
- ограниченный набор операций;
- трудности организации рекурсивных запросов;
- проблема рассогласования;
- другие проблемы РСУБД, связанные с параллельностью, изменениями схемы и слабыми средствами доступа.

Основные концепции ООП

Базовые понятия

Абстракция – это процесс идентификации наиболее важных аспектов сущности и игнорирование всех остальных ее малозначащих свойств.

Понятие **инкапсуляции** означает, что объект содержит как структуру данных, так и набор операций, с помощью которых этой структурой можно манипулировать.

Концепция **сокрытия информации** означает, что все внешние аспекты объекта отделяются от подробностей его внутреннего устройства, скрытых от внешнего мира.

Объект – это уникально идентифицируемая сущность, которая содержит атрибуты, описывающие состояния объектов “реального мира”, и связанные с ними действия.

Состояние объекта описывается одним или несколькими атрибутами:

- ❑ **Простой атрибут** может иметь примитивный тип (целое число, строка, действительное число и т. д.)
- ❑ **Составной атрибут** может содержать коллекции и/или ссылки.
- ❑ **Ссылочный атрибут** представляет связь между объектами.

Основные концепции ООП

В объектно-ориентированной системе каждому объекту в момент его создания присваивается **идентификатор объекта** (Object Identifier – OID) который обладает следующими свойствами:

- генерируется системой;
- уникально обозначает этот объект;
- инвариантен в том смысле, что его нельзя изменить во время жизненного цикла программы;
- не зависит от значений его атрибутов (т. е. от текущего состояния, два объекта могут иметь одинаковое состояние, но всегда обладают разными OID-идентификаторами);
- скрыт от пользователя (в идеале).

Преимущества использования **OID**-идентификаторов в качестве идентификаторов объектов:

- ☐ *эффективность;*
- ☐ *быстрота;*
- ☐ *невозможность изменения пользователем;*
- ☐ *независимость от содержания данных.*

Основные концепции ООП

В объектной технологии функции обычно называются **методами**, которые защищают от внешнего мира расположенные внутри атрибуты. Методы определяют поведение объекта.

Сообщения являются средством взаимодействия объектов. Сообщение представляет собой запрос, направленный одним объектом (отправителем) в адрес другого объекта (получателя) и требующий, чтобы объект-получатель выполнил один из своих методов.

Классы играют роль некоего шаблона для определения набора подобных объектов.

Объекты некоторого класса называются его **экземплярами** (instance)

Наследование (inheritance) позволяет определять один класс на основе более общего класса

Процесс образования суперкласса называется **обобщением** (geheralization), а процесс образования подкласса – **специализацией**.

Перегрузка (overloading) позволяет повторно использовать имя метода внутри определения класса и определений нескольких классов.

Связывание (binding) – процесс выбора соответствующего метода, основанный на типе объекта.

Объектно-ориентированные СУБД

Объектно-ориентированная модель данных (ООМД) – это (логическая) модель данных, которая учитывает семантику объектов, применяемую в объектно-ориентированном программировании.

Объектно-ориентированная база данных (ООБД) – это перманентный, совместно используемый набор (коллекция) объектов, определенный средствами ООМД.

Объектно-ориентированной СУБД (ООСУБД) – это система управления (менеджер) ООБД.

Минимальная модель, которой обязательно должна удовлетворять любая ООСУБД:

1. Обеспечивать функциональность базы данных.
2. Поддерживать идентичность объектов.
3. Обеспечивать инкапсуляцию.
4. Поддерживать объекты со сложным состоянием

Объектно-ориентированные СУБД

Происхождение объектно-ориентированной модели данных



Перманентность

Перманентный язык программирования – это язык, который позволяет его пользователям (прозрачно) сохранять данные непосредственно при последовательном выполнении программы, после чего эти данные смогут использоваться и многими другими программами.

Перманентные (persistent) объекты – такие объекты, которые сохраняются даже после завершения того пользовательского сеанса или прикладной программы, во время которых они были созданы.

Временные (transient) объекты – объекты, которые сохраняются только на время работы программы

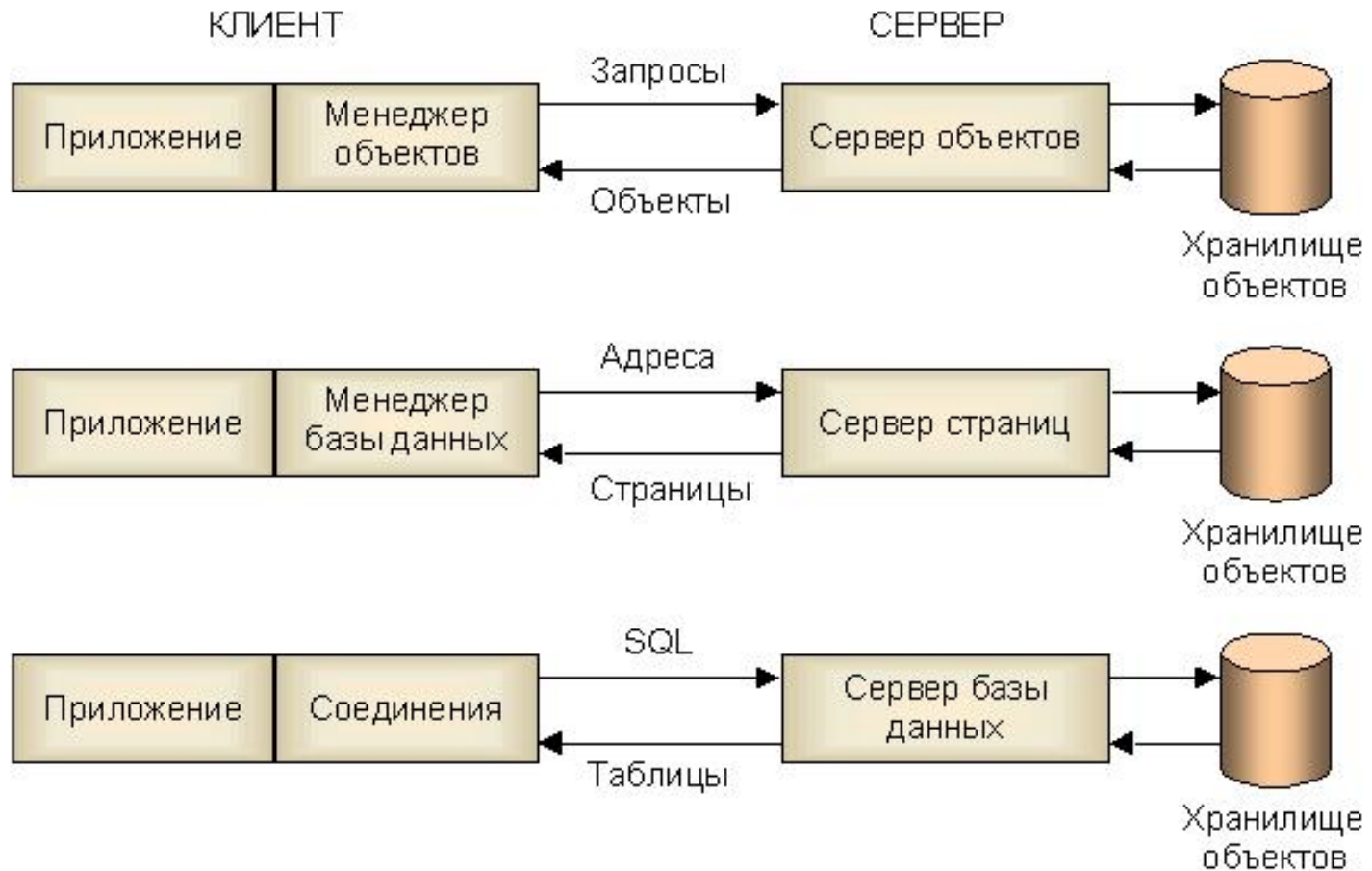
Схемы обеспечения перманентности

- ☐ Создание контрольных точек
- ☐ Сериализация
- ☐ Явная подкачка страниц

Объектно-ориентированные СУБД

Архитектура клиент/сервер

Три основные типа применяемых моделей в зависимости от функциональности их компонентов: **сервер объектов**, **сервер страниц**, **сервер базы данных**

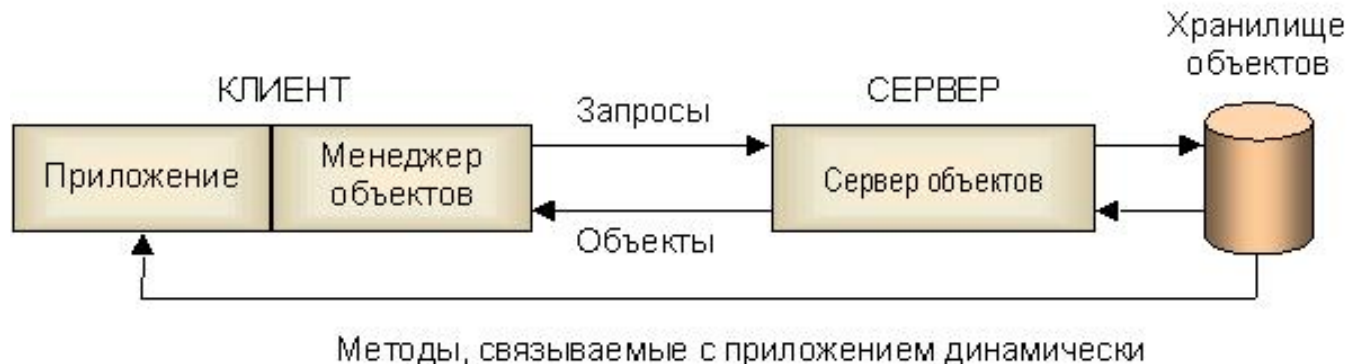
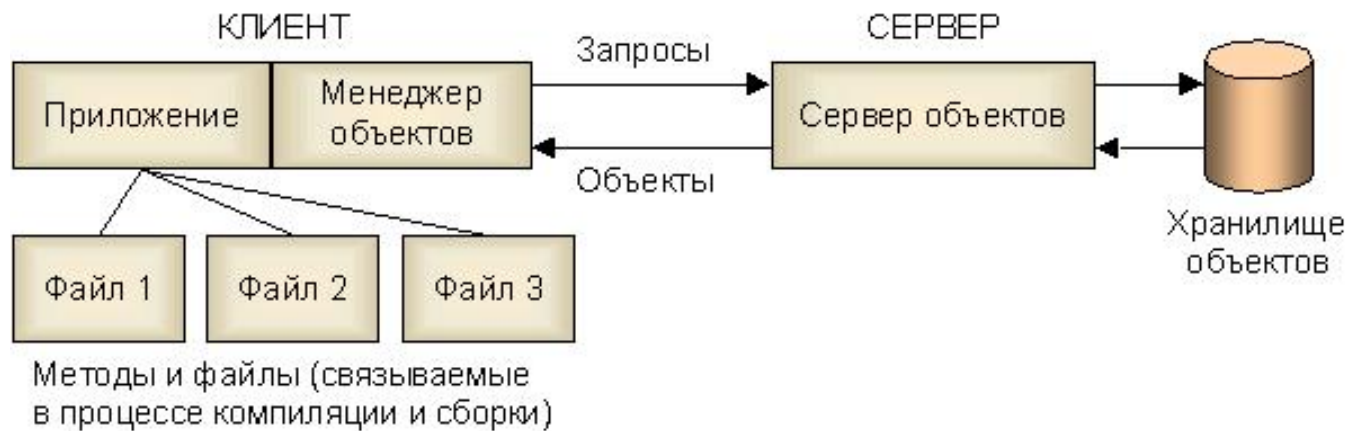


Объектно-ориентированные СУБД

Хранение и выполнение методов

Подходы для управления методами:

- хранение методов во внешних файлах
- хранение методов в базе данных



Объектно-ориентированные СУБД

Манифест объектно-ориентированных СУБД

1. *Поддержка составных объектов*
2. *Поддержка идентичности объектов*
3. *Поддержка инкапсуляции*
4. *Поддержка типов или классов*
5. *Поддержка наследования типов или классов от их предков*
6. *Поддержка динамического связывания*
7. *Язык DML должен обладать вычислительной полнотой*
8. *Набор типов данных должен быть расширяемым*
9. *Поддержка перманентности данных*
10. *Поддержка очень больших баз данных*
11. *Поддержка параллельной работы многих пользователей*
12. *Обеспечение восстановления после сбоев аппаратного и программного обеспечения*
13. *Предоставление простых способов создания запросов к данным*

Объектно-ориентированные СУБД

Преимущества ООСУБД

- *Улучшение возможности моделирования*
- *Расширяемость*
- *Устранение проблемы несоответствия*
- *Более выразительный язык запросов*
- *Поддержка эволюции схемы*
- *Поддержка долговременных транзакций*
- *Применимость для сложных специализированных приложений баз данных*
- *Повышенная производительность*

Объектно-ориентированные СУБД

Недостатки ООСУБД

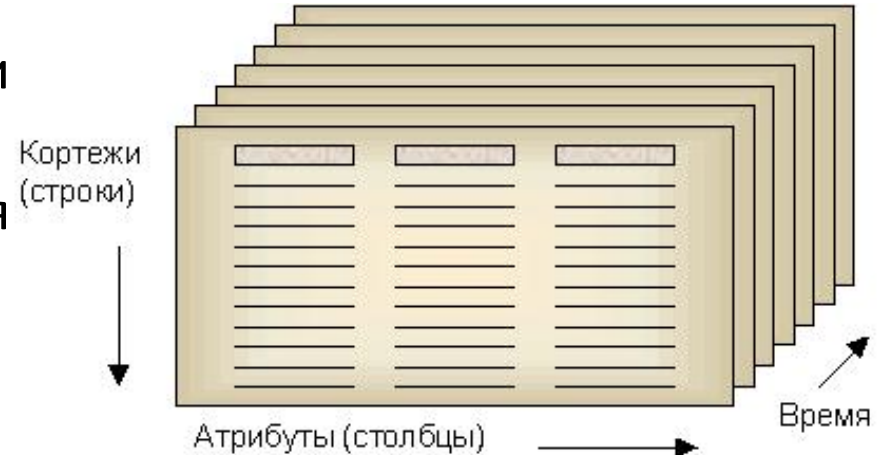
- *Отсутствие универсальной модели данных.*
- *Отсутствие стандартов.*
- *Влияние оптимизации запросов на инкапсуляцию.*
- *Влияние блокировки на уровне объекта на производительность.*
- *Сложность.*
- *Отсутствие поддержки представлений.*
- *Недостаточность средств обеспечения безопасности.*

Временные базы данных

Временные (или темпоральные) базы данных дополняют основные данные свойством времени

Понятия “времени” в приложении к СУБД предложенные Я. Бен-Зви:

- эффективное время (время вступления чего-либо в силу)
- время регистрации.



Эффективное время определяется как время, когда некоторый факт в действительности вступает в силу (теперь его называют “действительным временем”).

Время регистрации (называемое теперь “временем транзакции”) – это время, когда новое значение помещается в базу данных.

Временные базы данных

Временные модели данных

- ❑ Расширенная реляционная модель HRDM (Historical Relational Data Model)
- ❑ Временная реляционная модель TRM (Temporal Relational Model)

Отношение находится во **временной нормальной формы** TNF (Time Normal Form) тогда и только тогда, когда оно находится в нормальной форме Бойса-Кодда и не существует каких-либо временных зависимостей между неключевыми атрибутами”.

В модели HDBMS поддерживаются следующие правила для временных операций:

- разделение исторических данных, текущих данных и данных будущего на различные сегменты таким образом, чтобы оно было прозрачным для пользователя;
- поддержка всех типов транзакций над состояниями и сущностями будущего;
- отказ от наложения каких-либо ограничений, не являющихся необходимыми, на упорядочение во времени транзакций над данными будущего (например, основных принципов синхронизации транзакций).

Временные расширения языков баз данных

Язык TempSQL

Формат операторов SELECT в языке TempSQL:

```
SELECT {<целевой_список>}  
WHILE {<временное_выражение>}  
FROM {<список_отношений>}  
WHERE {<условие_поиска>}
```

Язык TempSQL позволяет включать во фразу **WHILE** подзапросы.

Язык TSQL

- Для оператора SELECT ... WHEN используется следующий синтаксис:

```
SELECT {<целевой_список>}  
FROM {<список_отношений>}  
WHERE {<условие_поиска>}  
WHEN {<временное_выражение>}
```

Активные базы данных

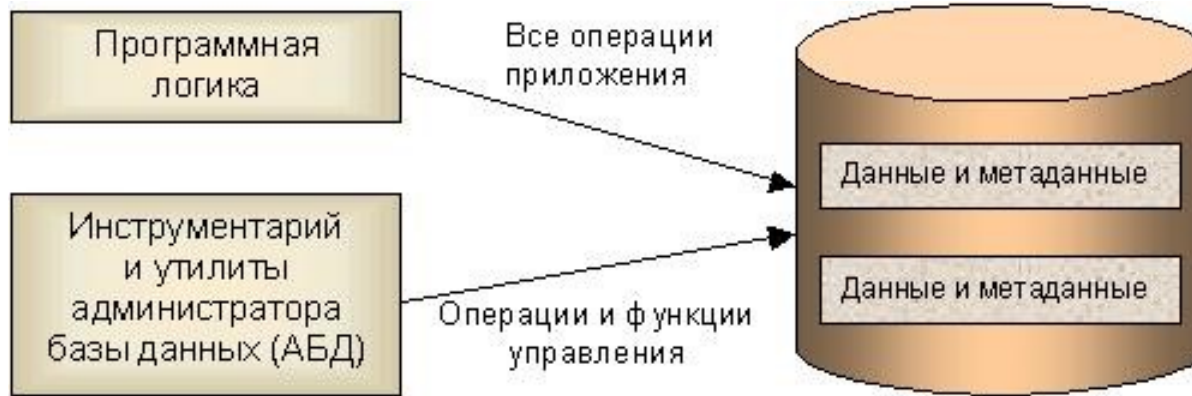
К терминологии активных баз данных относятся следующие понятия:

- ❑ *триггеры баз данных*, которые запускаются при наступлении предопределенного события (или комбинации событий);
- ❑ *хранимые процедуры*, встраивающие процедурную логику в среду базы данных, а не в подсистему приложения;
- ❑ *основанные на СУБД возможности мониторинга*, которые служат средствами управления активной природой таких псевдоинтеллектуальных баз данных, а также непосредственными действиями триггеров и хранимых процедур.

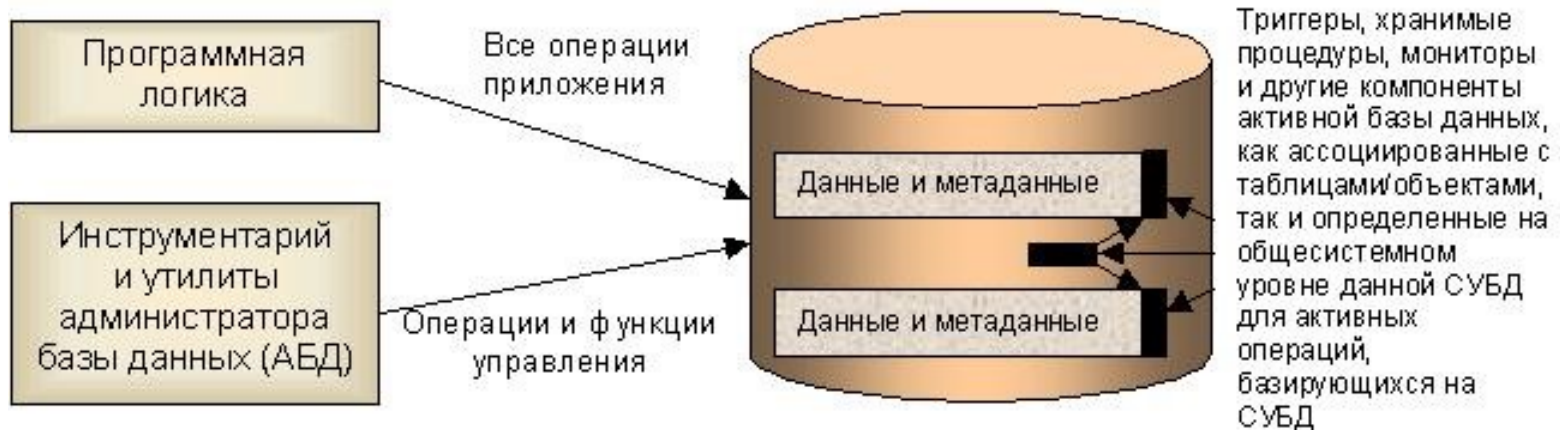
Преимущества использования активных баз данных:

- *Спецификация и поддержка бизнес-правил*
- *Разработка приложений*
- *Производительность функционирования*

Принципы активных систем баз данных



Пассивные базы данных



Активная система базы данных

При создании активных сред используются несколько основных конструкций базы данных: *ограничения*; *утверждения*; *хранимые процедуры*; *триггеры*;

Пространственные базы данных

К числу свойств геометрических отношений, которые должны быть сохранены в пространственной базе данных, относятся:

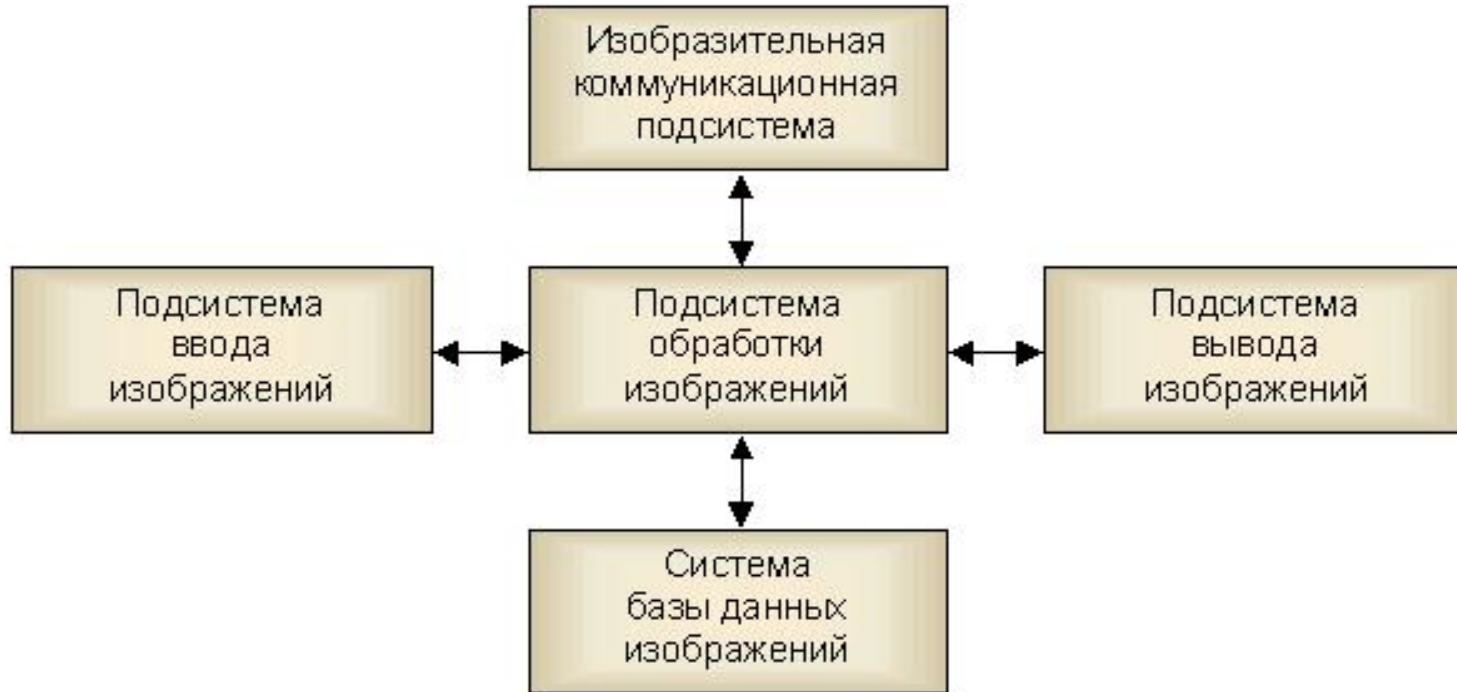
- *Соединение* (например, одного шоссе с другим);
- *Соседство* (например, какие районы соединяются данным шоссе);
- *Порядок* (например, линейная последовательность дорожных знаков вдоль шоссе);
- *Метрические отношения* (примерные расстояния, например, “близко”, “далеко” или “однодневный переезд”).

Дополнительные требования навигационных приложений:

- *Объемы данных;*
- *Доступ в режиме реального времени;*
- *Обработка запросов;*
- *Гранулярность*

Изобразительные информационные системы

Концептуальная архитектура изобразительной информационной системы



Система баз данных изображений выполняет следующие функции:

- *Анализ изображений и распознавание образов;*
- *Структурирование и понимание изображения;*
- *Пространственные рассуждения и выборка изобразительной информации*

Мультимедийные информационные системы

Типичная мультимедийная информационная система обычно оперирует следующими типами данных:

- “обычные” типы данных в базе данных (которые можно встретить в реляционной или сетевой базе данных);
- данные неподвижных изображений (как в фотографиях);
- другие типы графики;
- данные движущихся изображений (видео);
- аудио (голос, музыкальные звуки, звуки, издаваемые животными);
- текстовые данные, например, такие, которые можно найти в документах текстовых процессоров или файлах электронных таблиц.



Документальные информационно-поисковые системы

В теории документальных информационно-поисковых систем имеются фундаментальные понятия:

Пертинентность — соответствие смыслового содержания документа информационной потребности пользователя.

Релевантность — соответствие содержания документа информационному запросу в том виде, в каком он сформулирован, а документы, содержание которых отвечает запросу пользователя, называются *релевантными*.

Информационно-поисковым языком называется специализированный искусственный язык, предназначенный для описания основного смыслового содержания поступающих в систему сообщений, с целью обеспечения возможности последующего их поиска.

Критерия смыслового соответствия — набор правил определяющих степень смысловой близости между поисковыми образами документов и поисковыми предписаниями

Документальные информационно-поисковые системы

В теории документальных информационно-поисковых систем имеются фундаментальные понятия:

Пертинентность — соответствие смыслового содержания документа информационной потребности пользователя.

Релевантность — соответствие содержания документа информационному запросу в том виде, в каком он сформулирован, а документы, содержание которых отвечает запросу пользователя, называются *релевантными*.

Информационно-поисковым языком называется специализированный искусственный язык, предназначенный для описания основного смыслового содержания поступающих в систему сообщений, с целью обеспечения возможности последующего их поиска.

Критерия смыслового соответствия — набор правил определяющих степень смысловой близости между поисковыми образами документов и поисковыми предписаниями

Документальные информационно-поисковые системы

В теории документальных информационно-поисковых систем имеются фундаментальные понятия:

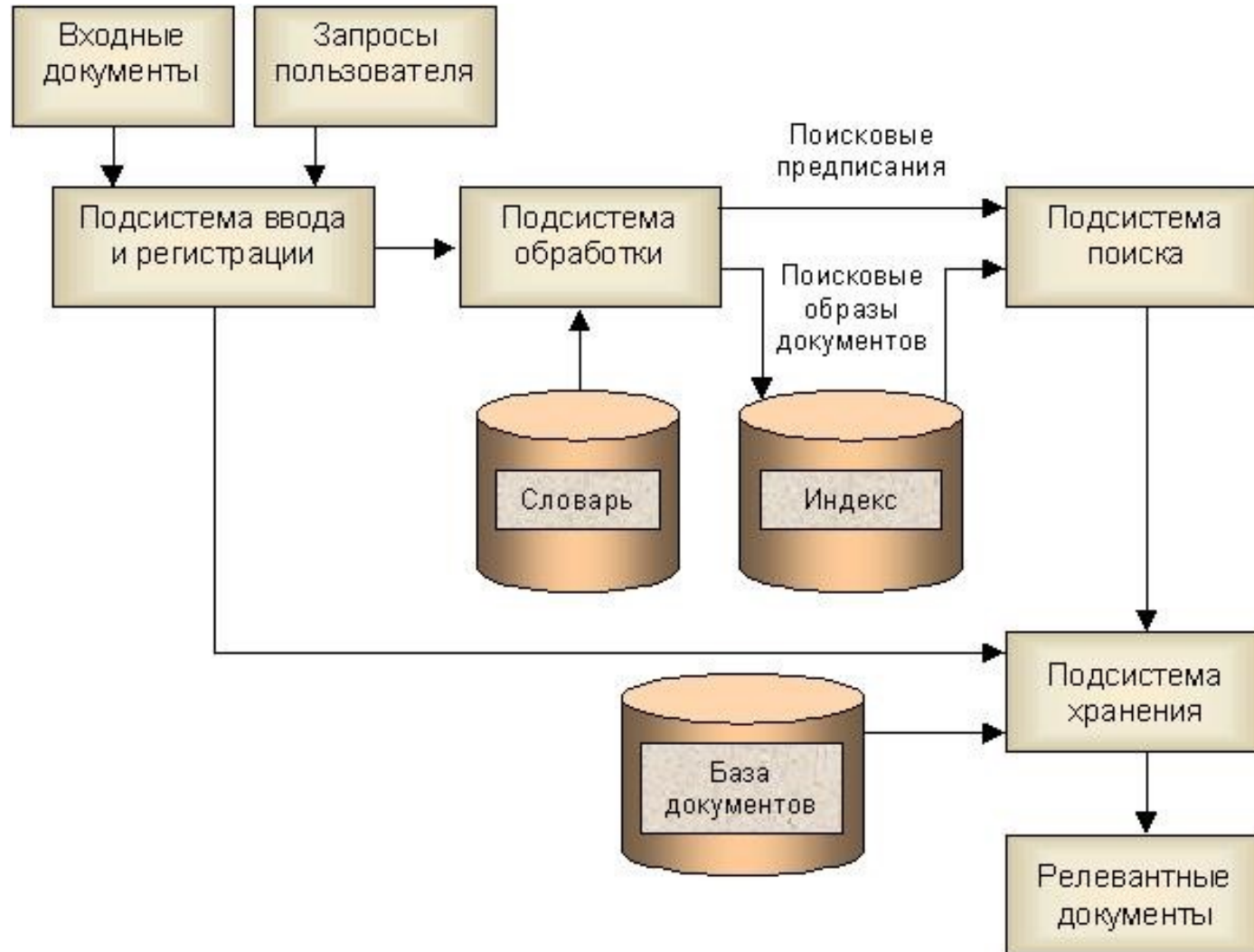
Пертинентность — соответствие смыслового содержания документа информационной потребности пользователя.

Релевантность — соответствие содержания документа информационному запросу в том виде, в каком он сформулирован, а документы, содержание которых отвечает запросу пользователя, называются *релевантными*.

Информационно-поисковым языком называется специализированный искусственный язык, предназначенный для описания основного смыслового содержания поступающих в систему сообщений, с целью обеспечения возможности последующего их поиска.

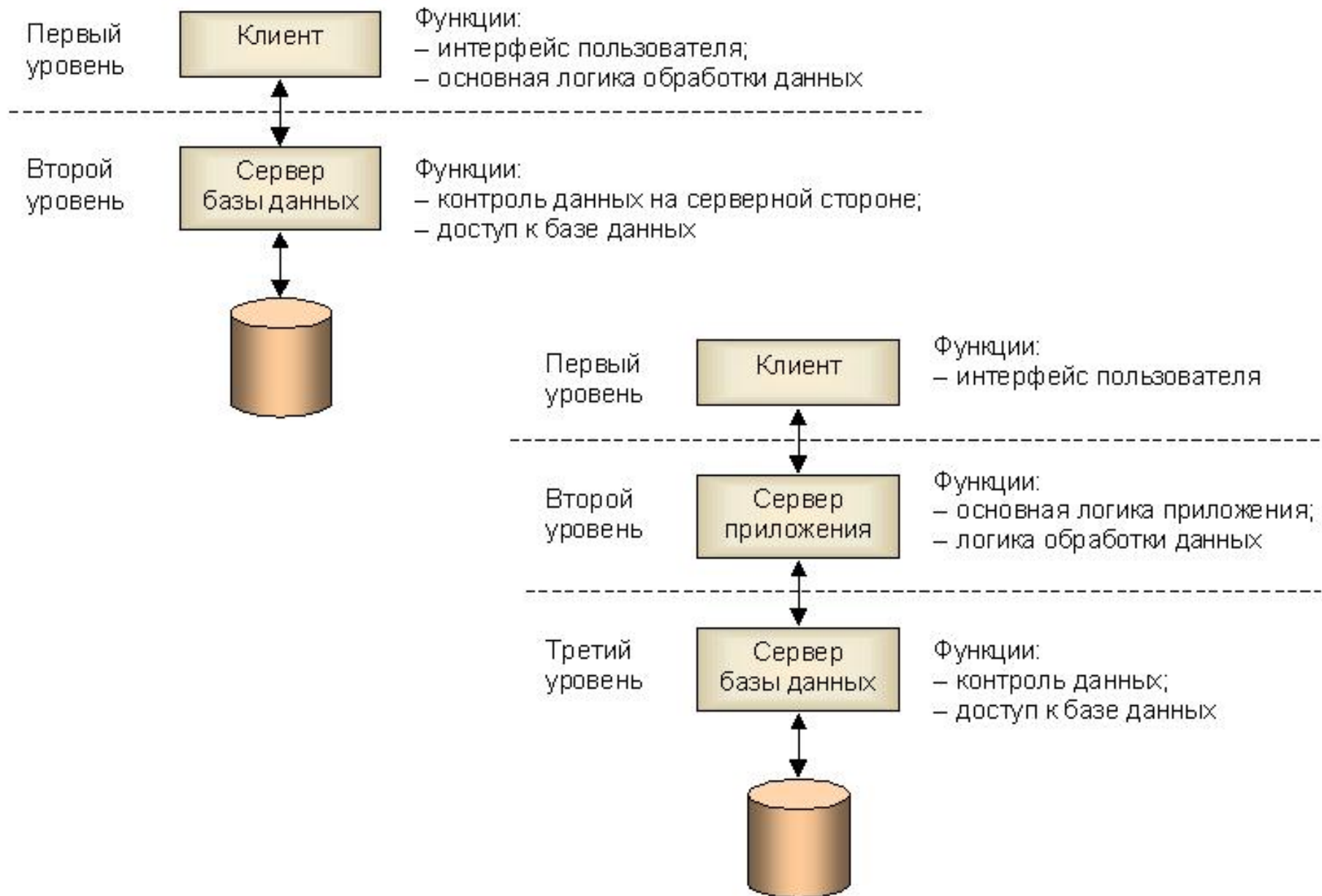
Критерия смыслового соответствия — набор правил определяющих степень смысловой близости между поисковыми образами документов и поисковыми предписаниями

Документальные информационно-поисковые системы



Общая структура документальной информационно-поисковой системы

Архитектура Web-СУБД



Преимущества и недостатки интеграции СУБД в среду Web

Преимущества	Недостатки
Преимущества использования функций СУБД	Недостаточная надежность
Простота реализации	Слабая защищенность
Независимость от платформы	Высокая стоимость
Графический интерфейс пользователя	Трудности с определением масштаба
Стандартизация	Ограниченная функциональность языка HTML
Межплатформенная поддержка	Отсутствие запоминания состояния
Прозрачный сетевой доступ	Высокие требования к пропускной способности сети
Масштабируемость развертывания	Недостаточная производительность
Инновационность	Несовершенство инструментов разработки